

BỘ CÔNG THƯƠNG
TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC
NGÀNH KHOA HỌC MÁY TÍNH

NGHIÊN CỨU HÀNH VI CHUYỂN ĐỘNG VÀ BỘ KHUNG TƯ THẾ
ỨNG DỤNG PHÁT TRIỂN HỆ THỐNG CẢNH BÁO TẾ NGÃ

CBHD: ThS. Đỗ Mạnh Quang
Sinh viên: Nguyễn Hữu Thắng
Mã số sinh viên: 2022601164

Hà Nội – Năm 2026

NGUYỄN HỮU THẮNG

KHOA HỌC MÁY TÍNH

BỘ CÔNG THƯƠNG
ĐẠI HỌC CÔNG NGHIỆP HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC
NGÀNH KHOA HỌC MÁY TÍNH

**NGHIÊN CỨU HÀNH VI CHUYỂN ĐỘNG VÀ BỘ KHUNG TƯ
THỂ ỨNG DỤNG PHÁT TRIỂN HỆ THỐNG CẢNH BÁO TẾ NGÃ**

CBHD: ThS. Đỗ Mạnh Quang

Sinh viên: Nguyễn Hữu Thắng

Mã số sinh viên: 2022601164

Hà Nội – Năm 2026

LỜI CẢM ƠN

Thực hiện một đề tài thực nghiệm không chỉ là quá trình vận dụng kiến thức lý thuyết vào thực tiễn, mà còn là hành trình khám phá chính mình – về tư duy, tinh thần làm việc và khả năng phối hợp nhóm. Và hành trình ấy của em sẽ khó có thể trọn vẹn nếu không có sự đồng hành và hỗ trợ từ rất nhiều người.

Đầu tiên, em xin gửi tới các thầy cô ở khoa Công nghệ thông tin, Trường Đại học Công nghiệp Hà Nội những lời chào trân trọng nhất, lời chúc sức khỏe và lòng biết ơn sâu sắc. Nhờ vào sự quan tâm, dạy dỗ và sự chỉ bảo tận tình của các thầy cô, em đã hoàn thành đề tài

Đặc biệt, em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến thầy Đỗ Mạnh Quang, người không chỉ truyền đạt kiến thức một cách cặn kẽ mà còn luôn khơi gợi cho em tinh thần học hỏi chủ động, sự tò mò sáng tạo và cách tư duy logic trong từng vấn đề kỹ thuật. Những góp ý tưởng chừng nhỏ nhưng lại rất "đắt giá" của thầy chính là kim chỉ nam để em điều chỉnh hướng tiếp cận, hiểu bài toán sâu sắc hơn và tìm ra hướng giải quyết phù hợp.

Dù kết quả còn khiêm tốn và không tránh khỏi thiếu sót, nhưng em tin rằng những trải nghiệm có được từ đề tài này sẽ là nền tảng quý báu cho những bước tiến vững chắc của em sau này.

Em xin chân thành cảm ơn!

Sinh viên thực hiện

Nguyễn Hữu Thắng

MỤC LỤC

LỜI CẢM ƠN	i
MỤC LỤC	ii
DANH MỤC CÁC TỪ VIẾT TẮT	v
DANH MỤC CÁC HÌNH ẢNH.....	viii
MỞ ĐẦU	1
CHƯƠNG 1. TỔNG QUAN VỀ BÀI TOÁN.....	4
1.1. Tổng quan về xử lý ảnh và thị giác máy tính	4
1.1.1. Khái niệm ảnh số và video số	4
1.1.2. Xử lý ảnh số.....	6
1.1.3. Thị giác máy tính	7
1.1.4. Bài toán nhận diện hành động	8
1.2. Bài toán phát hiện té ngã	10
1.2.1. Sự cấp thiết của bài toán.....	10
1.2.2. Các chiến lược phát hiện té ngã.....	11
1.3. Phát biểu bài toán của đề án	13
1.3.1. Mục tiêu của đề án	13
1.4. Thuận lợi và khó khăn.....	15
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT	16
2.1. Tổng quan về hệ thống đề xuất.....	16
2.1.1. Luồng xử lý dữ liệu của hệ thống	16
2.1.2. Biểu diễn dữ liệu bộ khung xương người.....	17
2.2. Kỹ thuật xử lý nhiễu và Nội suy dữ liệu khuyết thiếu.....	20
2.2.1. Lọc nhiễu dựa trên độ tin cậy	21
2.2.2. Nội suy tuyến tính.....	21
2.3. Trích xuất đặc trưng không gian và động học	22
2.3.1. Đặc trưng không gian gốc	23
2.3.2. Đặc trưng vận tốc	23
2.3.3. Đặc trưng độ cao trọng tâm	24

2.3.4. Đặc trưng góc cơ thể	24
2.3.5. Xây dựng Vector đặc trưng tổng hợp.....	25
2.4. Đóng gói dữ liệu chuỗi thời gian	26
2.4.1. Kỹ thuật Cửa sổ trượt.....	26
2.4.2. Kỹ thuật đệm và che giấu	27
2.5. Mô hình phân loại học sâu.....	28
2.5.1. Recurrent Neural Network-RNN	28
2.5.2. Long short-term memory -LSTM	29
2.5.3. Bi-directional Long Short-Term Memory (BiLSTM)	31
2.6. Mô hình trích xuất khung xương	32
2.6.1 Mô hình Yolo-pose	32
2.6.2. Mô hình Yolov26-pose.....	33
2.7. Các chỉ số đánh giá hiệu năng	39
2.8. Quy trình thực hiện.....	42
CHƯƠNG 3. THỰC NGHIỆM VÀ ĐÁNH GIÁ	43
3.1. Các kỹ thuật hỗ trợ	43
3.1.1. Ngôn ngữ sử dụng.....	43
3.1.2. Môi trường sử dụng.....	44
3.2. Dữ liệu thực nghiệm.....	45
3.2.1. Đặc điểm bộ dữ liệu	45
3.2.2. Tiền xử lý dữ liệu.....	50
3.2.3. Huấn luyện mô hình.....	53
3.2.4. Đánh giá mô hình	57
CHƯƠNG 4. XÂY DỰNG HỆ THỐNG.....	61
4.1. Giới thiệu chung về sản phẩm	61
4.1.1. Phân hệ Giám sát trực tuyến.....	61
4.1.2. Phân hệ Phân tích Video ngoại tuyến	61
4.1.3. Phân hệ Lịch sử và Cơ chế cảnh báo đa kênh	62
4.2. Phân tích thiết kế phần mềm demo	63
4.2.1. Biểu đồ Use Case tổng quan.....	63

4.2.2. Đặc tả Use Case	64
4.3. Công cụ và công nghệ sử dụng.....	74
4.3.1. Ngôn ngữ lập trình	74
4.3.2. Trí tuệ nhân tạo và xử lý ảnh.....	74
4.3.3. Framework Backend	75
4.3.4. Cơ sở dữ liệu và mở rộng.....	75
4.4. Hướng dẫn sử dụng hệ thống.....	75
KẾT LUẬN.....	83
TÀI LIỆU THAM KHẢO	84

DANH MỤC CÁC TỪ VIẾT TẮT

STT	Từ viết tắt	Nghĩa Tiếng Anh	Nghĩa Tiếng Việt
1	YOLO	You Only Look Once	Mô hình học sâu phát hiện đối tượng một giai đoạn
2	ADL	Activities of Daily Living	Các hoạt động sinh hoạt bình thường
3	mAP	Mean Average Precision	Độ chính xác trung bình
4	IoU	Intersection over Union	Độ trùng khớp (trùng lắp) giữa vùng dự đoán và thực tế
5	RPN	Region Proposal Network	Mạng đề xuất vùng
6	TP	True Positive	Dương tính thật (Dự đoán đúng tế ngã)
7	TN	True Negative	Âm tính thật (Dự đoán đúng bình thường)
8	FP	False Positive	Dương tính giả (Báo động nhầm)
9	FN	False Negative	Âm tính giả (Bỏ sót ca dự đoán)
10	AI	Artificial Intelligence	Trí tuệ nhân tạo
11	AUC	Area Under the Curve	Diện tích dưới đường cong (Đánh giá đặc tuyến hoạt động)
12	BCE	Binary Cross Entropy	Hàm mất mát entropy chéo nhị phân

13	BiLSTM	Bi-directional Long Short-Term Memory	Mạng nơ-ron bộ nhớ ngắn hạn dài hai chiều
14	CCTV	Closed-Circuit Television	Camera giám sát
15	CIoU	Complete Intersection over Union	Hàm mất mát độ trùm lấp hoàn chỉnh
16	CNN	Convolutional Neural Network	Mạng nơ-ron tích chập
17	COCO	Common Objects in Context	Bộ dữ liệu đối tượng tiêu chuẩn (17 điểm khớp)
18	DFL	Distribution Focal Loss	Hàm mất mát phân phối tiêu điểm
19	FPS	Frames Per Second	Số khung hình trên giây
20	IoU	Intersection over Union	Độ trùng khớp (trùm lấp) giữa vùng dự đoán và thực tế
21	LSTM	Long Short-Term Memory	Mạng nơ-ron bộ nhớ ngắn hạn dài
22	NMS	Non-Maximum Suppression	Thuật toán triệt tiêu cực đại cục bộ
23	OKS	Object Keypoint Similarity	Độ tương đồng điểm khớp đối tượng
24	RLE	Residual Log-Likelihood Estimation	Ước lượng hợp lý logarit dư
25	RNN	Recurrent Neural Network	Mạng nơ-ron hồi tiếp
26	ROC	Receiver Operating Characteristic	Đặc tuyến hoạt động (Biểu diễn đánh đổi giữa tỷ lệ dương tính thật và dương tính giả)

27	SPPF	Spatial Pyramid Pooling Fast	Khối gom cụm tháp không gian nhanh
----	------	---------------------------------	---------------------------------------

DANH MỤC CÁC HÌNH ẢNH

Hình 1.1 Ảnh ký tự chữ A và ma trận số của vùng chọn.....	4
Hình 1.2. Ma trận số biểu thị mức xám của các điểm ảnh.....	5
Hình 1.3 Xử lý ảnh số	7
Hình 1.4 Minh họa nhận diện hành động(1)	9
Hình 1.5 Minh họa nhận diện hành động(2)	10
Hình 2.1 Mô hình tổng quan của hệ thống.....	16
Hình 2.2 Cấu trúc khung xương của yolov26-pose	18
Hình 2.3 Nhận diện khung xương thực tế của yolov26-pose	19
Hình 2.4 Mô hình RNN	29
Hình 2.5 Mô hình LSTM.....	31
Hình 2.6 Mô hình BiLSTM	32
Hình 2.7 Kiến trúc mạng YOLOv26-Pose	34
Hình 3.1 Biểu tượng Python.....	43
Hình 3.2 Tổng quan bộ dữ liệu.....	46
Hình 3.3 Tổng quan file csv khung xương.....	46
Hình 3.4 Biểu đồ cột phân bố tỷ lệ bộ dữ liệu.....	47
Hình 3.5 Biểu đồ tròn phân bố tỷ lệ bộ dữ liệu	48
Hình 3.6 Phân phối số lượng frame trong video	48
Hình 3.7 Quỹ đạo rơi tự do theo trục y	49
Hình 3.8 Sự biến đổi rộng/cao của cơ thể	49
Hình 3.9 Minh họa chất lượng bộ dữ liệu	50
Hình 3.10 Dữ liệu đầu ra sau tiên xử lý	53
Hình 3.11 Kết quả huấn luyện	56

Hình 3.12 Báo cáo phân loại đánh giá mô hình	58
Hình 3.13 Ma trận nhầm lẫn.....	59
Hình 3.14 Đường cong ROC	60
Hình 4.1 Usecase tổng quan	63
Hình 4.2 Giao diện giám sát trực tuyến	76
Hình 4.3 Giao diện nhận diện khung xương trạng thái bình thường.....	77
Hình 4.4 Giao diện nhận diện khung xương ở trạng thái té ngã	77
Hình 4.5 Giao diện nhận diện khung xương ở chế độ riêng tư.....	78
Hình 4.6 Cảnh báo tin nhắn té ngã telegram.....	79
Hình 4.7 Giao diện chọn video phân tích.....	80
Hình 4.8 Giao diện phân tích video đã chọn.....	81
Hình 4.9 Giao diện tải video đã phân tích khung xương	81
Hình 4.10 Giao diện xem lịch sử cảnh báo	82

MỞ ĐẦU

1. Đặt vấn đề

Ngày nay, sự phát triển vượt bậc của Trí tuệ nhân tạo và Thị giác máy tính đã mở ra nhiều hướng đi mới trong việc ứng dụng công nghệ vào lĩnh vực chăm sóc sức khỏe. Trong đó, phát hiện sự cố té ngã là một bài toán mang ý nghĩa thực tiễn cao, đặc biệt cấp thiết đối với người cao tuổi hoặc người bệnh sống một mình. Té ngã nếu không được phát hiện và xử lý kịp thời trong "thời gian vàng" có thể dẫn đến những chấn thương nghiêm trọng, suy giảm khả năng vận động và thậm chí đe dọa đến tính mạng.

Trước đây, các giải pháp cảnh báo phổ biến thường phụ thuộc vào các thiết bị cảm biến đeo trên người. Tuy nhiên, phương pháp này bộc lộ nhiều hạn chế như: gây bất tiện khi phải đeo liên tục, thời lượng pin ngắn và rủi ro người dùng quên mang thiết bị. Để khắc phục những nhược điểm này, việc ứng dụng camera giám sát kết hợp với thuật toán AI đang trở thành xu hướng tất yếu. Cụ thể, phương pháp phân tích hành vi dựa trên khung xương người (skeleton-based) không chỉ mang lại độ chính xác cao, ít bị ảnh hưởng bởi môi trường xung quanh mà còn giải quyết triệt để bài toán bảo vệ quyền riêng tư của người dùng.

Xuất phát từ những yêu cầu thực tiễn và tính nhân văn đó, em đã quyết định lựa chọn đề tài: **“Nghiên cứu hành vi chuyển động và bộ khung tư thế ứng dụng phát triển hệ thống cảnh báo té ngã”** làm Đồ án tốt nghiệp.

2. Mục tiêu của đề tài

Đề tài hướng tới việc nghiên cứu và phát triển một hệ thống tự động giám sát hành vi thông qua camera theo thời gian thực. Cụ thể:

- Phân tích chuỗi dữ liệu khung xương của người dùng để nhận diện, phân loại chính xác các hành vi thông thường và sự kiện té ngã.
- Xây dựng cơ chế phát cảnh báo tức thời khi xảy ra sự cố nhằm hỗ trợ tối đa cho công tác theo dõi y tế, đảm bảo an toàn cho người dùng.

3. Nội dung nghiên cứu

Để đạt được các mục tiêu trên, đồ án tập trung giải quyết các nội dung chính sau:

- Nghiên cứu cơ sở lý thuyết: Tìm hiểu về xử lý ảnh, kiến trúc mạng trích xuất điểm nòng cốt khung xương (YOLO Pose) và các mạng học sâu chuyên xử lý dữ liệu chuỗi thời gian như LSTM, BiLSTM.
- Xây dựng và huấn luyện mô hình: Thực hiện thu thập tập dữ liệu, tiền xử lý (lọc nhiễu, chuẩn hóa, trích xuất đặc trưng động học). Sau đó,

tiến hành huấn luyện và đánh giá độ chính xác của mô hình BiLSTM trên tập dữ liệu thực nghiệm.

- Phát triển ứng dụng thực tế: Tích hợp mô hình AI vào một hệ thống Web ứng dụng có khả năng giám sát trực tuyến thời gian thực. Hệ thống được trang bị cơ chế cảnh báo đa kênh (còi báo động tại chỗ, gửi tin nhắn tự động qua Telegram) và có tính năng ẩn danh giúp bảo vệ quyền riêng tư.
- Hoàn thiện báo cáo Đồ án tốt nghiệp theo đúng quy định.

4. Phương pháp nghiên cứu

Đề tài được thực hiện dựa trên sự kết hợp giữa hai phương pháp:

Phương pháp nghiên cứu lý thuyết: Phân tích, tổng hợp các tài liệu học thuật, bài báo khoa học liên quan đến lĩnh vực nhận diện hành vi, mạng nơ-ron hồi quy và thuật toán trích xuất khung xương.

Phương pháp thực nghiệm và ứng dụng: Lập trình huấn luyện mô hình dựa trên các tập dữ liệu thực tế, đo lường các chỉ số hiệu năng phân loại; từ đó triển khai xây dựng phần mềm hệ thống hoàn chỉnh để kiểm chứng tính khả thi của giải pháp.

5. Cấu trúc của đồ án

Ngoài phần Mở đầu, Kết luận, Danh mục từ viết tắt, Danh mục hình ảnh và Tài liệu tham khảo, nội dung chính của đồ án được cấu trúc thành 4 chương như sau:

- **Chương 1. Tổng quan về bài toán:** Trình bày tổng quan về lĩnh vực thị giác máy tính, phân tích tính cấp thiết của bài toán phát hiện té ngã, khảo sát các phương pháp hiện có trên thế giới và xác định phạm vi nghiên cứu của đề tài.
- **Chương 2. Cơ sở lý thuyết:** Cung cấp nền tảng kiến thức phục vụ cho việc xử lý dữ liệu và nhận diện hành vi. Trọng tâm của chương là việc phân tích mô hình trích xuất khung xương YOLO Pose và các kiến trúc mạng xử lý chuỗi thời gian (RNN, LSTM, BiLSTM).
- **Chương 3. Thực nghiệm và đánh giá:** Trình bày chi tiết quá trình tiền xử lý dữ liệu (lọc nhiễu, nội suy, áp dụng kỹ thuật cửa sổ trượt), quy trình huấn luyện mạng BiLSTM và phân tích các kết quả đánh giá hiệu năng của hệ thống thông qua các chỉ số chuẩn xác.
- **Chương 4. Xây dựng hệ thống:** Đặc tả thiết kế hệ thống phần mềm. Trình bày quá trình tích hợp các công nghệ (FastAPI, OpenCV, WebSockets) để xây dựng ứng dụng Web giám sát video thời gian

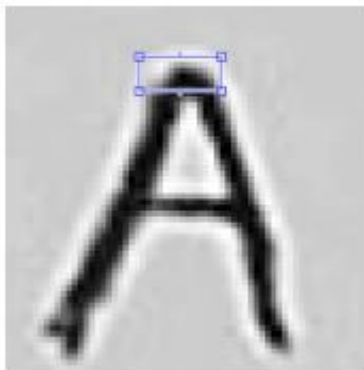
thực, đồng thời hướng dẫn sử dụng và đánh giá hoạt động thực tế của sản phẩm.

CHƯƠNG 1. TỔNG QUAN VỀ BÀI TOÁN

1.1. Tổng quan về xử lý ảnh và thị giác máy tính

1.1.1. Khái niệm ảnh số và video số

- Ảnh số là dạng ảnh đã được chuyển đổi từ ảnh liên tục (như ảnh thu từ camera) sang dạng số rời rạc. Nó thường được biểu diễn dưới dạng một ma trận hai chiều biểu diễn các giá trị cường độ (mức xám hoặc màu). Ảnh số là kết quả của quá trình số hóa ảnh liên tục, tạo ra một xấp xỉ của ngữ cảnh thực tế. Miền xác định của tọa độ ảnh số (x, y) là rời rạc và hữu hạn.



207	211	219	225	230	232	228	227	231	238	240	235	234	223
212	220	231	245	233	201	173	162	175	190	205	218	234	232
215	222	230	239	218	151	92	66	84	138	135	148	203	230
213	212	215	210	188	103	36	2	18	34	67	112	163	215
212	197	186	166	114	52	21	8	10	14	25	46	103	187
207	179	142	115	67	21	17	25	26	16	7	17	51	144

(Nguồn: internet)

Hình 1.1 Ảnh ký tự chữ A và ma trận số của vùng chọn

- Điểm ảnh (Pixel) là một phần tử của ảnh số tại tọa độ (x, y) với độ xám hoặc màu nhất định. Kích thước và khoảng cách giữa các điểm ảnh đó được chọn thích hợp sao cho mắt người cảm nhận sự liên tục về không gian và mức xám (hoặc màu) của ảnh số gần như ảnh thật. Mỗi phần tử trong ma trận được gọi là một phần tử ảnh. Trong hình 2.1 mỗi điểm ảnh là một ô mang một giá trị số biểu thị mức xám từ 0 đến 255.
- Mức xám của ảnh: Là kết quả của sự biến đổi tương ứng 1 giá trị độ sáng của 1 điểm ảnh với một giá trị nguyên dương. Thông thường nó xác định trong $[0, 255]$ tùy thuộc vào giá trị mà mỗi điểm ảnh được biểu diễn. Các thang giá trị mức xám thông thường: 2, 16, 32, 64, 128. Ảnh đa mức xám thường dùng là 256, như vậy mức xám thường xác

định trong khoảng $[0, 255]$ tùy thuộc vào giá trị mà mỗi điểm ảnh được biểu diễn.

19	39	96	184	243	244	242	235	228	218	225	239	249	252	202	72	6
7	45	123	206	255	254	251	246	242	238	240	251	254	255	221	125	41
32	73	146	217	253	247	241	236	236	232	235	248	255	255	237	150	60
45	76	133	171	187	181	168	161	166	158	160	175	198	211	183	113	50
6	27	83	76	64	67	40	36	51	40	26	36	67	72	51	39	30
32	58	92	82	64	56	34	17	15	13	17	21	26	4	1	1	16
99	139	155	162	152	132	122	100	81	90	107	113	90	56	34	30	20

(Nguồn: Internet)

Hình 1.2. Ma trận số biểu thị mức xám của các điểm ảnh

- Dựa trên số bit biểu diễn mỗi pixel, có các loại ảnh số phổ biến:
 - Ảnh nhị phân: Mỗi pixel chỉ có giá trị đen (0) hoặc trắng (1) và chỉ cần 1 bit/pixel. Thích hợp cho văn bản, vân tay, bản vẽ kiến trúc.
 - Ảnh xám (grayscale): Mỗi pixel là một màu xám, thông thường từ 0 (đen) đến 255 (trắng). Mỗi pixel có thể được biểu diễn bằng 8 bit (=1 byte).
 - Ảnh màu: Mỗi pixel có một màu cụ thể, được mô tả bằng lượng màu đỏ (Red), xanh lục (Green) và xanh lam (Blue). Ảnh màu có nhiều giá trị cho mỗi pixel; ảnh đơn sắc (monochrome) có 1 giá trị. Thông thường dùng 16-24 bit/pixel
- Video số là chuỗi các ảnh số (khung hình – frame) được hiển thị liên tiếp theo thời gian nhằm tạo ra cảm giác chuyển động. Mỗi khung hình trong video thực chất là một ảnh số độc lập. Khi các khung hình này được phát liên tục với một tốc độ nhất định (thường từ 24 đến 30 khung hình mỗi giây – FPS), mắt người sẽ cảm nhận được chuyển động liên tục.
- Trong các hệ thống thị giác máy tính, video thường được xử lý bằng cách phân tách thành các khung hình riêng lẻ. Các khung hình này sau đó được sử dụng để trích xuất thông tin cần thiết, chẳng hạn như phát hiện đối tượng, ước lượng tư thế cơ thể hoặc phân tích hành động của con người

Trong đề tài này, các khung hình video được sử dụng làm dữ liệu đầu vào cho quá trình ước lượng tư thế cơ thể người, từ đó trích xuất dữ liệu khung xương phục vụ cho việc phát hiện hành động tể ngã. Bên cạnh đó, do mỗi khung

hình thực chất là một ảnh số được cấu thành từ các điểm ảnh, nên các kỹ thuật tăng cường dữ liệu có thể được áp dụng trong quá trình huấn luyện mô hình.

1.1.2. Xử lý ảnh số

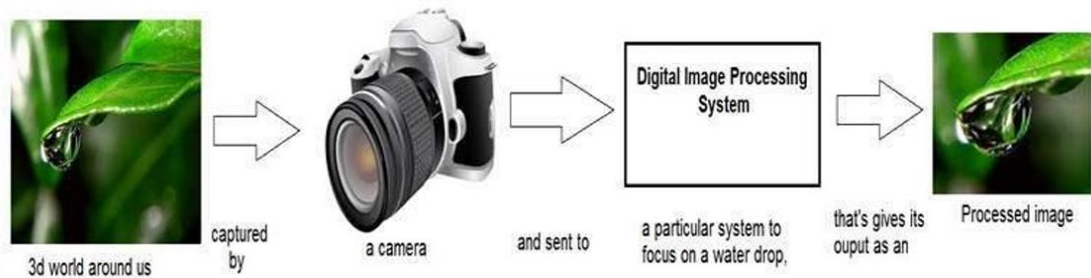
Xử lý ảnh số là quá trình phân tích và thao tác thay đổi ảnh đầu vào nhằm cải thiện chất lượng, trích xuất thông tin, hoặc chuẩn bị dữ liệu cho các mô hình trí tuệ nhân tạo. Trong lĩnh vực thị giác máy tính, xử lý ảnh đóng vai trò nền tảng để giúp máy tính hiểu được nội dung hình ảnh tương tự như con người. Các hoạt động xử lý ảnh có thể được chia thành các loại:

- Trên điểm (point operations): Giá trị điểm ảnh đầu ra chỉ phụ thuộc vào giá trị điểm ảnh tại cùng vị trí của ảnh đầu vào. Ví dụ: Ảnh âm bản ($s = \text{intensity}_{\text{max}} - r$), thay đổi độ tương phản, biến đổi log để nén vùng linh hoạt hoặc làm sáng ảnh, cân bằng histogram để tăng độ tương phản toàn cục.
- Cục bộ (local operations): Giá trị điểm ảnh đầu ra tại một vị trí nhất định phụ thuộc vào giá trị các điểm ảnh lân cận của điểm ảnh có cùng vị trí trên ảnh đầu vào. Lân cận của một điểm ảnh p tại (x, y) có thể là 4-lân-cận ngang và dọc (N4), 4-lân-cận chéo (ND), hoặc 8-lân-cận (N8). Lọc không gian là một ví dụ, kết hợp giá trị của pixel với các lân cận để làm mờ, làm trơn, hoặc làm sắc nét ảnh.
- Toàn cục (global operations): Giá trị điểm ảnh đầu ra tại một vị trí nhất định phụ thuộc vào toàn bộ pixel trên ảnh đầu vào. Ví dụ: Biến đổi toàn cục như Biến đổi Fourier.

❖ Một số phép toán cơ bản:

- Tích chập (Convolution): Là một phép toán cục bộ. Trong ảnh số (rời rạc), tích chập 2 chiều giữa ảnh $f(m, n)$ và nhân (kernel) $h(m, n)$ được tính bằng tổng các tích giữa pixel nguồn với các phần tử tương ứng trong nhân. Nhân chập thường có dạng vuông với kích thước lẻ, tâm của nhân chập thường ở giữa. Kích thước của ma trận kết quả có thể giữ nguyên, tăng hoặc giảm tùy thuộc vào kiểu chập (same, full, valid convolution). Tích chập được sử dụng trong các phép lọc không gian tuyến tính.
- Biến đổi Fourier (Fourier Transform - DFT): Chuyển đổi sự biểu diễn từ miền không gian thực sang miền tần số và ngược lại. Những thành phần tần số thấp đại diện cho vùng trơn mịn, còn tần số cao đại diện cho chi tiết (biên ảnh, nhiễu). Lọc trên miền tần số (sử dụng Biến đổi Fourier) cho phép lọc thông thấp (làm trơn) hoặc lọc thông cao (làm sắc nét) bằng cách loại bỏ các thành phần tần số tương ứng.

- ❖ Các phép toán này nhằm mục đích cải thiện chất lượng ảnh hoặc đáp ứng các yêu cầu phân tích.



(Nguồn: Internet)

Hình 1.3 Xử lý ảnh số

1.1.3. Thị giác máy tính

Thị giác máy tính (Computer Vision) là một lĩnh vực quan trọng của trí tuệ nhân tạo (Artificial Intelligence – AI), nghiên cứu các phương pháp và thuật toán giúp máy tính có khả năng thu nhận, phân tích và hiểu thông tin từ hình ảnh hoặc video. Nếu xử lý ảnh chủ yếu tập trung vào các phép biến đổi trên ảnh như lọc nhiễu, làm mờ, tăng độ tương phản hoặc cải thiện chất lượng ảnh, thì thị giác máy tính hướng tới việc trích xuất thông tin có ý nghĩa từ dữ liệu hình ảnh để phục vụ cho việc nhận dạng, phân tích và ra quyết định.

Đầu vào của các hệ thống thị giác máy tính thường là ảnh số hoặc video, trong khi đầu ra có thể là các thông tin có giá trị như nhận dạng đối tượng, xác định vị trí, phân tích hành vi hoặc dự đoán sự kiện xảy ra trong cảnh quan sát. Nhờ sự phát triển của các phương pháp học máy và học sâu, các hệ thống thị giác máy tính ngày càng đạt được độ chính xác cao và được ứng dụng rộng rãi trong nhiều lĩnh vực như giám sát an ninh, xe tự lái, y tế, robot và phân tích hành vi con người.

- Các bài toán cốt lõi trong thị giác máy tính:
 - Phân lớp ảnh là bài toán cơ bản trong thị giác máy tính, trong đó hệ thống cần xác định ảnh đầu vào thuộc về lớp đối tượng nào. Ví dụ, một mô hình phân lớp ảnh có thể xác định xem một bức ảnh chứa mèo, chó, ô tô hay con người. Trong bài toán này, đầu ra của mô hình thường là một nhãn hoặc xác suất thuộc về các lớp khác nhau. Phân lớp ảnh thường được sử dụng như bước nền tảng cho các bài toán phức tạp hơn.

- Phát hiện đối tượng là bước phát triển cao hơn so với phân lớp ảnh. Không chỉ xác định đối tượng nào xuất hiện trong ảnh, hệ thống còn phải xác định vị trí chính xác của từng đối tượng thông qua các khung giới hạn. Ví dụ, trong một bức ảnh có nhiều người, hệ thống phải phát hiện và xác định vị trí của từng người trong ảnh. Các thuật toán phổ biến cho bài toán này bao gồm các mô hình học sâu như YOLO, Faster R-CNN và SSD.
- Theo dõi đối tượng là bài toán thường được áp dụng cho dữ liệu video. Mục tiêu của bài toán này là xác định và theo dõi vị trí của một đối tượng cụ thể qua nhiều khung hình liên tiếp. Điều này cho phép hệ thống hiểu được sự di chuyển của đối tượng theo thời gian. Ví dụ, trong hệ thống giám sát an ninh, object tracking có thể được sử dụng để theo dõi chuyển động của một người trong khu vực quan sát.
- Nhận diện hành động là một trong những bài toán phức tạp trong thị giác máy tính, vì nó không chỉ phân tích một khung hình riêng lẻ mà cần xem xét sự thay đổi của đối tượng theo chuỗi thời gian. Hệ thống phải phân tích nhiều khung hình liên tiếp trong video để xác định hành vi mà đối tượng đang thực hiện, chẳng hạn như đi bộ, chạy, ngồi, nằm hoặc té ngã. Bài toán này thường sử dụng các mô hình học sâu có khả năng xử lý dữ liệu chuỗi, ví dụ như mạng nơ-ron hồi tiếp (RNN), LSTM hoặc BiLSTM.

Trong các bài toán của thị giác máy tính, đề tài đồ án thuộc bài toán nhận diện hành động. Các bài toán như phân lớp ảnh, phát hiện đối tượng và theo dõi đối tượng có thể đóng vai trò hỗ trợ trong việc xử lý dữ liệu đầu vào, tuy nhiên mục tiêu chính của hệ thống là phân tích chuỗi dữ liệu theo thời gian để phát hiện hành động té ngã của con người.

1.1.4. Bài toán nhận diện hành động

Human Action Recognition dịch ra là nhận diện hành động con người, có thể chia làm 2 loại: tĩnh và động. Kiểu tĩnh là dạng hành động như đứng, ngồi, nằm, ... , là kiểu hành động không di chuyển trong một khoảng thời gian. Còn kiểu động là mấy hành động như: đi, chạy, nhảy, ... chẳng hạn. Hành động kiểu tĩnh có thể phân loại dễ dàng bằng phân loại ảnh. Trong khi đó kiểu động lại là một chuỗi dịch chuyển của các bộ phận cơ thể người nên dùng phân loại ảnh là không chính xác, mà ta phải xét các frame có chứa hành động đó để phân loại video.

Ngày xưa, nhiều nhà nghiên cứu gắn một cái sensor vào người (điện thoại thông minh, ...) rồi dùng dữ liệu thu được từ sensor đi qua mô hình LSTM để

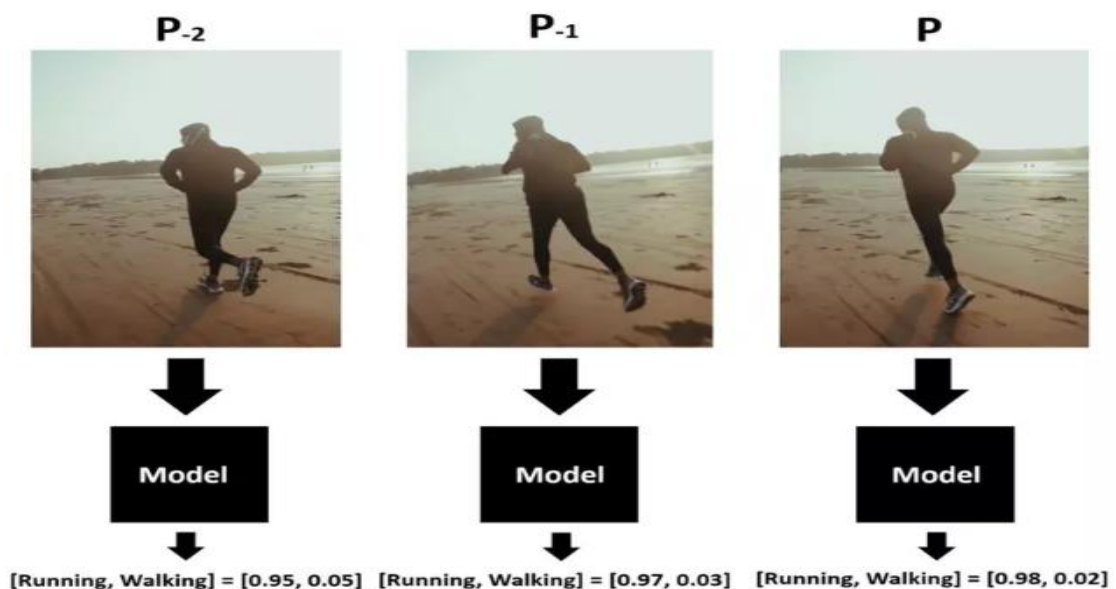
nhận diện. Ngày nay, thay vì gắn thêm sensor vào người thì các nhà nghiên cứu ứng dụng Computer Vision để nhận diện hành động của người. Ban đầu họ cũng thử Image Classification nhưng lại toang như hình dưới.



(Nguồn: Viblo)

Hình 1.4 Minh họa nhận diện hành động(1)

Bằng một cách nào đó, cũng với Image Classification, họ đã cải thiện độ chính xác của mô hình trên video. Thay vì lấy kết quả của một frame làm kết quả của video thì giờ họ lấy kết quả của 5, 10, n-frame liên tiếp nhau làm kết quả của video.



(Nguồn: Viblo)

Hình 1.5 Minh họa nhận diện hành động(2)

Như trên thì cả 3 frame liên nhau đều predict ra label "Running" với độ chính xác 95%. Vậy ta có thể khẳng định video này là "Running".

→ 2 cách này sẽ gây overfit model do chỉ chú trọng context của ảnh mà không chú ý tới hành động của người. Nếu bạn test với một ảnh có màu sắc khác biệt với ảnh train là toang ngay. Đó chính là lý do vì sao sau này người ta không dùng Image Classification thông thường cho video nữa, mà phải dùng các mạng có khả năng xử lý chuỗi thời gian như LSTM/BiLSTM, hoặc 3D-CNN, hoặc trích xuất Keypoint để mô hình thực sự học được chuyển động những mô hình này chúng ta sẽ tìm hiểu ở những phần sau.

1.2. Bài toán phát hiện té ngã

1.2.1. Sự cấp thiết của bài toán

Té ngã là một trong những nguyên nhân hàng đầu gây chấn thương nghiêm trọng, suy giảm sức khỏe và thậm chí tử vong, đặc biệt ở người cao tuổi, người có bệnh lý nền hoặc bệnh nhân trong quá trình phục hồi chức năng. Theo các nghiên cứu y khoa, té ngã có thể dẫn đến nhiều hậu quả nghiêm trọng như gãy xương, chấn thương sọ não, suy giảm khả năng vận động và các vấn đề tâm lý, từ đó làm giảm đáng kể mức độ tự lập cũng như chất lượng cuộc sống của người bệnh. Đối với người cao tuổi, các biến chứng sau té ngã thường nghiêm trọng hơn do khả năng hồi phục kém và nguy cơ phải nằm liệt giường trong thời gian dài.

Trong bối cảnh đó, các hệ thống phát hiện té ngã (Fall Detection Systems) được nghiên cứu và phát triển nhằm hỗ trợ giám sát tự động và phát hiện sự cố trong thời gian thực. Những hệ thống này có mục tiêu chính là nhận diện nhanh chóng sự kiện té ngã và gửi cảnh báo kịp thời đến người thân hoặc nhân viên y tế để có biện pháp hỗ trợ phù hợp. Việc phát hiện sớm không chỉ giúp giảm thiểu các biến chứng nguy hiểm mà còn góp phần làm giảm nỗi lo sợ té ngã của người cao tuổi, từ đó cải thiện chất lượng cuộc sống của họ.

Hiện nay, nhiều nghiên cứu đã được thực hiện nhằm phát triển các phương pháp và công nghệ phát hiện té ngã khác nhau, bao gồm các hệ thống sử dụng camera, cảm biến đeo trên người, cảm biến môi trường và các kỹ thuật trí tuệ nhân tạo. Những hệ thống này đặc biệt phù hợp trong các mô hình nhà thông minh (Smart Home), bệnh viện, trung tâm phục hồi chức năng và viện dưỡng lão. Bên cạnh việc nâng cao mức độ an toàn cho người cao tuổi, các hệ thống phát hiện té ngã còn giúp giảm bớt khối lượng công việc cho đội ngũ

chăm sóc y tế, đồng thời góp phần nâng cao hiệu quả quản lý và giám sát bệnh nhân.

1.2.2. Các chiến lược phát hiện té ngã

- Chiến lược dựa trên cảm biến đeo (Wearable-based): Trong những năm gần đây, nhiều nghiên cứu đã tập trung phát triển các hệ thống phát hiện té ngã dựa trên thiết bị đeo được kết nối thông qua các công nghệ truyền thông không dây. Các hệ thống này thường sử dụng các cảm biến chuyển động như gia tốc kế, con quay hồi chuyển và từ kế để thu thập dữ liệu về chuyển động của cơ thể người dùng. Dữ liệu thu được sau đó được xử lý bằng các thuật toán phân tích nhằm phát hiện các sự kiện té ngã và gửi cảnh báo kịp thời, ta có thể thấy quá các công trình nghiên cứu sau:
 - Pierleoni và cộng sự đã đề xuất một hệ thống phát hiện té ngã sử dụng kết hợp nhiều loại cảm biến gồm từ kế, con quay hồi chuyển và gia tốc kế. Việc kết hợp nhiều cảm biến giúp khắc phục hạn chế của việc chỉ sử dụng một gia tốc kế đơn lẻ, từ đó cải thiện khả năng nhận diện chuyển động của cơ thể. Tuy nhiên, nghiên cứu này chưa trình bày đầy đủ bộ dữ liệu thử nghiệm và phân thảo luận về các thách thức cũng như hạn chế của hệ thống vẫn còn khá ngắn gọn.
 - Baga và cộng sự đã đề xuất một hệ thống nhằm hỗ trợ quản lý các bệnh thoái hóa thần kinh thông qua việc thu nhỏ kích thước của các cảm biến đeo. Nghiên cứu này giới thiệu một kiến trúc hệ thống dạng mô-đun có thể được tùy chỉnh cho nhiều loại bệnh khác nhau. Mặc dù bài báo có thảo luận khá chi tiết về các vấn đề liên quan đến bảo mật dữ liệu, phần đánh giá khả năng phát hiện và định lượng kết quả vẫn chưa được trình bày đầy đủ.
 - Avvenuti và cộng sự đã phát triển một hệ thống giám sát trong đó các cảm biến thu thập dữ liệu theo thời gian thực và thực hiện xử lý trước khi gửi cảnh báo đến người chăm sóc. Hệ thống này có khả năng thu thập dữ liệu từ cả cảm biến đeo và cảm biến không dây, đồng thời sử dụng băng thông thấp để truyền cảnh báo. Nhờ đó, hệ thống có thể tiết kiệm tài nguyên mạng và năng lượng trong quá trình vận hành.
 - Một hướng tiếp cận khác được Wang và cộng sự đề xuất thông qua mô hình phát hiện té ngã mang tên WiFall. Phương pháp này hoạt động dựa trên mạng không dây và không yêu cầu người dùng phải đeo thiết bị hay thay đổi phần cứng của môi trường. Tuy nhiên, hệ thống này chủ yếu hoạt động hiệu quả trong môi trường chỉ có một

người, do đó còn hạn chế khi áp dụng trong các môi trường có nhiều người.

Nhìn chung, các phương pháp phát hiện té ngã dựa trên thiết bị đeo cho thấy hiệu quả trong việc thu thập dữ liệu chuyển động của người dùng. Tuy nhiên, chúng vẫn tồn tại một số hạn chế như sự bất tiện khi người dùng phải đeo thiết bị liên tục, phụ thuộc vào pin và khả năng người dùng quên mang thiết bị. Do đó, các phương pháp phát hiện té ngã dựa trên thị giác máy tính đang ngày càng được quan tâm trong các nghiên cứu gần đây.

- Chiến lược dựa trên thị giác máy tính (Vision-based): Bên cạnh các hệ thống dựa trên cảm biến đeo, nhiều nghiên cứu đã tập trung phát triển các phương pháp phát hiện té ngã dựa trên thị giác máy tính thông qua việc phân tích hình ảnh hoặc video từ camera giám sát. Các phương pháp này thường khai thác đặc trưng về chuyển động, hình dạng cơ thể hoặc cấu trúc tư thế để nhận diện sự kiện té ngã, ta có thể thấy quá các công trình nghiên cứu sau:
 - Rougier và cộng sự đã giới thiệu một kỹ thuật phát hiện té ngã tập trung vào sự thay đổi hình dạng cơ thể và các mẫu chuyển động của con người. Phương pháp này cho kết quả khả quan trên các video ghi lại các hoạt động sinh hoạt hàng ngày cũng như các tình huống tai nạn thực tế. Tuy nhiên, do dựa trên dữ liệu video nên hệ thống có thể gây ra các vấn đề liên quan đến quyền riêng tư.
 - Miguel và cộng sự đã phát triển một thiết bị phát hiện té ngã chi phí thấp dành cho môi trường nhà thông minh. Hệ thống sử dụng nhiều thuật toán khác nhau làm đầu vào cho mô hình học máy và đạt độ chính xác cao trong phát hiện té ngã. Tuy nhiên, thiết bị chỉ hoạt động hiệu quả trong điều kiện ánh sáng ban ngày.
 - Foroughi và cộng sự đã nghiên cứu các ứng dụng giám sát trong chăm sóc người cao tuổi và sử dụng các sự kiện dựa trên tư thế để phát hiện té ngã trong môi trường gia đình. Các thử nghiệm cho thấy phương pháp có khả năng nhận diện nhiều dạng té ngã khác nhau như té ngã sang bên, té ngã về phía trước hoặc phía sau. Tuy nhiên, phạm vi hoạt động của hệ thống chỉ khoảng 4 đến 5 mét.
 - Galvão và cộng sự đã đề xuất một hệ thống phát hiện té ngã sử dụng thông tin khung xương thu được từ camera Kinect v2. Hệ thống có thể theo dõi tối đa hai người trong cùng một khung hình và sử dụng dữ liệu về cấu trúc cơ thể để phân tích chuyển động. Tuy nhiên, các tập dữ liệu

được sử dụng trong nghiên cứu vẫn còn hạn chế về quy mô và chưa cung cấp đủ số lượng mẫu huấn luyện lớn.

Nhìn chung, các phương pháp phát hiện té ngã dựa trên thị giác máy tính đã đạt được nhiều kết quả tích cực nhờ khả năng phân tích trực tiếp hành vi và chuyển động của con người thông qua dữ liệu hình ảnh hoặc video. Tuy nhiên, các hệ thống này vẫn tồn tại một số hạn chế như phụ thuộc vào điều kiện ánh sáng, vấn đề quyền riêng tư và yêu cầu tài nguyên tính toán lớn. Trong những năm gần đây, các phương pháp dựa trên biểu diễn khung xương cơ thể đang được quan tâm nhiều hơn nhờ khả năng mô tả chuyển động của con người một cách hiệu quả và giảm ảnh hưởng của nhiễu nền.

1.3. Phát biểu bài toán của đề án

Từ những nền tảng lý thuyết về thị giác máy tính và quá trình khảo sát thực trạng, phần này sẽ định nghĩa một cách tường minh giới hạn, mục tiêu, chiến lược giải quyết và các thách thức của bài toán phát hiện té ngã mà đề án này hướng tới.

1.3.1. Mục tiêu của đề án

- Mục tiêu của đề án là xây dựng một hệ thống có khả năng tự động phân tích chuỗi dữ liệu khung xương của con người theo thời gian để phát hiện sự kiện té ngã. Cụ thể, hệ thống cần phân loại hành vi của con người thành hai trạng thái:
 - té ngã (Fall) – hành vi mất thăng bằng và ngã xuống, cần phát cảnh báo; và
 - sinh hoạt bình thường (Activities of Daily Living – ADL / No Fall) như đi, đứng, ngồi, cúi hoặc nằm.
- Đề án lựa chọn hướng tiếp cận dựa trên thị giác máy tính để phát hiện té ngã từ dữ liệu video. Thay vì xử lý trực tiếp ảnh RGB với khối lượng tính toán lớn và tiềm ẩn mô tả cấu trúc tư thế và chuyển động của con người thông qua các điểm khớp. Dữ liệu khung xương được trích xuất từ video và sử dụng làm đặc trưng đầu vào cho mô hình học máy.

1.3.2. Đặc tả đầu vào và đầu ra

- Đầu vào:

- Đầu vào của hệ thống là chuỗi dữ liệu khung xương của cơ thể người theo thời gian, được biểu diễn dưới dạng các tọa độ của các điểm khớp cơ thể.
- Cụ thể, tại mỗi khung hình t , cơ thể người được mô tả bởi 17 điểm khớp (keypoints) theo chuẩn skeleton phổ biến. Mỗi điểm khớp được xác định bởi tọa độ hai chiều (x_i, y_i) trên mặt phẳng ảnh.
- Do đó, tại một khung hình t , dữ liệu khung xương có thể được biểu diễn dưới dạng vector:

$$[x_1, y_1, x_2, y_2, \dots, x_{17}, y_{17}] \quad (1.1)$$

- Với một chuỗi hành động gồm T khung hình liên tiếp, dữ liệu đầu vào của mô hình sẽ được biểu diễn thành một ma trận chuỗi thời gian:

$$X \in \mathbb{R}^{T \times (17 \times 2)} \quad (1.2)$$

Trong đó:

- T : số khung hình trong chuỗi chuyển động
- 17: số điểm khớp cơ thể
- 2: số chiều tọa độ của mỗi điểm khớp (x, y)
- Trong giai đoạn huấn luyện, các dữ liệu khung xương này được lấy từ bộ dữ liệu có sẵn. Trong giai đoạn triển khai hệ thống, các điểm khớp được trích xuất trực tiếp từ video hoặc camera thời gian thực thông qua mô hình ước lượng tư thế như MediaPipe Pose.
- Đầu ra:
 - Đầu ra của hệ thống là kết quả phân loại hành vi của con người dựa trên chuỗi chuyển động khung xương.
 - Mô hình sẽ dự đoán xác suất thuộc về hai lớp hành vi:
 - Fall: hành vi té ngã
 - No Fall (ADL – Activities of Daily Living): các hoạt động sinh hoạt bình thường
 - Cụ thể mô hình sinh ra 1 giá trị xác suất:

$$\hat{Y} \in [0, 1] \quad (1.3)$$

Trong đó: $\hat{Y}$ là xác suất dự đoán thuộc lớp Fall

- Giá trị này sau đó được chuyển thành nhãn nhị phân thông qua một ngưỡng T như sau:

- $Y = 1$ (nếu $\hat{Y} > T$)
- $Y = 0$ (trong trường hợp ngược lại)

Trong đó:

- $Y = 1$: Dự đoán là té ngã (Fall)
- $Y = 0$: Dự đoán là hoạt động bình thường (No Fall)
- Khi phát hiện hành vi té ngã, hệ thống có thể kích hoạt cơ chế cảnh báo để hỗ trợ giám sát và đảm bảo an toàn cho người được theo dõi.

1.4. Thuận lợi và khó khăn

- Thuận lợi:

- Tính khả thi cao về hạ tầng (Tận dụng Camera có sẵn): Khác với các phương pháp dùng cảm biến sàn nhà đắt đỏ hay radar phức tạp, hướng đi này tận dụng trực tiếp hệ thống camera giám sát (CCTV) vốn đã được lắp đặt phổ biến tại các hộ gia đình, bệnh viện và viện dưỡng lão. Điều này giúp giảm thiểu tối đa chi phí triển khai hệ thống.
- Trải nghiệm người dùng phi tiếp xúc (Non-invasive): Khắc phục hoàn toàn nhược điểm của các thiết bị đeo (Wearable-based) như đồng hồ thông minh hay đai cảm biến. Người cao tuổi hoặc bệnh nhân không cần phải nhớ việc đeo thiết bị lên người, không cảm thấy vướng víu hay khó chịu trong sinh hoạt hàng ngày.
- Bảo vệ quyền riêng tư (Privacy-preserving): Đây là ưu điểm cốt lõi của việc dùng dữ liệu Khung xương (Skeleton). Thay vì phân tích toàn bộ bức ảnh màu (RGB) làm lộ diện mạo, không gian phòng ngủ hay các hoạt động riêng tư, hệ thống chỉ trích xuất và lưu giữ các "đoạn thẳng và điểm tọa độ" mô phỏng dáng người.

Khó khăn:

- Hiện tượng che khuất (Occlusion): Trong môi trường thực tế (nhà ở, bệnh viện), khi một người vấp ngã, cơ thể họ rất dễ bị che khuất bởi các đồ nội thất (bàn, ghế, giường, sofa). Điều này dẫn đến việc thuật toán trích xuất khung xương bị mất dấu, sinh ra các bộ tọa độ khuyết thiếu (Missing data) hoặc nhiễu loạn ở một số khung hình. Mô hình phân loại cần phải có cơ chế chịu lỗi tốt trước hiện tượng này.

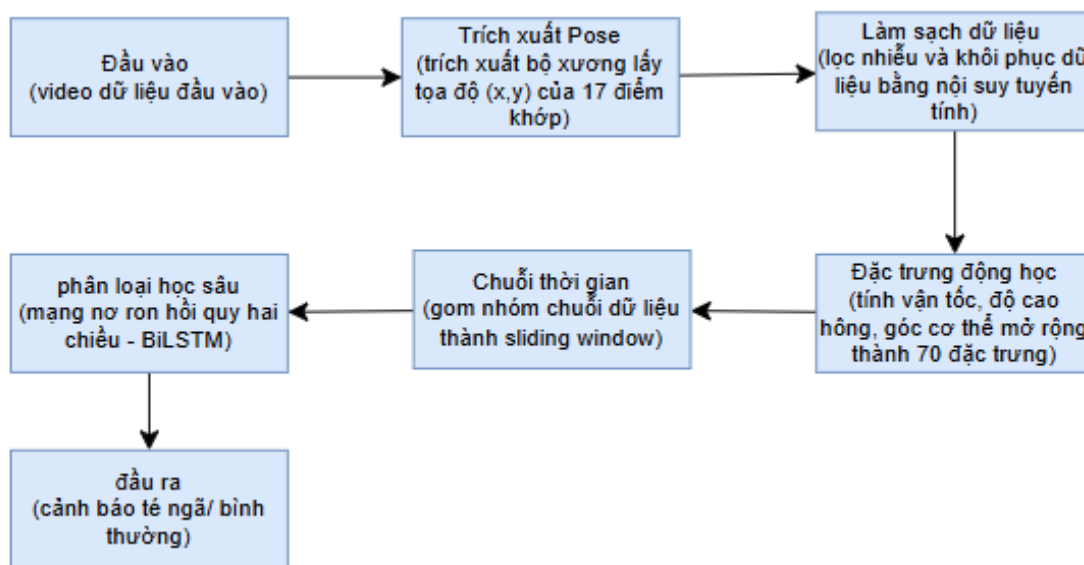
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về hệ thống đề xuất

Hệ thống phát hiện té ngã được thiết kế dựa trên sự kết hợp giữa kỹ thuật ước lượng khung xương người và mạng học sâu chuyên hồi quy chuỗi thời gian. Thay vì xử lý trực tiếp các pixel điểm ảnh (vốn đòi hỏi tài nguyên tính toán rất lớn và dễ bị nhiễu bởi ánh sáng), hệ thống tập trung vào việc phân tích các tọa độ khớp xương biến thiên theo thời gian để nhận diện các hành vi bất thường.

2.1.1. Luồng xử lý dữ liệu của hệ thống

Quy trình xử lý của hệ thống được thực hiện qua một chuỗi các công đoạn tuần tự, đảm bảo dữ liệu thô từ video được chuyển hóa thành các đặc trưng có ý nghĩa trước khi đưa vào mô hình phân loại.



Hình 2.1 Mô hình tổng quan của hệ thống

- Dữ liệu đầu vào (Input Video): Tiếp nhận luồng video từ camera hoặc tệp tin. Video thực chất là một tập hợp các khung hình (frames) liên tiếp.
- Trích xuất tọa độ xương khớp (Pose Estimation): Sử dụng các thuật toán thị giác máy tính để xác định vị trí của các khớp xương trọng yếu trên cơ thể (như đầu gối, hông, vai, cổ) dưới dạng tọa độ (x, y).

- Tiền xử lý và Khôi phục tín hiệu: Xử lý các sai số do che khuất hoặc nhận diện nhầm bằng các thuật toán lọc nhiễu và nội suy, đảm bảo tính liên tục của dữ liệu.
- Trích xuất đặc trưng động học (Feature Engineering): Tính toán các thông số hỗ trợ như vận tốc di chuyển, góc nghiêng cơ thể và sự thay đổi độ cao của trọng tâm.
- Gom nhóm chuỗi thời gian (Sliding Window): Chia nhỏ dòng dữ liệu liên tục thành các đoạn (windows) có độ dài cố định để mô hình có thể quan sát được hành động trong một khoảng thời gian nhất định.
- Mô hình phân loại BiLSTM: Sử dụng mạng nơ-ron học sâu hai chiều để phân tích mối quan hệ phụ thuộc giữa các trạng thái cơ thể trong quá khứ và tương lai.
- Kết quả (Output): Đưa ra xác suất và nhãn phân loại cuối cùng: "Té ngã" (Fall) hoặc "Hành động bình thường" (Non-fall).

2.1.2. Biểu diễn dữ liệu bộ khung xương người

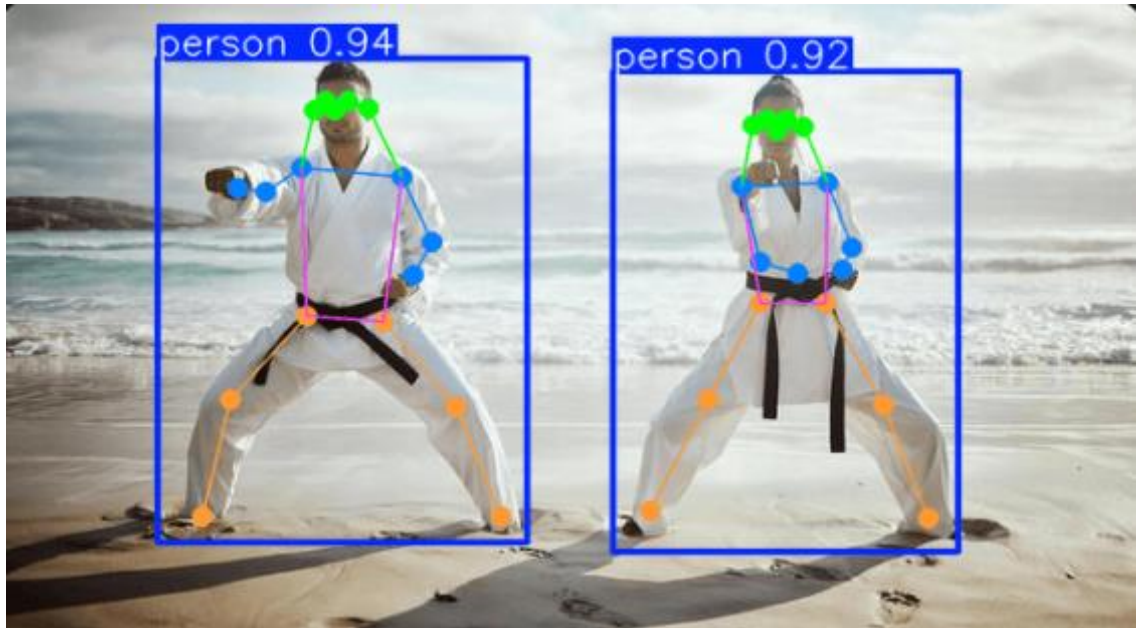
- Trích xuất khung xương người hay ước lượng tư thế là một bài toán quan trọng trong lĩnh vực thị giác máy tính. Mục tiêu của bài toán này là phát hiện, xác định vị trí và liên kết các điểm khớp quan trọng trên cơ thể người (như đầu, vai, khuỷu tay, đầu gối...) từ hình ảnh hoặc video, từ đó tạo ra một mô hình khung xương 2D hoặc 3D đại diện cho tư thế của con người.
- Đối với hệ thống cảnh báo ngã, việc trích xuất khung xương giúp loại bỏ các thông tin nhiễu từ môi trường xung quanh (quần áo, phong nền, đồ vật), chỉ giữ lại dữ liệu chuyển động cốt lõi của cơ thể để phân tích.
- Tiêu chuẩn dữ liệu COCO (17 điểm khớp): Mô hình YOLOv26-pose được sử dụng trong hệ thống được huấn luyện dựa trên bộ dữ liệu chuẩn COCO (Common Objects in Context). Trong chuẩn COCO, cơ thể người được mô hình hóa bằng 17 điểm đặc trưng (Keypoints). Các điểm này được đánh số thứ tự từ 0 đến 16, cụ thể như sau:



(Nguồn: LearnOpenCV)

Hình 2.2 Cấu trúc khung xương của yolov26-pose

- Vùng đầu (5 điểm): 0: Nose (Mũi), 1: Left Eye (Mắt trái), 2: Right Eye (Mắt phải), 3: Left Ear (Tai trái), 4: Right Ear (Tai phải).
- Vùng thân trên và tay (6 điểm): 5: Left Shoulder (Vai trái), 6: Right Shoulder (Vai phải), 7: Left Elbow (Khuỷu tay trái), 8: Right Elbow (Khuỷu tay phải), 9: Left Wrist (Cổ tay trái), 10: Right Wrist (Cổ tay phải).
- Vùng thân dưới và chân (6 điểm): 11: Left Hip (Hông trái), 12: Right Hip (Hông phải), 13: Left Knee (Đầu gối trái), 14: Right Knee (Đầu gối phải), 15: Left Ankle (Mắt cá chân trái), 16: Right Ankle (Mắt cá chân phải).



(Nguồn: LearnOpenCV)

Hình 2.3 Nhận diện khung xương thực tế của yolov26-pose

- Biểu diễn toán học của dữ liệu khung xương - Để máy tính có thể xử lý và phân loại hành vi từ các điểm khớp đã nhận diện, dữ liệu khung xương cần được số hóa dưới dạng các đại lượng toán học có cấu trúc. Quá trình này được mô tả qua các cấp độ biểu diễn sau:
 - Đại diện điểm khớp (Keypoint Representation): Mỗi điểm khớp thứ i ($i \in \{0, 1, \dots, 16\}$ theo chuẩn COCO) tại một khung hình t được biểu diễn bởi một vector ba thành phần:

$$P(i, t) = (x(i, t), y(i, t), c(i, t)) \quad (2.1)$$

Trong đó:

- $x(i, t), y(i, t)$: Tọa độ không gian 2D của điểm khớp trong khung hình, thường được chuẩn hóa về khoảng $[0, 1]$ dựa trên kích thước ảnh để đảm bảo tính bất biến về quy mô (scale invariance).
 - $c(i, t)$: Độ tin cậy (confidence score) từ mô hình trích xuất, đại diện cho xác suất điểm khớp thực sự tồn tại tại tọa độ đó.
- Biểu diễn thực thể tại một thời điểm (Spatial Vector): Tư thế của con người tại thời điểm t là một tập hợp hữu hạn các điểm khớp, được biểu diễn bằng một ma trận đặc trưng không gian $K(t)$ có kích thước 17×3 :

$$K_t = \begin{bmatrix} P_{0,t} \\ P_{1,t} \\ \vdots \\ P_{16,t} \end{bmatrix} = \begin{bmatrix} x_{0,t} & y_{0,t} & c_{0,t} \\ x_{1,t} & y_{1,t} & c_{1,t} \\ \vdots & \vdots & \vdots \\ x_{16,t} & y_{16,t} & c_{16,t} \end{bmatrix} \quad (2.2)$$

Dữ liệu này phản ánh cấu trúc hình học của cơ thể tại một thời điểm đơn lẻ.

- Biểu diễn chuỗi thời gian - hành động té ngã không thể xác định qua một khung hình đơn lẻ mà phải dựa trên sự biến thiên vị trí của bộ xương theo thời gian. Do đó, đầu vào của hệ thống là một chuỗi gồm T khung hình liên tiếp:

$$X = \{K(1), K(2), \dots, K(T)\} \quad (2.3)$$

Về mặt toán học, dữ liệu này được lưu trữ dưới dạng một tensor 3 chiều có kích thước:

$$(T, 17, 3) \quad (2.4)$$

Trong đó:

- T: Độ dài chuỗi thời gian (sequence length)
- 17: Số lượng điểm khớp theo chuẩn COCO
- 3: Các thành phần thông tin (x, y, c)
- Cấu trúc dữ liệu lưu trữ (Flat Representation): Trong các bài toán thực tế sử dụng file CSV, mỗi hàng đại diện cho một khung hình t thường được "làm phẳng" (flatten) từ ma trận K_t thành một vector đặc trưng V_t :

$$V_t = [x_0, y_0, c_0, x_1, y_1, c_1, \dots, x_{16}, y_{16}, c_{16}] \quad (2.5)$$

Với độ dài vector là $17 \times 3 = 51$ phần tử. Tập hợp các vector này qua thời gian tạo thành một bảng dữ liệu động học, phục vụ cho các giai đoạn tiền xử lý và nội suy tín hiệu tiếp theo trong hệ thống.

2.2. Kỹ thuật xử lý nhiễu và Nội suy dữ liệu khuyết thiếu

Dữ liệu chuỗi thời gian trích xuất từ các mô hình nhận diện khung xương thường không hoàn hảo. Trong điều kiện thực tế, chuyển động của con người thường xuyên bị che khuất (occlusion), góc quay camera không thuận lợi, hoặc các cử động lướt qua quá nhanh khiến thuật toán nhận diện sai lệch. Nếu đưa trực tiếp luồng dữ liệu thô này vào mạng nơ-ron sâu, các giá trị nhiễu sẽ làm hỏng quá trình cập nhật trọng số và làm giảm nghiêm trọng độ chính xác của hệ thống. Do đó, bước làm sạch và khôi phục tín hiệu là bắt buộc.

2.2.1. Lọc nhiễu dựa trên độ tin cậy

Mỗi điểm neo (keypoint) trên cơ thể người được trích xuất từ mô hình nhận diện thường được biểu diễn dưới dạng một vector ba chiều:

$$K_i = (x_i, y_i, c_i) \quad (2.6)$$

Trong đó:

- x_i, y_i : là tọa độ không gian 2D của khớp xương thứ i trên khung hình.
 - c_i : là điểm độ tin cậy (Confidence score), có giá trị nằm trong khoảng $[0, 1]$, thể hiện mức độ chắc chắn của mô hình đối với vị trí tọa độ vừa dự đoán.
- Phương pháp xử lý: Để loại bỏ các tín hiệu sai lệch (outliers), hệ thống đề xuất thiết lập một ngưỡng độ tin cậy tối thiểu (Confidence Threshold) ký hiệu là τ . Trong phạm vi nghiên cứu của bài toán này, ngưỡng τ được thiết lập ở mức 0.3.
 - Ý nghĩa: Các điểm có độ tin cậy thấp hơn 0.3 (tức là mô hình nhận diện đang "đoán mò" vị trí khớp do bị che khuất) sẽ bị cưỡng ép chuyển thành giá trị khuyết thiếu (Missing Values / NaN). Việc chủ động tạo ra các lỗ hổng dữ liệu NaN này thực chất là bước tiền đề an toàn, ngăn chặn mạng nơ-ron học phải các quỹ đạo chuyển động "bóng ma" (những chuyển động co gập không có thực của khung xương). Các giá trị NaN này sẽ được khôi phục một cách có cơ sở toán học ở bước tiếp theo.
 - Sau khi lọc nhiễu dựa trên độ tin cậy dữ liệu sẽ có dạng:

$$K_i = (x_i, y_i) \quad (2.7)$$

2.2.2. Nội suy tuyến tính

Sau bước lọc nhiễu, chuỗi thời gian tọa độ của một khớp xương sẽ xuất hiện các khoảng trống NaN. Tuy nhiên, các mạng nơ-ron như LSTM yêu cầu đầu vào phải là một ma trận liên tục và không chứa giá trị vô định. Để giải quyết vấn đề này, đề tài sử dụng phương pháp Nội suy tuyến tính dọc theo trục thời gian.

- Cơ sở toán học của nội suy tuyến tính:
 - Nội suy tuyến tính dựa trên giả định rằng: Trong một khoảng thời gian rất ngắn (vài khung hình - tương đương vài phần mười giây), chuyển động của cơ thể người là một chuyển động đều, quỹ đạo của khớp xương tạo thành một đường thẳng nối giữa điểm bắt đầu và điểm kết thúc của khoảng bị khuất.

- Giả sử quỹ đạo của tọa độ trục X của một khớp xương theo thời gian t bị khuyết thiếu tại thời điểm t. Hệ thống sẽ tìm kiếm khung hình hợp lệ gần nhất trước đó (tại thời điểm t₁) và khung hình hợp lệ gần nhất sau đó (tại thời điểm t₂).
- Giá trị khuyết thiếu X(t) được ước lượng bằng công thức:

$$X(t) = X(t_1) + (t - t_1)/(t_2 - t_1) \times [X(t_2) - X(t_1)] \quad (2.8)$$
- Xử lý các trường hợp ngoại biên:
 - Phương pháp nội suy tuyến tính thông thường chỉ hoạt động khi điểm khuyết thiếu bị kẹp giữa hai điểm hợp lệ (t₁ < t < t₂). Tuy nhiên, trong thực tế thường phát sinh các trường hợp dị biệt ở hai đầu biên: khớp xương bị che khuất ngay từ những khung hình đầu tiên của video, hoặc mất hoàn toàn ở cuối video. Lúc này, không tồn tại t₁ hoặc t₂ để thực hiện công thức nội suy.
 - Để khắc phục giới hạn này và hoàn thiện ma trận dữ liệu, hệ thống triển khai cơ chế xử lý kẹp:
 - Lan truyền giá trị biên: Thuật toán nội suy được thiết lập để quét theo cả hai hướng (forward và backward). Các giá trị khuyết thiếu ở đầu hoặc cuối chuỗi thời gian sẽ được lấp đầy bằng cách lấy tiệm cận giá trị hợp lệ gần nhất. Việc này giúp giữ nguyên trạng thái vị trí cuối cùng của khớp xương thay vì tạo ra khoảng trống.
 - Điền khuyết bằng 0: Đối với các trường hợp cực đoan (ví dụ: góc máy camera chỉ quay nửa thân trên khiến toàn bộ tọa độ chân bị khuất trong 100% thời lượng video), hệ thống kích hoạt cơ chế điền khuyết để đồng bộ toàn bộ tọa độ vô định đó về giá trị 0. Kỹ thuật này đảm bảo tính toàn vẹn về mặt số chiều của ma trận đầu vào trước khi tiến hành trích xuất các đặc trưng động học.

2.3. Trích xuất đặc trưng không gian và động học

Trong các bài toán nhận diện hành vi, đặc biệt là nhận diện té ngã (Fall Detection), việc chỉ sử dụng tọa độ thô của các khớp xương là không đủ để mô hình nắm bắt được bản chất vật lý của chuyển động. Các điểm yếu của dữ liệu thô bao gồm sự phụ thuộc vào độ phân giải camera, khoảng cách từ người đến ống kính và góc máy.

Do đó, bước trích xuất đóng vai trò cốt lõi. Mục tiêu của phần này là biến đổi dữ liệu tọa độ không gian thô thành các đại lượng có ý nghĩa về mặt động

học, giúp mô hình học được trạng thái mất thăng bằng và sự suy giảm độ cao đột ngột. Hệ thống trích xuất một vector đặc trưng tổng hợp bao gồm 4 nhóm chính.

Giả sử mô hình trích xuất khung xương (Pose Estimation) trả về tập hợp $N = 17$ điểm khớp xương chuẩn (chuẩn COCO format) cho mỗi người trong khung hình.

2.3.1. Đặc trưng không gian gốc

Đặc trưng không gian gốc định vị tư thế của cơ thể trong không hình học 2D. Tại khung hình thứ t , tọa độ của điểm khớp xương thứ i được biểu diễn bởi vector vị trí:

$$P_t^i = (x_t^i, y_t^i) \quad (2.9)$$

Trong đó $i \in \{1, 2, \dots, 17\}$

Để loại bỏ sự phụ thuộc vào kích thước ảnh gốc (W, H), các tọa độ này cần được chuẩn hóa (Normalization) về khoảng $[0, 1]$:

$$\tilde{x}_t^i = \frac{x_t^i}{W}, \tilde{y}_t^i = \frac{y_t^i}{H} \quad (2.10)$$

Tập hợp các tọa độ không gian tại khung hình t tạo thành một vector cột S_t có số chiều là $17 \times 2 = 34$:

$$S_t = [\tilde{x}_t^1, \tilde{y}_t^1, \tilde{x}_t^2, \tilde{y}_t^2, \dots, \tilde{x}_t^{17}, \tilde{y}_t^{17}]^T \in \mathbb{R}^{34} \quad (2.11)$$

Ý nghĩa: Cung cấp cho mô hình hình thái tổng thể (pose) của đối tượng tại một thời điểm cắt ngang (static posture).

2.3.2. Đặc trưng vận tốc

Tế ngã là một hành động mang tính đột ngột với gia tốc lớn. Do đó, việc bổ sung đặc trưng vận tốc giúp mô hình nhận biết sự thay đổi trạng thái thay vì chỉ nhìn vào các tư thế tĩnh.

Vận tốc tuyến tính của điểm khớp xương thứ i tại thời điểm t được tính bằng đạo hàm bậc nhất của vị trí theo thời gian. Trong không gian rời rạc của video, công thức được xấp xỉ bằng sự chênh lệch tọa độ giữa hai khung hình liên tiếp:

$$V_t^i = P_t^i - P_{t-1}^i \quad (2.12)$$

Triển khai ra các thành phần tọa độ:

$$v_{x,t}^i = \tilde{x}_t^i - \tilde{x}_{t-1}^i \quad (2.13)$$

$$v_{y,t}^i = \tilde{y}_t^i - \tilde{y}_{t-1}^i \quad (2.14)$$

(Lưu ý: Tại khung hình đầu tiên $t = 0$, vận tốc được gán bằng 0 do không có khung hình trước đó).

Tập hợp các vector vận tốc tạo thành vector đặc trưng động học V_t với 34 chiều:

$$V_t = [v_{x,t}^1, v_{y,t}^1, \dots, v_{x,t}^{17}, v_{y,t}^{17}]^T \in \mathbb{R}^{34} \quad (2.15)$$

Ý nghĩa: Các giá trị vận tốc trục $y(v_{y,t})$ mang giá trị dương lớn (do trục Y của ảnh hướng xuống) sẽ là tín hiệu "báo động" cực mạnh cho hành vi rơi tự do của cơ thể.

2.3.3. Đặc trưng độ cao trọng tâm

Hành vi té ngã luôn đi kèm với sự suy giảm đột ngột trọng tâm của cơ thể so với mặt đất. Trong hình học 2D, trung điểm của hai khớp hông (Left Hip và Right Hip) là điểm xấp xỉ lý tưởng nhất cho Trọng tâm cơ thể (Center of Mass - CoM).

Gọi tọa độ hông trái là $(x_{HL,t}, y_{HL,t})$ và hông phải là $(x_{HR,t}, y_{HR,t})$. Tọa độ trọng tâm hông $C_{hip,t}$ được tính toán bằng trung bình cộng:

$$C_{hip,t} = \left(\frac{x_{HL,t} + x_{HR,t}}{2}, \frac{y_{HL,t} + y_{HR,t}}{2} \right) \quad (2.16)$$

Đặc trưng được trích xuất là giá trị trục Y của trọng tâm, ký hiệu là h_t :

$$h_t = y_{C_{hip,t}} \in \mathbb{R}^1 \quad (2.17)$$

Ý nghĩa: Chuỗi thời gian của h_t sẽ có dạng một đường tiệm cận ngang khi đi bộ, nhưng sẽ là một đường dốc đứng (hệ số góc lớn) khi xảy ra sự cố té ngã. Đây là bộ lọc triệt tiêu các hành động nhiễu như vung tay, gập đầu (những hành động có vận tốc V_t lớn nhưng độ cao h_t không đổi).

2.3.4. Đặc trưng góc cơ thể

Sự khác biệt cốt lõi giữa trạng thái bình thường (đứng, đi bộ) và trạng thái té ngã nằm ở phương của cơ thể: chuyển từ phương dọc thẳng đứng sang phương ngang mặt đất.

Để lượng hóa điều này, ta sử dụng một vector đại diện cho trục cột sống (Spine Vector), nối từ trọng tâm Hông (C_{hip}) lên trọng tâm Cổ (C_{neck}). Tọa độ Cổ $C_{neck,t}$ được xấp xỉ qua trung điểm hai vai:

$$C_{neck,t} = \left(\frac{x_{SL,t} + x_{SR,t}}{2}, \frac{y_{SL,t} + y_{SR,t}}{2} \right) \quad (2.18)$$

Vector cột sống $\vec{u}_t$ được xác định:

$$\vec{u}_t = C_{neck,t} - C_{hip,t} = (\Delta x_t, \Delta y_t) \quad (2.19)$$

Sử dụng hàm lượng giác nghịch đảo để tính toán góc nghiêng của cơ thể θ_t so với trục hoành (trục X):

$$\theta_t = \arctan\left(\frac{\Delta y_t}{\Delta x_t}\right) \quad (2.20)$$

Trong lập trình thực tế, hàm atan2(dy, dx) được sử dụng để bảo toàn dấu của góc trong hệ tọa độ cực $[-\pi, \pi]$, tránh lỗi chia cho 0 khi cơ thể hoàn toàn thẳng đứng ($\Delta x_t \approx 0$). Góc này sau đó có thể được chuẩn hóa về mức $[0, 1]$ bằng công thức:

$$\tilde{\theta}_t = \frac{\theta_t + \pi}{2\pi} \in \mathbb{R}^1 \quad (2.21)$$

2.3.5. Xây dựng Vector đặc trưng tổng hợp

Thay vì nạp trực tiếp dữ liệu tọa độ thô vào mạng nơ-ron, sự kết hợp giữa tri thức vật lý (kinematics) và toán học học không gian đã giúp tạo ra một biểu diễn dữ liệu ưu việt hơn.

Tại mỗi khung hình t , các đặc trưng được ghép nối (concatenate) lại với nhau để tạo thành vector đặc trưng cuối cùng F_t :

$$F_t = S_t \oplus V_t \oplus [h_t] \oplus [\theta_t] \quad (2.22)$$

Kiểm tra số chiều (Dimensions) của vector F_t :

$$\begin{aligned} Dim(F_t) &= 34 \text{ (Không gian)} + 34 \text{ (Vận tốc)} + 1 \text{ (Độ cao hông)} + \\ &\quad 1 \text{ (Góc cơ thể)} = 70 \text{ chiều} \end{aligned} \quad (2.23)$$

→ Vector 70 chiều này là một Dense Feature Vector (Vector đặc trưng cô đặc). Nó hội tụ đủ các yếu tố: tĩnh (tọa độ cơ thể), động (vận tốc di chuyển) và bối cảnh hành vi (góc nghiêng, suy giảm độ cao). Việc thiết kế đặc trưng thủ công

(Hand-crafted Feature Engineering) này giúp giảm tải đáng kể khối lượng tính toán cho các mô hình phân loại phía sau (như SVM, LSTM hay GCN), giúp hệ thống tăng tốc độ nội suy (inference speed), đáp ứng được yêu cầu nhận diện Real-time (thời gian thực) của đề án.

2.4. Đóng gói dữ liệu chuỗi thời gian

Để mạng nơ-ron có thể học được sự thay đổi của các đặc trưng động học (như gia tốc, sự biến thiên góc cơ thể) theo thời gian, dữ liệu đầu vào không thể là các vector đơn lẻ mà phải là một ma trận chuỗi. Tuy nhiên, trong thực tế, các đoạn video thu thập được thường có độ dài ngắn khác nhau. Việc chuẩn hóa cấu trúc dữ liệu chuỗi được thực hiện qua hai kỹ thuật cốt lõi: Cửa sổ trượt (Sliding Window) và Đệm - Che giấu (Padding & Masking).

2.4.1. Kỹ thuật Cửa sổ trượt

Kỹ thuật cửa sổ trượt là phương pháp trích xuất các chuỗi con (sub-sequences) có độ dài cố định từ một chuỗi dữ liệu gốc dài hơn. Phương pháp này giải quyết hai bài toán lớn: giới hạn độ trễ (latency) cho hệ thống nhận diện thời gian thực và gia tăng số lượng mẫu dữ liệu huấn luyện (Data Augmentation).

Giả sử một video đầu vào có tổng cộng T khung hình. Sau bước trích xuất đặc trưng, toàn bộ video được biểu diễn bằng một ma trận:

$$V = [F_1, F_2, F_3, \dots, F_T]^T \in \mathbb{R}^{T \times 70} \quad (2.24)$$

Ta định nghĩa hai siêu tham số (hyperparameters) cho thuật toán cửa sổ trượt:

1. L (Sequence Length / Window Size): Độ dài của cửa sổ, tức là số khung hình mỗi chuỗi con sẽ chứa (ví dụ: $L = 30$ frames tương đương 1 giây video).
2. S (Step / Stride): Bước trượt của cửa sổ sau mỗi lần trích xuất.

Quy trình trượt được thực hiện như sau:

- Chuỗi thứ nhất: Lấy từ khung hình 1 đến L Ký hiệu:

$$X_1 = [F_1, F_2, \dots, F_L] \quad (2.25)$$

- Chuỗi thứ hai: Lấy từ khung hình $1 + S$ đến $L + S$ Ký hiệu:

$$X_2 = [F_{1+S}, F_{2+S}, \dots, F_{L+S}] \quad (2.26)$$

- Chuỗi thứ k :

$$X_k = [F_{1+(k-1)S}, \dots, F_{L+(k-1)S}] \quad (2.27)$$

Tổng số lượng chuỗi con N trích xuất được từ video gốc được tính bằng công thức:

$$N = \left\lfloor \frac{T-L}{S} \right\rfloor + 1 \quad (2.28)$$

Ý nghĩa của bước trượt S :

Nếu $S < L$, các cửa sổ sẽ có sự chồng lấp dữ liệu (Overlap) với độ dài là $L - S$. Việc cho phép dữ liệu chồng lấp mang lại hai ưu điểm vượt trội:

- Tăng cường dữ liệu (Data Augmentation): Từ một video té ngã duy nhất, hệ thống tạo ra được nhiều mẫu học (samples) với các thời điểm bắt đầu hành động khác nhau, giúp mô hình học được tính bất biến về mặt thời gian (Temporal Invariance).
- Bảo toàn tính toàn vẹn của hành động: Tránh trường hợp một hành động té ngã bị cắt đứt làm đôi ở phần ranh giới giữa hai cửa sổ liền kề.

Kết quả của bước này sẽ biến đổi ma trận 2D thành một Tensor 3D có kích thước: $(N \times L \times 70)$. Đây chính là hình dạng chuẩn để nạp vào các mô hình học chuỗi như LSTM, GRU hoặc Temporal Convolutional Networks (TCN).

2.4.2. Kỹ thuật đệm và che giấu

Các mô hình Deep Learning tính toán song song dựa trên ma trận (Batch Processing), do đó yêu cầu tất cả các mẫu dữ liệu trong một Batch phải có cùng chung một kích thước. Tuy nhiên, trong quá trình tiền xử lý hoặc khi nhận diện thực tế, có những hành động kết thúc sớm hoặc thuật toán theo dõi (tracking) bị mất dấu, dẫn đến việc chuỗi thu được có độ dài $T' < L$. Để giải quyết vấn đề này mà không làm biến dạng dữ liệu, kỹ thuật Padding và Masking được áp dụng song hành.

a) Kỹ thuật đệm (Padding):

Khi độ dài chuỗi T' nhỏ hơn độ dài chuẩn L , hệ thống sẽ chèn thêm $(L - T')$ vector chứa toàn các giá trị giả (Pad values) vào cuối chuỗi (Post-padding) hoặc đầu chuỗi (Pre-padding) để lấp đầy cửa sổ. Thông thường, giá trị đệm được chọn là 0 (Zero-padding). Chuỗi dữ liệu sau khi đệm $\hat{X}$ có dạng:

$$\hat{X} = [F_1, F_2, \dots, F_{T'}, \vec{0}, \vec{0}, \dots, \vec{0}] \in \mathbb{R}^{L \times 70} \quad (2.29)$$

$$\text{Trong đó, } \vec{0} = [0, 0, \dots, 0]^T \in \mathbb{R}^{70}$$

b) Kỹ thuật che giấu:

Mặc dù Padding giúp thống nhất kích thước Tensor, nhưng việc thêm một lượng lớn các vector 0 vào dữ liệu có thể làm nhiễu loạn quá trình học của mạng nơ-ron (ví dụ: làm sai lệch các phép tính trung bình, hoặc làm mạng nhầm lẫn rằng đối tượng đang đứng im hoàn toàn).

Để ngăn chặn điều này, một ma trận mặt nạ (Masking) được sinh ra để báo cho mô hình biết đâu là dữ liệu thực, đâu là dữ liệu giả. Cho một chuỗi có độ dài thực T' , vector Mask M độ dài L được định nghĩa:

$$M_j = \begin{cases} 1 \text{ (True), nếu } 1 \leq j \leq T' \\ 0 \text{ (False), nếu } T' < j \leq L \end{cases} \quad (2.30)$$

Khi mô hình (ví dụ: lớp LSTM hoặc cơ chế Attention) tính toán, trạng thái ẩn (Hidden State) hoặc trọng số chú ý sẽ được nhân nhị phân với vector Mask này. Nếu $M_j = 0$, tín hiệu lan truyền ngược (Backpropagation) tại bước thời gian đó sẽ bị chặn lại, và giá trị Loss (hàm mất mát) sẽ bỏ qua các khung hình padding.

2.5. Mô hình phân loại học sâu

2.5.1. Recurrent Neural Network-RNN

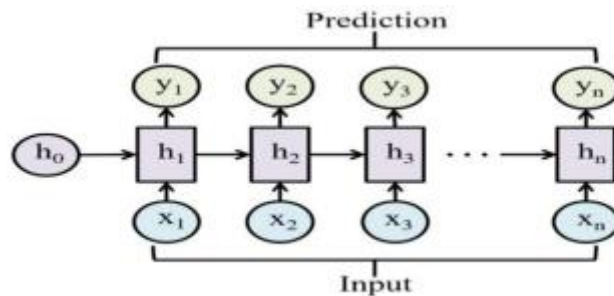
- Khái niệm

Mạng nơ-ron truyền thống (Feed-forward Neural Network) giả định rằng các đầu vào và đầu ra là độc lập với nhau. Tuy nhiên, giả định này không phù hợp với dữ liệu dạng chuỗi (Sequence Data) như văn bản, âm thanh hay dữ liệu chuỗi thời gian, nơi ngữ cảnh của từ/dữ liệu hiện tại phụ thuộc rất lớn vào dữ liệu ở trước đó.

Mạng nơ-ron hồi tiếp (RNN) ra đời để giải quyết vấn đề này. Đặc điểm cốt lõi của RNN là nó sở hữu các "vòng lặp" bên trong, cho phép thông tin được lưu lại. RNN có một trạng thái ẩn (Hidden state) đóng vai trò như một "bộ nhớ", giúp mạng nắm bắt được thông tin về những gì đã được tính toán ở các bước thời gian trước đó.

- Cơ chế hoạt động

Thay vì xử lý toàn bộ dữ liệu cùng lúc, RNN xử lý dữ liệu tuần tự theo từng bước thời gian. Một mạng RNN có thể được hình dung bằng cách "trái phải" dọc theo chiều dài của chuỗi dữ liệu.



(Nguồn: Viblo)

Hình 2.4 Mô hình RNN

Tại một bước thời gian t bất kỳ, RNN thực hiện tính toán dựa trên hai nguồn thông tin:

- Đầu vào hiện tại (x_t): Dữ liệu tại thời điểm t
- Trạng thái ẩn từ bước trước (h_{t-1}): "Ký ức" mang thông tin của các bước $t - 1, t - 2, \dots$

- Công thức

Quá trình cập nhật trạng thái ẩn và tính toán đầu ra tại bước thời gian t được mô tả bằng các phương trình sau:

- Tính trạng thái ẩn:

$$h_t = \tanh(W_{hh}h_{t-1} + W_{hx}x_t + b_h) \quad (2.31)$$

- Tính đầu ra:

$$y_t = W_{hy}h_t + b_y \quad (2.32)$$

Trong đó:

- x_t, y_t : Lần lượt là vector đầu vào và vector đầu ra tại bước thời gian t
- h_t, h_{t-1} : Vector trạng thái ẩn tại thời điểm hiện tại (t) và quá khứ ($t - 1$). Ở bước đầu tiên ($t = 1$), h_0 thường được khởi tạo bằng vector 0
- .
- $\tanh$: Hàm kích hoạt phi tuyến (Hyperbolic Tangent), giúp chuẩn hóa giá trị trong khoảng $[-1, 1]$.
- W_{xh}, W_{hh}, W_{hy} : Các ma trận trọng số. Đặc biệt lưu ý, kiến trúc RNN sử dụng chung (chia sẻ) một bộ ma trận trọng số này cho tất cả các bước thời gian t , giúp giảm thiểu số lượng tham số cần huấn luyện.
- b_h, b_y : Các vector độ lệch (bias).

Tuy đơn giản và dễ triển khai, RNN gặp khó khăn khi học các quan hệ dài hạn do hiện tượng gradient biến mất hoặc bùng nổ.

2.5.2. Long short-term memory -LSTM

LSTM khắc phục hạn chế của RNN bằng cách sử dụng các cổng điều khiển (quên, vào, ra) để kiểm soát dòng thông tin trong chuỗi. Mô hình này có khả năng ghi nhớ dài hạn và thường được sử dụng trong các bài toán phân tích hành vi.

- Long Short-Term Memory (LSTM) là một dạng đặc biệt của Recurrent Neural Network (RNN), được trang bị các cơ chế cổng:

- Cell state (bộ nhớ dài hạn): Như một "băng chuyền" chuyên chở thông tin xuyên suốt chuỗi thời gian.

- 3 cổng điều khiển: Quên (Forget Gate), Đầu vào (Input Gate), Đầu ra (Output Gate).
- Kiến trúc 3 cổng:
 - Forget Gate (f_t): Quyết định phần nào của cell state cũ (C_{t-1}) cần bị loại bỏ, phần nào sẽ được giữ lại.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (2.33)$$

Hàm sigmoid cho đầu ra giá trị từ $[0, 1]$: 1 = “giữ nguyên”, 0 = “xóa hoàn toàn”.

- Input Gate & Candidate Cell State

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (2.34)$$

$$\tilde{C} = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (2.35)$$

Input Gate (i_t): Lọc thông tin mới từ đầu vào (x_t) để lưu trữ vào cell state.

Candidate Cell State ($\tilde{C}$): Tạo ra giá trị đề xuất cập nhật cho cell state, được “nén” trong khoảng $[-1, 1]$ bởi hàm tanh.

- Cell State Mới

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C} \quad (2.36)$$

Phần $f_t \odot C_{t-1}$ “giữ lại” thông tin cũ theo tỷ lệ do forget gate quyết định.

Phần $i_t \odot \tilde{C}$ bổ sung thông tin mới, đảm bảo rằng chỉ dữ liệu có giá trị được lưu giữ.

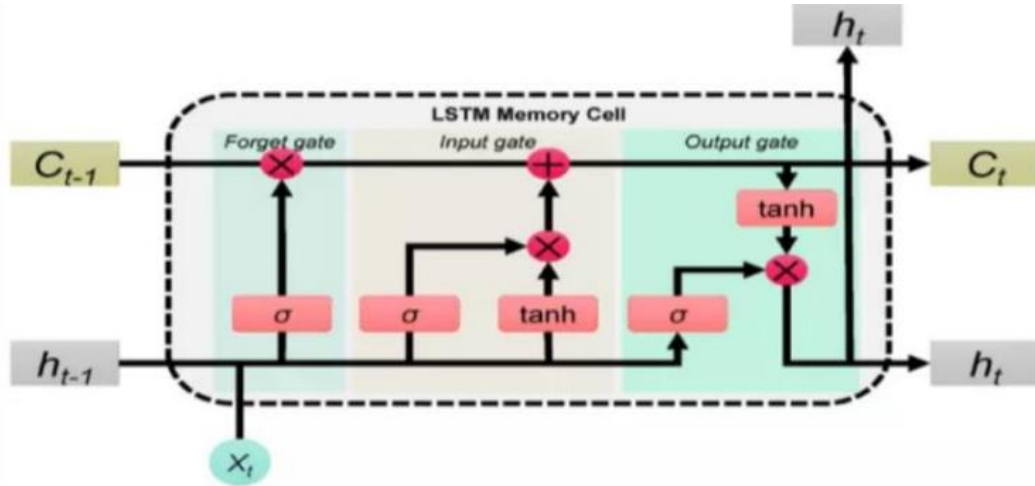
- Output Gate & Hidden State

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (2.37)$$

$$h_t = o_t \odot \tanh(C_t) \quad (2.38)$$

Output Gate (o_t): Quyết định phần nào của cell state sẽ được “trích xuất” ra làm hidden state.

Hidden State (h_t): Kết quả cuối cùng được đưa đi làm đầu ra cho bước thời gian hiện tại, sau khi cell state được “đi qua” hàm tanh.



(Nguồn: Viblo)

Hình 2.5 Mô hình LSTM

2.5.3. Bi-directional Long Short-Term Memory (BiLSTM)

BiLSTM mở rộng từ LSTM bằng cách sử dụng hai nhánh mạng LSTM hoạt động song song theo hai chiều thời gian: tiến và lùi. Nhờ đó, mô hình có thể khai thác ngữ cảnh từ cả quá khứ và tương lai tại mỗi thời điểm, từ đó nâng cao độ chính xác trong nhận diện hành vi.

Tại mỗi thời điểm t , hai trạng thái ẩn từ hai hướng được tính như sau:

- Tính trạng thái ẩn nhánh truyền xuôi (Forward):

Lớp này xử lý chuỗi từ quá khứ đến tương lai ($t = 1 \rightarrow T$).

$$\vec{h}_t = \text{LSTMf}(x_t, \vec{h}_{t-1}) \quad (2.39)$$

- Tính trạng thái ẩn nhánh truyền ngược (Backward):

Lớp này xử lý chuỗi từ tương lai về quá khứ ($t = T \rightarrow 1$).

$$\overleftarrow{h}_t = \text{LSTMb}(x_t, \overleftarrow{h}_{t+1}) \quad (2.40)$$

- Kết hợp trạng thái ẩn (Output / Combined Hidden State):

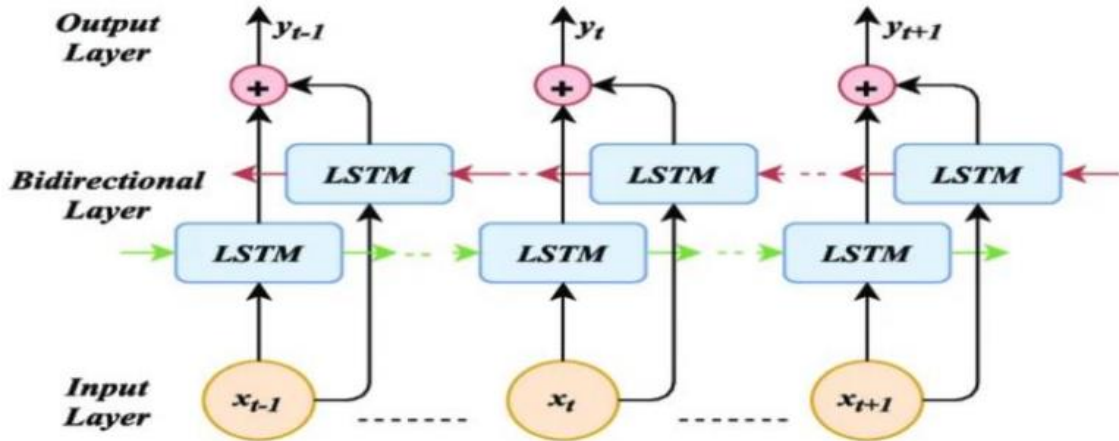
Đầu ra của mạng BiLSTM tại thời điểm t là sự kết hợp (nối vector) của cả hai trạng thái ẩn xuôi và ngược, giúp mô hình nắm bắt ngữ cảnh từ cả hai phía.

$$h_t = [\vec{h}_t; \overleftarrow{h}_t] \quad (2.41)$$

Trong đó:

- x_t : Vector dữ liệu đầu vào tại thời điểm t
- $\vec{h}_t$: Vector trạng thái ẩn của nhánh truyền xuôi tại thời điểm t
- $\overleftarrow{h}_t$: Vector trạng thái ẩn của nhánh truyền ngược tại thời điểm t
- $\vec{h}_{t-1}$: Vector trạng thái ẩn của nhánh truyền xuôi ở bước thời gian trước đó ($t - 1$).
- $\overleftarrow{h}_{t+1}$: Vector trạng thái ẩn của nhánh truyền ngược ở bước thời gian tiếp theo ($t + 1$).

- $\overrightarrow{LSTM}, \overleftarrow{LSTM}$: Các hàm tính toán tiêu chuẩn của một cell LSTM (bao gồm các cổng input, forget, output và cell state) cho hai nhánh độc lập.
- h_t : Vector trạng thái ẩn tổng hợp cuối cùng tại thời điểm t
- Dấu $[;]$: Phép toán nối vector (Concatenation). Nếu $\vec{h}_t$ và $\overleftarrow{h}_t$ đều có kích thước d , thì h_t sẽ có kích thước $2d$



(Nguồn: Viblo)

Hình 2.6 Mô hình BiLSTM

2.6. Mô hình trích xuất khung xương

Trong bài toán phát hiện hành vi té ngã, việc nhận diện chính xác tư thế và chuyển động của cơ thể người đóng vai trò cốt lõi. Thay vì xử lý trực tiếp trên toàn bộ điểm ảnh của video - vốn đòi hỏi tài nguyên tính toán khổng lồ và dễ bị nhiễu bởi bối cảnh, đề tài tiếp cận theo hướng trích xuất khung xương. Dữ liệu chuỗi tọa độ các khớp xương sau khi được trích xuất sẽ được sử dụng làm đầu vào cho mạng BiLSTM để mô hình hóa sự phụ thuộc theo thời gian nhằm phân loại hành vi té ngã.

2.6.1 Mô hình Yolo-pose

- Sự phát triển từ yolo sang yolo-pose

YOLO (You Only Look Once) ban đầu được biết đến là một mô hình học sâu một giai đoạn (one-stage) tối ưu cho bài toán phát hiện đối tượng (Object Detection). Đặc điểm nổi bật của YOLO là khả năng chia bức ảnh thành các lưới (grid) và dự đoán trực tiếp các hộp giới hạn (bounding boxes) cùng với xác suất của lớp đối tượng trong một lần truyền duy nhất (forward pass). Điều này giúp YOLO đạt tốc độ xử lý thời gian thực (real-time).

Theo thời gian, để đáp ứng các bài toán thị giác máy tính phức tạp hơn, kiến trúc YOLO đã được mở rộng thành nhiều dạng khác nhau, tiêu biểu như:

- YOLO-Det: Dạng cơ bản dùng để phát hiện đối tượng (Bounding box).
- YOLO-Seg: Mở rộng thêm nhánh phân vùng thực thể (Instance Segmentation) để tạo mask cho đối tượng.
- YOLO-Pose: Được thiết kế chuyên biệt cho bài toán ước lượng tư thế người (Human Pose Estimation).

Thay vì sử dụng phương pháp từ trên xuống (Top-down) truyền thống - tức là phải dùng một mạng phát hiện người trước, sau đó mới cắt ảnh người đưa vào mạng thứ hai để tìm khớp xương (rất chậm khi có nhiều người), YOLO-Pose hoạt động theo phương pháp ước lượng một giai đoạn (Single-stage). Mạng không chỉ dự đoán tọa độ hộp giới hạn bao quanh người mà còn dự đoán song song tọa độ (x, y) và độ tin cậy (confidence) của 17 điểm đặc trưng (keypoints) tương ứng với 17 khớp xương trên cơ thể người.

- Sự phù hợp Yolo-pose với đề tài té ngã

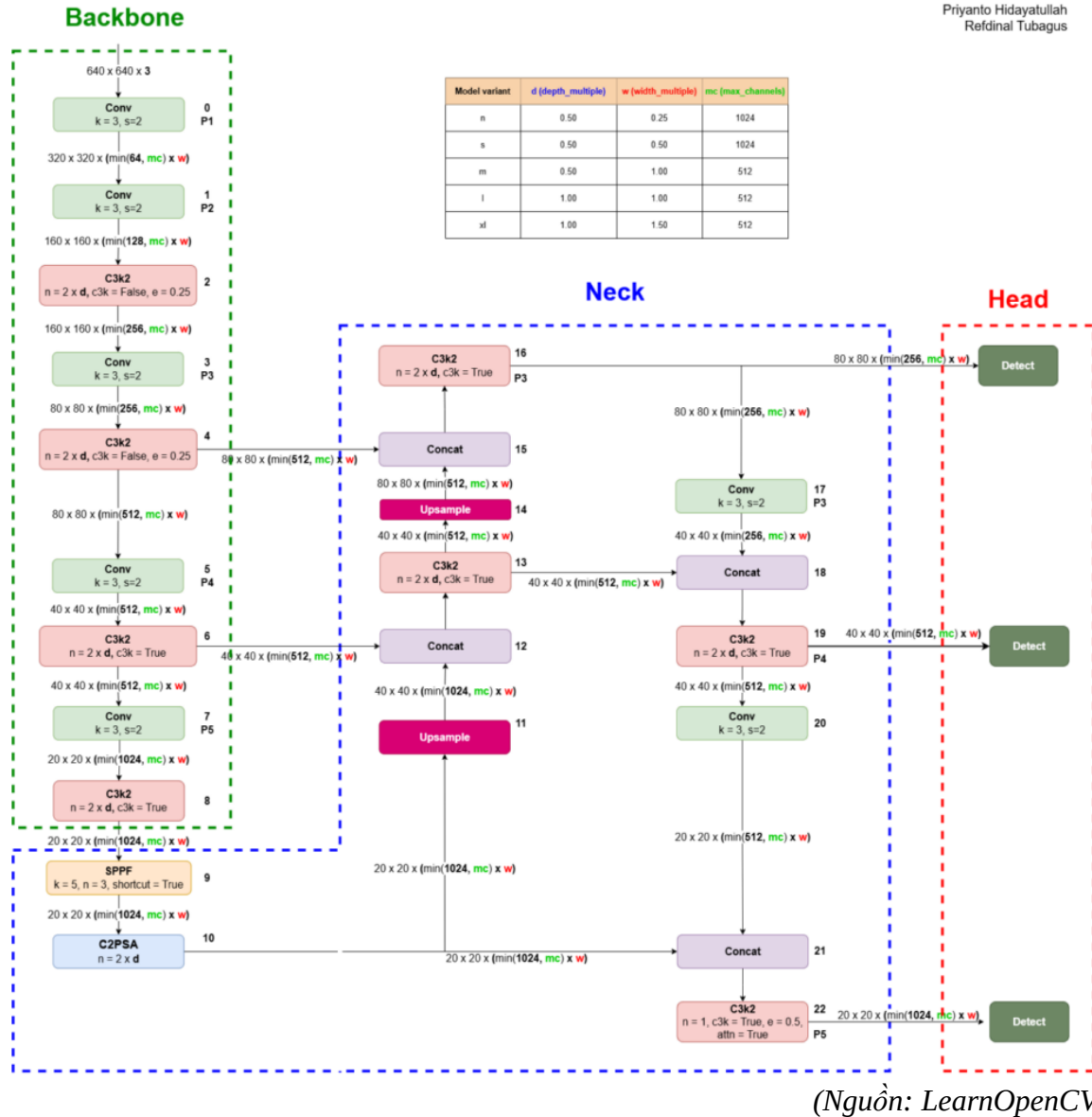
Việc lựa chọn YOLO-Pose làm bộ trích xuất đặc trưng không gian cho mạng BiLSTM phía sau mang lại những ưu điểm vượt trội:

- Tốc độ xử lý thời gian thực: Trong bối cảnh cảnh báo té ngã (đặc biệt cho người già), độ trễ tính bằng mili-giây là yếu tố sống còn. YOLO-Pose giải quyết bài toán ước lượng tư thế trong một lần quét mạng (one-pass), đáp ứng hoàn hảo yêu cầu real-time.
- Độ chính xác và khả năng chống che khuất (Robustness to Occlusion): Khi té ngã, các bộ phận cơ thể thường bị che khuất lẫn nhau hoặc bị che bởi đồ vật (bàn, ghế). Thuật toán Regression trực tiếp của YOLO-Pose kết hợp với thông tin ngữ cảnh toàn cục giúp mô hình vẫn dự đoán được vị trí khớp xương bị khuất.
- Hỗ trợ môi trường đa đối tượng: Nhờ kế thừa khả năng phát hiện đa đối tượng của YOLO, YOLO-Pose không bị giảm tốc độ tuyến tính khi số lượng người trong khung hình tăng lên, cho phép giám sát té ngã trong môi trường công cộng.

2.6.2. Mô hình Yolov26-pose

YOLOv26-pose là phiên bản kiến trúc nâng cao, được tinh chỉnh tối ưu hóa sự cân bằng giữa tốc độ và độ chính xác để phù hợp cho việc trích xuất khung xương trên đa nền tảng, kể cả các thiết bị biên. Vì vậy trong khuôn khổ đề tài này sẽ được áp dụng Yolov26-pose để trích xuất khung xương.

- Kiến trúc mạng Yolov26-pose



Hình 2.7 Kiến trúc mạng YOLOv26-Pose

Kiến trúc mạng của YOLOv26-pose kế thừa sự ưu việt của dòng YOLOv26 cơ bản, nhưng được thiết kế lại phần đầu ra Head để phục vụ song song hai tác vụ: định vị người và phát hiện các khớp xương. Cụ thể kiến trúc gồm 3 phần chính:

- Backbone: có nhiệm vụ trích xuất các đặc trưng hình ảnh từ mức độ thấp (cạnh, góc) đến mức độ cao (hình khối, bộ phận cơ thể người). Theo sơ đồ kiến trúc YOLOv26, Backbone hoạt động theo cơ chế sau:
 - o Các khối tích chập (Conv) giảm kích thước: Đầu vào (ảnh 3 kênh) đi qua các khối Conv với kích thước kernel $k=3$ và bước trượt $s=2$. Việc sử dụng $s=2$ giúp giảm một nửa độ phân giải

- không gian của bản đồ đặc trưng (feature map) sau mỗi khối, giúp tăng trường nhìn (receptive field) và giảm chi phí tính toán.
- Mạng C3k2: Đây là khối trích xuất đặc trưng cốt lõi tạo ra các đặc trưng có độ trừu tượng cao, được cấu hình qua các tham số n (số lần lặp), $c3k$ và e .
 - Kết nối đa cấp độ: Backbone không chỉ tạo ra một đầu ra duy nhất mà trích xuất đặc trưng ở 3 cấp độ khác nhau (tại các khối C3k2 số 4, 6 và 8). Ba luồng đặc trưng này sẽ được đẩy trực tiếp sang phần Neck, giúp mô hình giữ lại được cả chi tiết không gian (để xác định chính xác điểm khớp) và thông tin ngữ nghĩa (để nhận biết đó là người).
 - Neck: Trong bài toán giám sát té ngã, con người có thể đứng gần camera (kích thước lớn) hoặc ngã ở xa góc phòng (kích thước nhỏ). Phần Neck của YOLOv26 được thiết kế để giải quyết bài toán đa tỷ lệ này:
 - Khối SPPF (Spatial Pyramid Pooling Fast) mang tính đột phá: Được đặt ở đầu Neck (khối số 9), mô-đun này cho phép gom cụm (pooling) ở nhiều kích thước khác nhau. Điểm nâng cấp cốt lõi của YOLOv26 là thêm một Shortcut (kết nối tắt) vào SPPF, cho phép tích hợp trực tiếp đầu vào với đầu ra. Điều này giúp dòng chảy gradient mượt mà hơn và ổn định quá trình tối ưu hóa các biểu diễn ngữ nghĩa bậc cao.
 - Cơ chế Self-Attention qua khối C2PSA: Nằm sau SPPF, khối C2PSA áp dụng cơ chế tự chú ý (Self-attention) để tích hợp khả năng mô hình hóa toàn cục (global modeling). Nhờ hiểu được "ngữ cảnh toàn cục", mạng có thể suy luận logic vị trí các khớp xương bị che khuất (ví dụ: tay bị che lấp khi người ngã sấp).
 - Mạng FPN-PAN (Trộn đặc trưng): Sử dụng các khối Upsample (dùng phương pháp nội suy nearest neighbor để tăng độ phân giải) kết hợp với khối Concat (để gộp các bản đồ đặc trưng, giữ nguyên độ phân giải nhưng tăng số kênh).
 - Tối ưu hóa khối C3k2 cuối cùng (Khối số 22): Tại khối C3k2 cuối cùng trước khi đưa vào Head, YOLOv26 cố định tham số lặp $n=1$ để tránh lãng phí tài nguyên tính toán, đồng thời kích hoạt cơ chế attention ($attn=True$) bên trong module PSABlock.

Điều này giúp nâng cao mô hình hóa ngữ cảnh toàn cục mà không làm tăng độ trễ.

- Head: Mạng dự đoán đa tác vụ NMS-Free (Pose Head), đây là nơi chứa nhiều cải tiến sâu sắc nhất của YOLOv26 so với các thế hệ trước. Pose Head lấy đầu vào từ 3 khối C3k2 ở Neck (khối 16 cho vật thể nhỏ, 19 cho vật thể vừa, 22 cho vật thể lớn). Tại đây, mạng thực hiện dự đoán song song hai luồng:
 - Nhánh Bounding Box (Định vị người - NMS-Free):
 - Loại bỏ DFL (Distribution Focal Loss): Khác với các bản cũ dùng DFL dự đoán phân phối hộp giới hạn gây tốn kém tính toán, YOLOv26 chuyển sang dự đoán tọa độ trực tiếp (explicit box regression), giúp suy luận nhanh hơn.
 - Huấn luyện NMS-Free End-to-End: Trong quá trình huấn luyện, YOLOv26 sử dụng chiến lược Gán nhãn kép (Dual Assignment): kết hợp nhánh "một-nhiều" (cung cấp tín hiệu giám sát mạnh mẽ giúp hội tụ nhanh) và nhánh "một-một" (gắn mỗi người với một dự đoán duy nhất). Kết hợp với thuật toán ProGLoss (tự động dịch chuyển trọng số từ nhánh "một-nhiều" sang "một-một" theo tiến trình huấn luyện).
 - Khi suy luận thực tế (inference), nhánh "một-nhiều" bị loại bỏ, mô hình chỉ sử dụng nhánh "một-một" bằng phương pháp Top-K score-based inference (chọn các dự đoán có điểm phân loại cao nhất). Nhờ đó, loại bỏ hoàn toàn quá trình triệt tiêu cực đại cục bộ (NMS) rườm rà, giải quyết triệt để vấn đề độ trễ khi có nhiều người chồng chéo lấp nhau trong khung hình.
 - Gán nhãn STAL (Small-Target-Aware Label Assignment): Đảm bảo gán ít nhất 4 anchor cho các mục tiêu bé hơn 8x8 pixels. Cực kỳ hữu ích để không bỏ sót người bị té ngã ở khoảng cách rất xa so với camera.
 - Tối ưu hóa MuSGD: Sử dụng bộ tối ưu hóa lai giữa Muon và SGD giúp tham số hội tụ nhanh, ổn định quá trình huấn luyện.
 - Nhánh Keypoints (Trích xuất 17 điểm khớp xương): Thay vì các phương pháp hồi quy truyền thống dễ bị nhiễu, nhánh Pose của YOLOv26 tích hợp phương pháp Residual Log-Likelihood

Estimation (RLE). Phương pháp này tối ưu hóa việc học các phân phối xác suất sai số của tọa độ (x, y) của 17 điểm khớp xương. Kết quả là mạng có khả năng định vị keypoints với độ chính xác cao ngay cả trong các trường hợp tư thế biến dạng mạnh (như đang gập người tẻ ngã), đồng thời quy trình giải mã (decoding) được tối ưu hóa ở mức thấp nhất giúp mô hình duy trì tốc độ khung hình (FPS) cực cao, hoàn thành xuất sắc vai trò bộ trích xuất đặc trưng theo thời gian thực cho mạng BiLSTM.

- Hàm mất mát:

Do YOLOv26-pose là một mạng học đa tác vụ, nó phải đồng thời giải quyết hai bài toán: (1) Phát hiện người và (2) Ước lượng vị trí 17 điểm khớp xương. Do đó, tổng hàm mất mát của mô hình là sự kết hợp có trọng số của các hàm mất mát thành phần. Đặc biệt, dựa trên những cải tiến của phiên bản YOLOv26, tổng hàm mất mát L_{total} được định nghĩa như sau:

$$L_{total} = \lambda_{box}L_{box} + \lambda_{cls}L_{cls} + \lambda_{pose}L_{pose} + L_{Prog} \quad (2.42)$$

Trong đó, $\lambda_{box}, \lambda_{cls}, \lambda_{pose}$ là các siêu tham số (hyperparameters) dùng để cân bằng tầm quan trọng của từng nhánh.

• **Hàm mất mát hộp giới hạn (Bounding Box Loss - L_{box})**

Điểm khác biệt lớn nhất của YOLOv26 so với các phiên bản trước (như YOLOv8) là loại bỏ hoàn toàn hàm mất mát Distribution Focal Loss (DFL). Thay vì dự đoán phân phối xác suất của các cạnh hộp giới hạn gây tốn kém tính toán, YOLOv26 quay trở lại tối ưu hóa trực tiếp tọa độ hộp (Explicit Box Regression).

Hàm mất mát hộp giới hạn thường sử dụng CIoU (Complete Intersection over Union) để đánh giá độ trùm lấp giữa hộp dự đoán và hộp thực tế (ground truth):

$$L_{box} = 1 - CIoU = 1 - \left(IoU - \frac{\rho^2(b, b^{gt})}{c^2} - \alpha v \right) \quad (2.43)$$

Trong đó:

- IoU: Tỷ lệ diện tích giao nhau trên diện tích hợp nhất của 2 hộp.
- $\rho^2(b, b^{gt})$: Khoảng cách Euclidean bình phương giữa tâm hộp dự đoán và hộp thực tế.
- c: Đường chéo của hộp nhỏ nhất bao quanh cả 2 hộp.
- α, v : Các tham số đo lường sự nhất quán về tỷ lệ khung hình (aspect ratio).

→ Ý nghĩa trong bài toán: Giúp mạng định vị chính xác toàn bộ cơ thể người ngã dù tư thế co cụm hay đang rộng.

- **Hàm mất mát phân loại (Classification Loss – Lcls)**

Hàm này đo lường sai số trong việc dự đoán nhãn lớp (trong đồ án này lớp chủ yếu là "Person" - Người). Mô hình sử dụng hàm Binary Cross Entropy (BCE):

$$L_{cls} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)] \quad (2.44)$$

Trong đó: y_i là nhãn thực tế (1 nếu là người, 0 nếu không phải), p_i là xác suất mô hình dự đoán.

- **Hàm mất mát điểm khớp xương (Pose/Keypoint Loss - Lpose)**

Đây là trái tim của mạng YOLOv26-pose dùng để trích xuất dữ liệu cho BiLSTM. Hàm mất mát này dựa trên chỉ số OKS (Object Keypoint Similarity) kết hợp với kỹ thuật RLE (Residual Log-Likelihood Estimation).

- Chỉ số OKS: Đo lường độ tương đồng của từng khớp:

$$OKS_i = \exp\left(\frac{-d_i^2}{2s^2k_i^2}\right) \quad (2.45)$$

Trong đó: d_i là khoảng cách Euclidean giữa điểm dự đoán và điểm thực tế; s là diện tích bao quanh người (scale); k_i là hằng số đặc trưng cho từng loại khớp (ví dụ: khớp hông có k_i lớn hơn so với mắt vì hông dễ định vị hơn).

- Tích hợp RLE (Residual Log-Likelihood Estimation): Thay vì chỉ dùng hồi quy L1/L2 thông thường, YOLOv26-pose áp dụng RLE để học phân phối sai số của các tọa độ điểm khớp (x, y). Nếu mô hình dự đoán một điểm khớp ở vị trí bị che khuất (ví dụ tay bị cơ thể che khuất khi ngã), RLE sẽ giúp mạng dự đoán thêm độ lệch chuẩn (độ không chắc chắn) của điểm đó. Từ đó hàm loss phạt ít hơn đối với các điểm bị che khuất, giúp mô hình linh hoạt và ổn định hơn.

- **Cân bằng mất mát động (Progressive Loss Balancing - L Prog) và Gán nhãn kép**

Như đã đề cập ở kiến trúc End-to-End NMS-Free, YOLOv26 sử dụng cơ chế Gán nhãn kép (Dual Assignment) trong quá trình huấn luyện: một nhánh gán "một-nhiều" (one-to-many) và một nhánh gán "một-một" (one-to-one). Hàm L Prog được thiết kế để thay đổi trọng số động giữa hai nhánh theo thời gian (epoch) t :

$$L_{prog}(t) = \alpha(t) \cdot Loss_{(one-to-many)} + (1 - \alpha(t)) \cdot Loss_{(one-to-one)} \quad (2.46)$$

Trong đó: $\alpha(t)$ là trọng số thay đổi giảm dần theo thời gian huấn luyện.

→ Ý nghĩa:

- Ở các epoch đầu ($\alpha(t)$ lớn): Mạng dùng tín hiệu "một-nhiều" để học nhanh, tìm ra càng nhiều dự đoán có thể càng tốt (tăng Recall).
- Ở các epoch cuối ($\alpha(t)$ tiến về 0): Trọng số dịch chuyển sang "một-một". Mô hình ép buộc mỗi người bị ngã chỉ xuất ra duy nhất một hộp bounding box chính xác nhất. Điều này cho phép khi mạng được đem đi chạy thực tế (Inference), ta có thể bỏ hẳn thuật toán NMS mà không bị tình trạng vẽ đè nhiều khung hình lên một người, tối ưu hóa triệt để tốc độ nhận diện.

2.7. Các chỉ số đánh giá hiệu năng

Trong các bài toán phân loại và phát hiện đối tượng, việc đánh giá mô hình thông qua các chỉ số hiệu năng là vô cùng quan trọng nhằm đảm bảo mô hình hoạt động chính xác và đáng tin cậy. Các chỉ số phổ biến nhất bao gồm True Positive (TP), True Negative (TN), False Positive (FP) và False Negative (FN). Đây là nền tảng để xây dựng các chỉ số hiệu suất khác như độ chính xác, precision, recall và mAP.

True Positive (TP) là số lượng trường hợp mà mô hình dự đoán đúng là thuộc lớp Positive (dương). Ngược lại, True Negative (TN) là số lượng trường hợp mà mô hình dự đoán đúng là thuộc lớp Negative (âm). False Positive (FP) là những trường hợp mà mô hình dự đoán sai, tức là dự đoán là Positive trong khi thực tế là Negative. Cuối cùng, False Negative (FN) là khi mô hình dự đoán một mẫu là Negative trong khi thực tế nó lại thuộc lớp Positive.

Một trong những chỉ số phổ biến nhất là độ chính xác (Accuracy). Chỉ số này đo lường tỷ lệ tổng số dự đoán đúng (bao gồm cả TP và TN) trên toàn bộ tập dữ liệu. Nó phản ánh mức độ chính xác tổng thể của mô hình, đặc biệt hữu ích khi các lớp dữ liệu cân bằng. Tuy nhiên, trong các trường hợp mất cân bằng lớp, độ chính xác không phản ánh đầy đủ hiệu năng mô hình.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (2.47)$$

trong đó TP là các trường hợp mô hình dự đoán đúng lớp 1, còn TN là dự đoán đúng lớp 0. FP xảy ra khi mô hình dự đoán là lớp 1 nhưng thực tế là lớp 0, ngược lại, FN là khi mô hình dự đoán là lớp 0 nhưng thực tế lại thuộc lớp 1.

Precision, hay độ chính xác theo chiều dương, trả lời cho câu hỏi: trong số các mẫu được mô hình dự đoán là Positive, có bao nhiêu phần trăm là đúng. Precision cao cho thấy mô hình ít mắc lỗi dạng False Positive – điều này đặc biệt quan trọng trong các ứng dụng nhạy cảm như phát hiện gian lận, chẩn đoán y tế hoặc phân loại hồ sơ xấu (BAD). Công thức của precision là tỷ lệ giữa TP trên tổng số TP và FP.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (2.48)$$

trong đó, TP (True Positive) là số mẫu thực tế thuộc lớp 1 và được dự đoán đúng, còn FP (False Positive) là số mẫu thực tế không phải lớp 1 nhưng lại bị dự đoán nhầm thành lớp 1.

Recall, hay còn gọi là độ bao phủ, đo lường khả năng của mô hình trong việc phát hiện đúng tất cả các trường hợp Positive. Nó được tính bằng tỷ lệ giữa TP trên tổng số TP và FN. Recall cao cho thấy mô hình ít bỏ sót các trường hợp dương tính, rất quan trọng trong các ứng dụng như phát hiện ung thư, giám sát an ninh, v.v.

$$\text{Recall} = \frac{TP}{TP + FN} \quad (2.49)$$

Trong đó, TP (True Positive) là số mẫu thuộc lớp 1 được mô hình dự đoán đúng, còn FN (False Negative) là số mẫu thực tế là lớp 1 nhưng mô hình lại dự đoán nhầm thành lớp 0.

Các bài toán phát hiện đối tượng trong ảnh còn có các chỉ số đánh giá đặc thù riêng. Đầu tiên là một chỉ số đặc biệt quan trọng trong các bài toán phát hiện đối tượng (object detection) là IoU – Intersection over Union. IoU đo mức độ trùng khớp giữa vùng dự đoán (bounding box) và vùng thực tế (ground truth). Chỉ số này được tính bằng diện tích phần giao giữa hai bounding box

chia cho diện tích phần hợp của chúng. Mô hình được xem là phát hiện đúng khi IoU vượt qua một ngưỡng nhất định (thường là 0.5). IoU là thước đo nền tảng cho các chỉ số nâng cao như mAP.

$$\text{IoU} = \frac{|B_p \cap B_{gt}|}{|B_p \cup B_{gt}|} \quad (2.50)$$

Trong đó, B_p là hộp bao dự đoán từ mô hình, còn B_{gt} là hộp bao thực tế. Dấu $\cap$ thể hiện phần giao nhau giữa hai hộp, còn $\cup$ là phần hợp, tức là toàn bộ vùng được bao phủ bởi ít nhất một trong hai hộp. Các biểu thức $B_p \cap B_{gt}$ và $B_p \cup B_{gt}$ lần lượt đại diện cho vùng chung và vùng tổng giữa hộp dự đoán và hộp thực tế.

Tiếp theo là mAP – mean Average Precision, một độ đo chuẩn để đánh giá hiệu suất của các mô hình phát hiện đối tượng như Faster R-CNN, SSD hay YOLO. mAP được tính bằng trung bình cộng của các chỉ số Average Precision (AP) cho từng lớp đối tượng trong tập dữ liệu. Chỉ số này kết hợp cả precision và recall ở nhiều ngưỡng IoU khác nhau, giúp mô hình được đánh giá toàn diện hơn về khả năng nhận diện nhiều đối tượng.

Cuối cùng, một chỉ số hiệu năng không thể thiếu là FPS – Frames per Second, hay số khung hình trên giây. FPS đo tốc độ xử lý của mô hình trong thực tế, cho biết mô hình có thể xử lý bao nhiêu khung hình ảnh mỗi giây. Mô hình có FPS cao sẽ phù hợp hơn với các ứng dụng thời gian thực như giám sát giao thông, robot tự hành hoặc các hệ thống theo dõi video trực tiếp.

$$\text{FPS} = \frac{N}{T} \quad (2.51)$$

trong đó N là tổng số khung hình được xử lý và T là tổng thời gian thực hiện (tính bằng giây).

Như vậy, chương này đã trình bày tổng quan về các mô hình sử dụng, khái niệm cơ bản và các chỉ số đánh giá quan trọng như Accuracy, Precision, Recall, IoU, mAP và FPS – những thước đo then chốt giúp đánh giá hiệu quả của mô hình trong các bài toán phân loại và phát hiện đối tượng. Đây là cơ sở để hiểu rõ chất lượng mô hình trong thực tế. Tiếp theo, Chương 3 sẽ đi sâu vào phương pháp giải quyết bài toán, trình bày chi tiết quy trình xử lý dữ liệu, lựa

chọn mô hình, các bước huấn luyện và đánh giá. Những nội dung này đóng vai trò cốt lõi trong việc hiện thực hóa hệ thống từ ý tưởng đến ứng dụng thực tiễn.

2.8. Quy trình thực hiện

Hệ thống phát hiện té ngã được xây dựng dựa trên sự kết hợp giữa YOLOv26-Pose và BiLSTM. YOLOv26-Pose được sử dụng để trích xuất các điểm khớp cơ thể (keypoints) từ video, trong khi BiLSTM học chuỗi chuyển động theo thời gian để phân loại hành vi té ngã.

Dữ liệu đầu vào là chuỗi khung xương thay vì ảnh RGB, giúp tăng khả năng tổng quát hóa và đảm bảo quyền riêng tư. Mô hình được huấn luyện với 100 epochs, sử dụng Early Stopping và mixed precision để tối ưu hiệu năng.

Hiệu suất được đánh giá qua các chỉ số Accuracy, Precision, Recall, AUC, trong đó ưu tiên Recall để tránh bỏ sót té ngã. Hệ thống cũng được kiểm tra trong các điều kiện thực tế như che khuất và nhiễu chuyển động.

Quá trình tối ưu bao gồm điều chỉnh learning rate và sử dụng ReduceLROnPlateau, kết hợp kỹ thuật sliding window và đặc trưng động học để cải thiện độ chính xác. Mô hình tốt nhất được triển khai dưới dạng ONNX và tích hợp vào hệ thống Web, cho phép phát hiện té ngã theo thời gian thực và đưa ra cảnh báo kịp thời.

CHƯƠNG 3. THỰC NGHIỆM VÀ ĐÁNH GIÁ

3.1. Các kỹ thuật hỗ trợ

3.1.1. Ngôn ngữ sử dụng



(Nguồn: Internet)

Hình 3.1 Biểu tượng Python

Khi triển khai xây dựng hệ thống em lựa chọn Python là ngôn ngữ để thực hiện. Python là ngôn ngữ lập trình bậc cao, mã nguồn mở và đa nền tảng. Python sử dụng rộng rãi để phát triển các ứng dụng web, phát triển phần mềm, khoa học dữ liệu máy tính (ML). Với cú pháp đơn giản, khả năng mở rộng cao, cùng hệ sinh thái thư viện phong phú như PyTorch, OpenCV, NumPy, Python giúp đơn giản hóa quy trình từ xử lý dữ liệu, huấn luyện mô hình đến dự đoán và triển khai.

- Một số thư viện Python phổ biến được sử dụng trong đề tài:
 - FastAPI & Uvicorn: FastAPI là một web framework hiện đại, hiệu năng cao được sử dụng để xây dựng các API (như tải video lên, lấy lịch sử) và đặc biệt là quản lý các kết nối WebSocket (@app.websocket). Uvicorn đóng vai trò là máy chủ ASGI để vận hành ứng dụng FastAPI.
 - Ultralytics (YOLO): Thư viện chính thức để khởi tạo và suy diễn mô hình YOLO11-pose (yolo11n-pose.pt), phục vụ việc trích xuất tọa độ 17 khớp xương người từ các khung hình camera hoặc video đầu vào.
 - ONNX Runtime: Thay vì sử dụng PyTorch hay TensorFlow nặng nề, hệ thống sử dụng ONNX Runtime (ort.InferenceSession) để chạy mô hình mạng nơ-ron hồi quy song hướng BiLSTM (best_bilstm_model.onnx). Việc chuyển đổi sang định dạng ONNX giúp tăng tốc độ suy diễn (inference) lên đáng kể, phù hợp cho bài toán thời gian thực.
 - OpenCV (cv2): Thư viện thị giác máy tính cốt lõi, chịu trách nhiệm đọc/ghi video, giải mã hình ảnh từ Base64, vẽ đồ họa (vẽ khung xương,

làm mờ bảo vệ quyền riêng tư) và mã hóa lại khung hình để gửi qua Web UI.

- NumPy & Joblib: NumPy được dùng để xử lý các ma trận vector đặc trưng, tính toán vận tốc và góc của các khớp xương. Joblib được dùng để tải đối tượng chuẩn hóa dữ liệu (scaler.pkl) trước khi đưa vào mạng BiLSTM.
- SQLite3: Hệ quản trị cơ sở dữ liệu quan hệ nhỏ gọn được tích hợp sẵn, dùng để lưu trữ lịch sử các sự kiện té ngã (thời gian, độ tin cậy, đường dẫn ảnh chụp).
- Requests & Asyncio/Threading: requests được dùng để giao tiếp với API của Telegram nhằm gửi tin nhắn và hình ảnh cảnh báo. Asyncio và ThreadPoolExecutor được ứng dụng để xử lý song song (chạy AI ở luồng nền) mà không làm nghẽn luồng truyền video của Web.

3.1.2. Môi trường sử dụng

Khác với quá trình huấn luyện mô hình thường đòi hỏi các nền tảng đám mây như Kaggle hay Google Colab, quá trình triển khai thực tế của hệ thống này được thực hiện trên môi trường máy tính cục bộ với cấu hình đáp ứng yêu cầu xử lý thời gian thực.

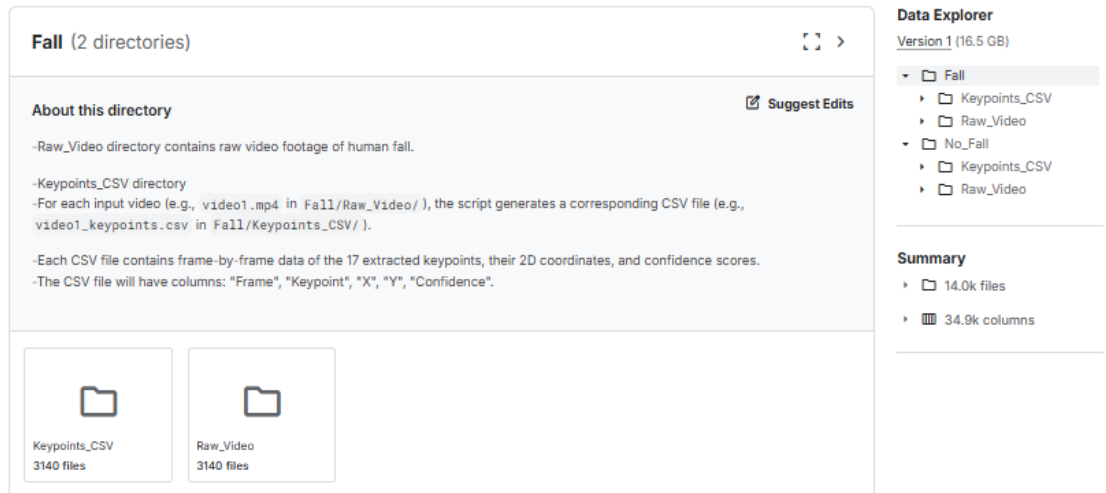
- Về phần cứng :
 - CPU: Vì xử lý đa luồng (ví dụ: Intel Core i5/i7 thế hệ 10 trở lên) để xử lý đa nhiệm giữa FastAPI, giải mã Video và chạy Backend.
 - RAM: Tối thiểu 8GB (khuyến nghị 16GB) để duy trì bộ đệm khung hình (buffer) và tải các mô hình học sâu.
 - GPU (Tùy chọn nhưng khuyến nghị): Card đồ họa NVIDIA (hỗ trợ CUDA) để tăng tốc cho YOLO11-pose và ONNX Runtime, giúp đạt tốc độ khung hình (FPS) cao nhất.
 - Thiết bị đầu vào: Camera (Webcam) kết nối trực tiếp hoặc thông qua IP.
- Về phần mềm:
 - Hệ điều hành: Đa nền tảng (Windows 10/11, Ubuntu 20.04+ hoặc macOS).
 - Môi trường thực thi: Python 3.9 trở lên.
 - Trình duyệt web: Google Chrome hoặc Microsoft Edge (yêu cầu hỗ trợ đầy đủ HTML5 và WebSocket để hiển thị giao diện Frontend).
- Kiến trúc vận hành: Khi khởi chạy, hệ thống đóng vai trò là một máy chủ Web (chạy tại địa chỉ 127.0.0.1:8000). Người dùng có thể truy cập qua

trình duyệt để thao tác trên 3 module chính: Giám sát camera trực tiếp ,Phân tích video ngoại tuyến và Xem lại lịch sử). Mọi tương tác video hai chiều giữa trình duyệt và mô hình AI đều được truyền tải theo thời gian thực thông qua giao thức WebSocket.

3.2. Dữ liệu thực nghiệm

3.2.1. Đặc điểm bộ dữ liệu

- 1) Nguồn gốc và cấu trúc dữ liệu:
 - Link bộ dữ liệu:
<https://www.kaggle.com/datasets/payutch/fall-videodataset?select=Fall>
 - Bộ dữ liệu được sử dụng để huấn luyện và đánh giá mô hình trong đề án là "Fall Video Dataset" trên Kaggle. Đây là một bộ dữ liệu quy mô lớn, được tổng hợp từ 3 nguồn dữ liệu video giám sát té ngã công khai uy tín, bao gồm: Fall Vision (Đại học Harvard), Video-Based Fall Detection Dataset (với 2017 chuỗi hành động), và Multiple cameras fall dataset. Việc sử dụng dữ liệu tổng hợp từ nhiều nguồn giúp mô hình có khả năng tổng quát hóa cao, thích nghi với nhiều góc quay, điều kiện ánh sáng và bối cảnh môi trường khác nhau.
 - Về mặt cấu trúc, bộ dữ liệu được chia làm 2 thư mục chính tương ứng với 2 nhãn phân loại:
 - Fall (Ngã): Chứa các chuỗi video ghi lại cảnh người bị té ngã.
 - No_Fall (Không ngã/Bình thường): Chứa các video mô tả các hoạt động sinh hoạt thường ngày (đi lại, ngồi xuống, cúi nhặt đồ,...). Đây là các hành động có tính gây nhầm lẫn (confounding events) cao, giúp kiểm tra khả năng chống báo động giả của mô hình.
 - Bên trong mỗi thư mục nhãn tiếp tục được chia thành 2 thư mục con: Raw_Video (Video thô gốc) và Keypoints_CSV (Dữ liệu khung xương dạng bảng). Trong phạm vi bài toán kết hợp YOLO-pose và BiLSTM, các tệp **.csv** là nguồn dữ liệu đầu vào trực tiếp cho mạng chuỗi thời gian. Mỗi tệp CSV lưu trữ thông tin trích xuất theo từng khung hình (frame-by-frame) của 17 điểm khớp xương (Keypoints) với cấu trúc các cột: Frame, Keypoint (Tên khớp), X (Tọa độ ngang), Y (Tọa độ dọc), và Confidence (Độ tin cậy của phép trích xuất)

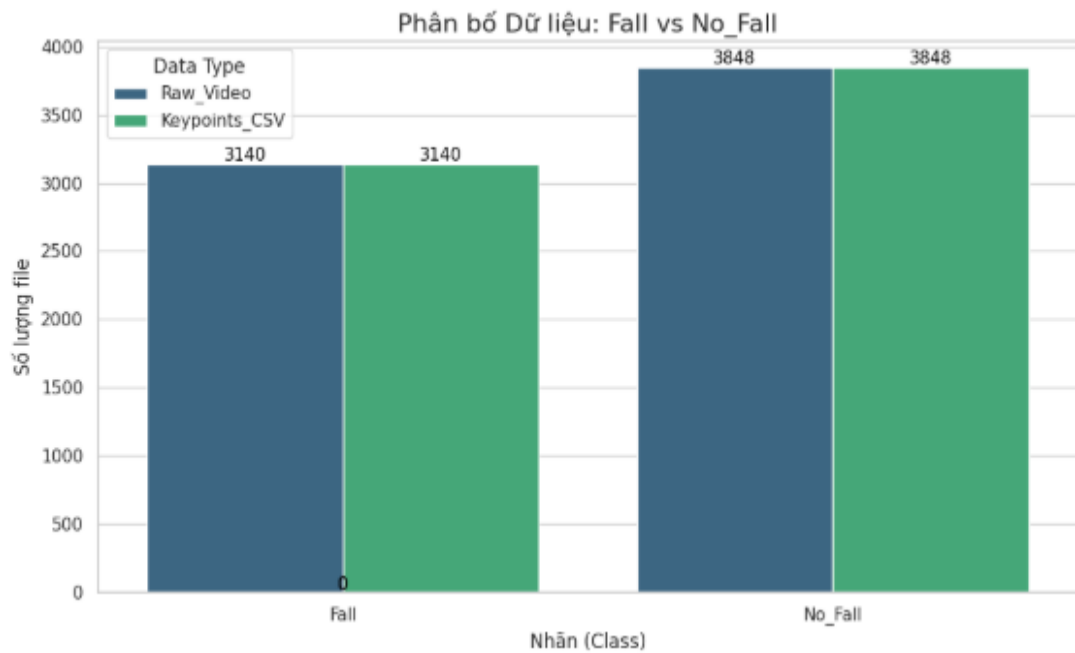


Hình 3.2 Tổng quan bộ dữ liệu

# Frame	Keypoint	# X	# Y	# Confidence
1 56	17 unique values	280 1.19k	145 657	0.09 1
1	Nose	425.469388961792	163.05227994918823	0.9989495873451233
1	Left Eye	423.16311836242676	148.1211519241333	0.995879755783081
1	Right Eye	422.3357677459717	147.76320576667786	0.9938367009162903
1	Left Ear	421.32777214050293	147.39112973213196	0.9947530031204224
1	Right Ear	420.2655601501465	147.78650879859924	0.9976077079772949
1	Left Shoulder	387.90696144104004	218.15772235393524	0.9984920024871826
1	Right Shoulder	335.44947624206543	208.86612117290497	0.9999769926071167
1	Left Elbow	385.64887046813965	307.48849511146545	0.09506028145551682
1	Right Elbow	338.4921455383301	310.2019786834717	0.987382709980011
1	Left Wrist	400.6897830963135	376.69650435447693	0.0880860760807991
1	Right Wrist	374.6978759765625	394.663302898407	0.9634082913398743
1	Left Hip	381.8585014343262	384.6080446243286	0.9986586570739746
1	Right Hip	340.07649421691895	387.19664454460144	0.9993925094604492
1	Left Knee	388.4187412261963	508.505083322525	0.7343020439147949
1	Right Knee	366.53417587280273	513.6194550991058	0.9861142635345459
1	Left Ankle	394.32437896728516	629.3761396408081	0.9123378396034241
1	Right Ankle	280.1809501647949	595.5323481559753	0.9877609014511108

Hình 3.3 Tổng quan file csv khung xương

2) Quy mô và tỷ lệ phân bố các nhãn



Hình 3.4 Biểu đồ cột phân bố tỷ lệ bộ dữ liệu

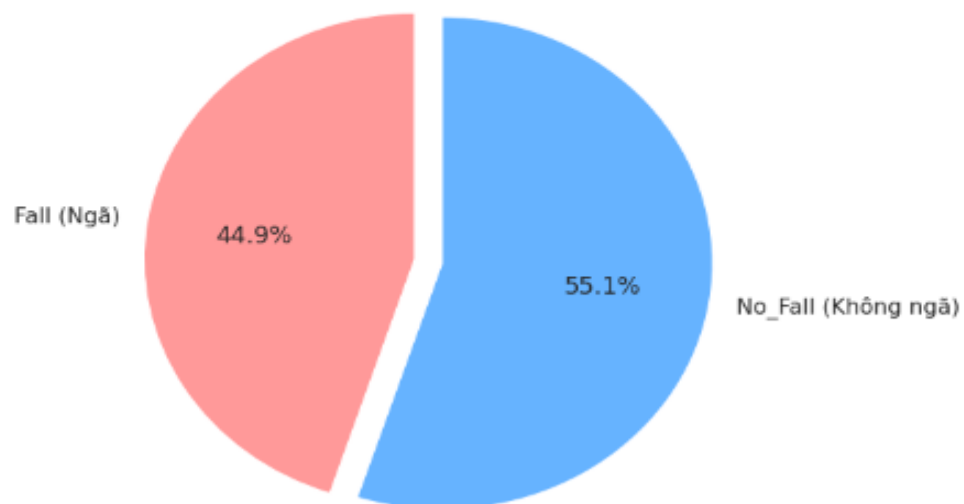
- Quá trình thống kê tự động trên tập dữ liệu cho thấy tổng số lượng mẫu thu thập được là 6.988 tệp CSV (tương ứng với 6.988 video). Cụ thể:
 - Số lượng mẫu nhãn Fall (Ngã): 3.140 tệp (chiếm 44.9%).
 - Số lượng mẫu nhãn No_Fall (Không ngã): 3.848 tệp (chiếm 55.1%).

Đang tìm kiếm file CSV...

✓ Đã tìm thấy: 3140 file Fall

✓ Đã tìm thấy: 3848 file No_Fall

Tỷ lệ phân bố dữ liệu (Class Balance)

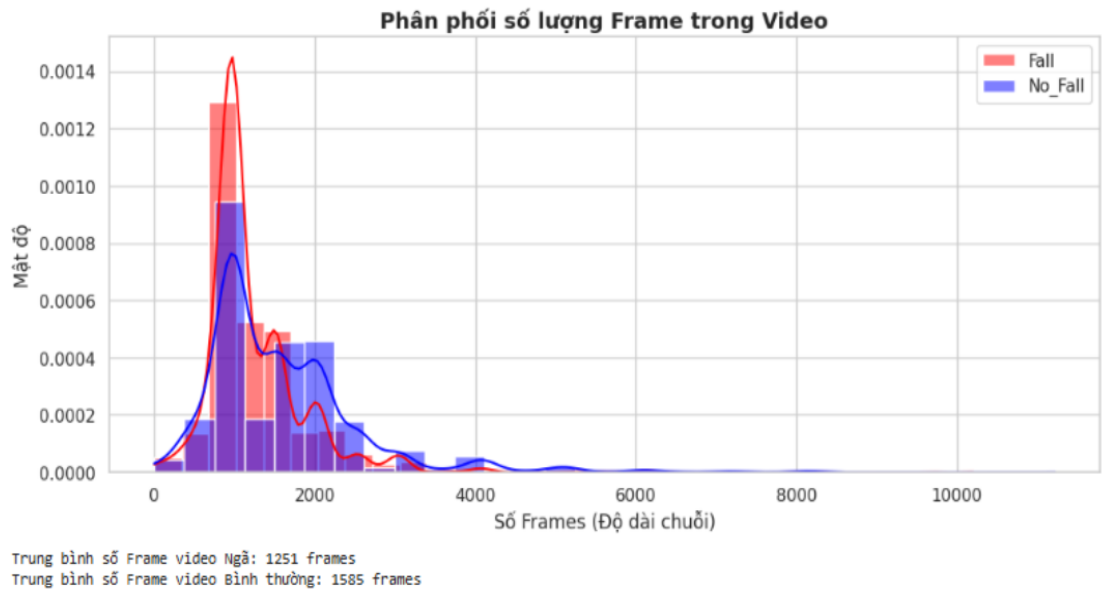


Hình 3.5 Biểu đồ tròn phân bố tỷ lệ bộ dữ liệu

→ Nhận xét: Tỷ lệ phân bố giữa hai lớp dữ liệu Ngã và Không Ngã đạt mức 44.9% - 55.1%. Đây là một tỷ lệ lý tưởng và khá cân bằng (balanced data). Mức độ chênh lệch nhỏ này giúp mạng BiLSTM trong quá trình huấn luyện không bị rơi vào tình trạng thiên lệch (bias) dự đoán ưu tiên cho lớp đa số, đảm bảo độ nhạy (Recall) cao khi phát hiện hành vi té ngã.

3) Phân phối độ dài chuỗi thời gian (Temporal Distribution)

- Bản chất của thuật toán BiLSTM là xử lý chuỗi thời gian, do đó độ dài của chuỗi (số lượng frame trong một video) ảnh hưởng trực tiếp đến kiến trúc mạng. Dựa trên biểu đồ phân phối mật độ:



Hình 3.6 Phân phối số lượng frame trong video

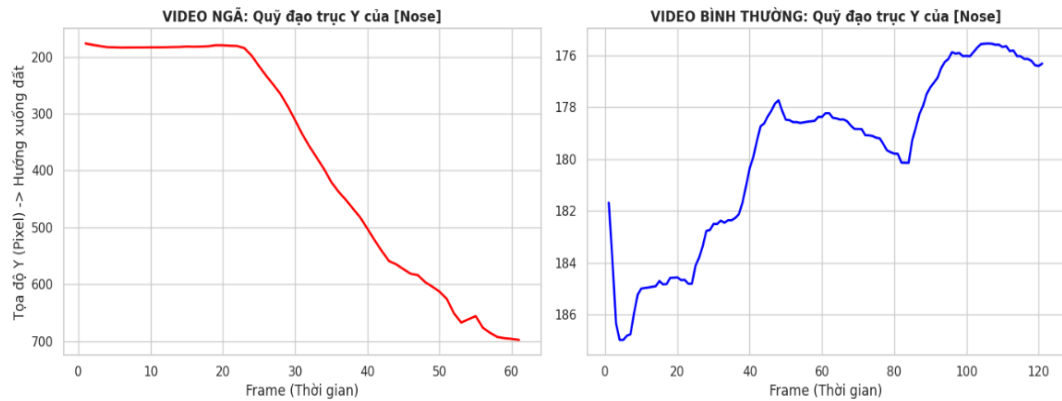
→ Nhận xét: Đa số các video có độ dài tập trung trong khoảng từ 500 đến 2000 frames. Phân phối độ dài giữa hai lớp Fall và No_Fall có sự tương đồng nhất định. Tính chất chuỗi dài (long-sequence) này khẳng định sự lựa chọn mạng BiLSTM thay vì các thuật toán Machine Learning truyền thống (như SVM hay Random Forest) là hoàn toàn hợp lý, vì BiLSTM có cơ chế cổng quên (forget gate) giúp giải quyết triệt để bài toán suy giảm gradient (vanishing gradient) khi nhớ các thông tin ở khoảng thời gian xa.

4) Các đặc trưng không gian và động học của dữ liệu

- Để mạng nơ-ron có thể phân loại chính xác, dữ liệu các điểm khớp cần thể hiện rõ sự khác biệt mang tính quy luật giữa hành vi té ngã và sinh

hoạt bình thường. Phân tích thăm dò dữ liệu (EDA) trên tập CSV chỉ ra 2 đặc trưng quan trọng nhất:

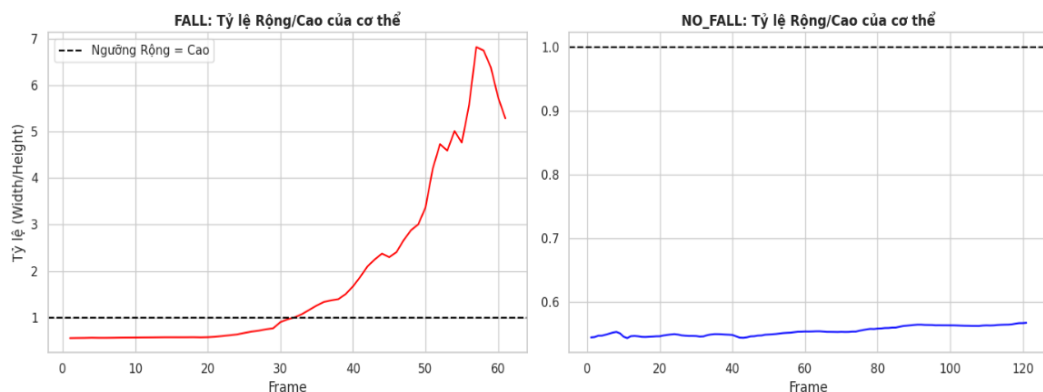
- Quỹ đạo rơi tự do theo trục Y (Y-axis Trajectory):



Hình 3.7 Quỹ đạo rơi tự do theo trục y

→ Phân tích sự dịch chuyển của điểm khớp Mũi (Nose) - đại diện cho phần đầu cơ thể: Trong chuỗi video té ngã (Màu đỏ), tọa độ Y tăng lên một cách liên tục và đột biến trong một khoảng thời gian ngắn (chú ý: trong xử lý ảnh, gốc tọa độ (0,0) nằm ở góc trên bên trái, do đó giá trị Y tăng tức là cơ thể đang tiến nhanh về phía mặt đất). Ngược lại, ở video bình thường (Màu xanh), tọa độ Y của điểm mũi duy trì sự ổn định trên trục ngang, chỉ dao động nhẹ khi con người bước đi.

- Sự biến đổi tỷ lệ Rộng/Cao của cơ thể :



Hình 3.8 Sự biến đổi rộng/cao của cơ thể

→ Khi trích xuất tạo độ bao quanh 17 điểm khớp xương, tỷ lệ giữa chiều Rộng (Width) và chiều Cao (Height) của cơ thể phản ánh rõ rệt tư thế. Biểu đồ cho thấy: Ở trạng thái bình thường (No_Fall), tư thế đứng/đi lại khiến chiều cao luôn lớn hơn chiều rộng (Tỷ lệ Rộng/Cao < 1). Tuy nhiên, khi xảy ra sự cố té ngã, cơ thể chuyển từ trạng thái thẳng đứng

sang nằm ngang trên mặt sàn, chiều rộng tăng vọt trong khi chiều cao giảm mạnh, khiến dải tỷ lệ bứt phá vượt qua ngưỡng 1.0 (đường đứt nét màu đen). Đây là đặc trưng không gian (spatial feature) mang tính quyết định để BiLSTM phân loại nhãn.

5) Chất lượng dữ liệu trích xuất (Missing Data Analysis)



Hình 3.9 Minh họa chất lượng bộ dữ liệu

→ Yếu tố cuối cùng đánh giá chất lượng bộ dữ liệu là sự nguyên vẹn của thông tin tọa độ. Biểu đồ Heatmap kiểm tra các giá trị khuyết thiếu (Missing Keypoints) cho thấy màu đen bao phủ gần như toàn bộ (đại diện cho dữ liệu có đầy đủ tọa độ X, Y hợp lệ). Dữ liệu không bị nhiễu do mất điểm khớp ở các frame quan trọng. Điều này cho thấy thuật toán trích xuất xương khớp hoạt động cực kỳ ổn định, cung cấp bộ dữ liệu thô (raw data) đủ "sạch" để thực hiện các bước Tiền xử lý (Preprocessing) và đưa vào huấn luyện mô hình phân lớp BiLSTM.

3.2.2. Tiền xử lý dữ liệu

Dữ liệu thô được trích xuất khung xương tuy đã có dạng chuỗi tọa độ, nhưng mang nhiều độ nhiễu và có độ dài không đồng nhất. Để mạng tuần hoàn BiLSTM có thể học được các mẫu không gian - thời gian (spatiotemporal patterns) một cách hiệu quả nhất, quy trình tiền xử lý dữ liệu được thiết kế thành một pipe-line tự động bao gồm 5 bước chính:

Bước 1: Phân chia tập dữ liệu

- Toàn bộ tập dữ liệu CSV được chia thành 3 tập: Huấn luyện (Train), Xác thực (Validation) và Kiểm thử với tỷ lệ 70% - 15% - 15%. Để đảm bảo tính khách quan và tránh hiện tượng mất cân bằng lớp ở các tập con, thuật toán chia dữ liệu phân tầng (Stratified Sampling) được áp

dụng. Điều này đảm bảo tỷ lệ video "Ngã" và "Không ngã" trong cả 3 tập Train, Val, Test đều giữ nguyên cấu trúc phân bố của dữ liệu gốc.

Bước 2: Lọc nhiễu và Điền khuyết dữ liệu

- Trong thực tế, khi đối tượng bị che khuất một phần (occlusion) hoặc chuyển động quá nhanh, mô hình YOLOv26-pose có thể đưa ra các điểm khớp sai lệch.
 - Lọc nhiễu: Các điểm tọa độ (X, Y) có độ tin cậy thấp (dưới ngưỡng $CONF_THRESHOLD = 0.3$) sẽ bị loại bỏ và chuyển thành giá trị rỗng (NaN). Việc này ngăn chặn mạng BiLSTM học phải các quỹ đạo chuyển động sai logic (ví dụ: chân bắt ngờ dịch chuyển 2 mét trong 1 frame).
 - Nội suy (Interpolation): Vì chuyển động của con người là liên tục theo tính chất vật lý, khoảng trống dữ liệu (NaN) được điền khuyết bằng phương pháp nội suy tuyến tính (Linear Interpolation). Phương pháp này sẽ nối một đường thẳng mượt mà giữa khung hình hợp lệ trước và sau điểm bị mất, giúp phục hồi quỹ đạo của khớp xương mà không làm gãy chuỗi thời gian.

Bước 3: Trích xuất đặc trưng động học nâng cao

- Mặc dù 34 đặc trưng gốc (tọa độ X, Y của 17 điểm khớp) đã mô tả tư thế con người, nhưng hành vi té ngã mang tính chất biến thiên theo thời gian. Do đó, quy trình tiền xử lý đã tính toán thêm các đặc trưng động học phụ trợ (Extra Features) để giúp BiLSTM nhận diện hành vi dễ dàng hơn:
 - Vận tốc (Velocity - 34 đặc trưng): Được tính bằng đạo hàm bậc 1 của tọa độ theo thời gian (sự chênh lệch vị trí giữa frame hiện tại và frame trước đó). Đặc trưng này giúp mô hình nắm bắt được gia tốc rơi đột ngột - dấu hiệu đặc trưng nhất của té ngã.
 - Độ cao trọng tâm (Hip Height - 1 đặc trưng): Tọa độ Y trung bình của khớp hông trái và phải. Sự sụt giảm nhanh chóng của biến này đại diện cho việc toàn bộ cơ thể đang hướng xuống mặt đất.
 - Góc cơ thể (Body Angle - 1 đặc trưng): Tính toán góc lệch giữa trục cổ (Neck) và trung tâm hông (Mid-hip). Khi đứng, góc này xấp xỉ 90 độ (hoặc -90 độ); khi ngã xuống sàn, góc này chuyển về xấp xỉ 0 độ (hoặc 180 độ).
 - Tổng cộng, dữ liệu đầu vào được nâng cấp từ 34 lên 70 đặc trưng (Features) cho mỗi khung hình (frame).

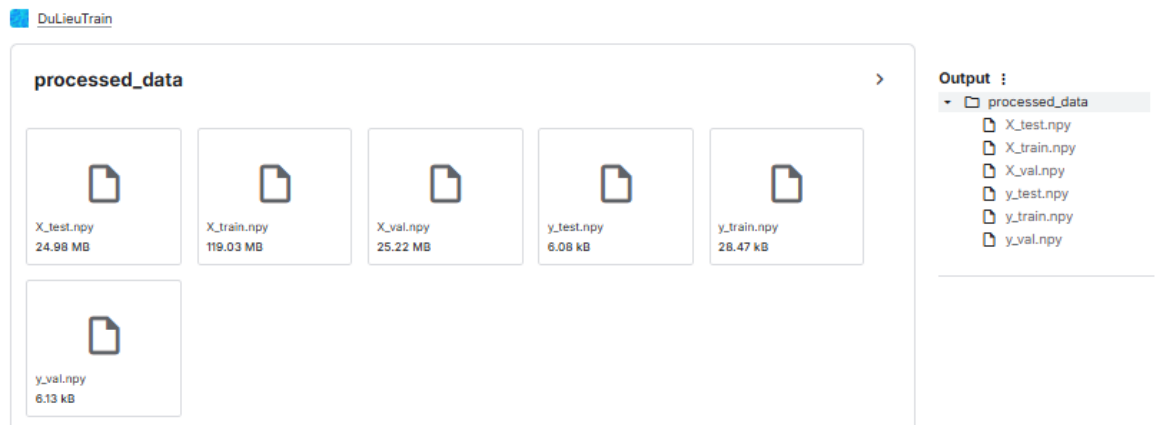
Bước 4: Chuẩn hóa dữ liệu

- Do các đặc trưng có thang đo khác nhau (tọa độ tính bằng Pixel có thể lên tới hàng trăm, trong khi góc cơ thể tính bằng Radian rất nhỏ), dữ liệu được chuẩn hóa về khoảng $[0, 1]$ sử dụng thuật toán MinMaxScaler.
- Để ngăn chặn hiện tượng rò rỉ dữ liệu (Data Leakage), bộ scaler chỉ được học (fit) các tham số Min-Max duy nhất trên tập Train. Sau đó, các trọng số này mới được áp dụng (transform) để chuẩn hóa cho tập Val và Test. Quá trình fit cũng được thực hiện theo từng phần (partial_fit) nhằm tối ưu hóa bộ nhớ RAM.

Bước 5: Tạo chuỗi bằng Cửa sổ trượt và Đệm dữ liệu

- Mạng BiLSTM yêu cầu đầu vào là một tensor 3 chiều có định dạng: [Số lượng mẫu, Số bước thời gian, Số lượng đặc trưng]. Quá trình chuyển đổi được thực hiện như sau:
 - Kỹ thuật Padding: Các video có độ dài ngắn hơn 60 frames sẽ được đệm thêm các giá trị nhân tạo (PAD_VALUE = -99.0). Khi đưa vào huấn luyện, lớp Masking của mạng Nơ-ron sẽ được cấu hình để bỏ qua con số -99.0 này, giúp mô hình không bị nhiễu bởi dữ liệu đệm.
 - Cửa sổ trượt (Sliding Window): Để nhận diện hành vi trong thời gian thực, chuỗi dữ liệu dài được cắt thành các đoạn ngắn với chiều dài cửa sổ SEQ_LENGTH = 60 (tương đương khoảng 2 giây của video gốc 30fps). Bước trượt STEP = 30 được áp dụng, tạo ra sự chồng lấp 50% (overlap) giữa các cửa sổ liên kề. Sự chồng lấp này rất quan trọng, đảm bảo rằng một khoảnh khắc té ngã diễn ra nhanh chóng sẽ không bị cắt đứt làm đôi ở giữa hai cửa sổ trượt.
- ❖ Kết quả đầu ra:
 - Kết thúc quy trình tiền xử lý, dữ liệu được ép kiểu về định dạng float32 (giúp giảm 50% tài nguyên bộ nhớ so với float64) và xuất ra 6 ma trận tensor (X_train, y_train, X_val, y_val, X_test, y_test). Tập dữ liệu lúc này đã ở trạng thái tối ưu nhất, hoàn toàn sẵn sàng để đưa vào mạng BiLSTM huấn luyện phân loại hành vi.

Output Data



Hình 3.10 Dữ liệu đầu ra sau tiền xử lý

- Đặc biệt, tập dữ liệu lúc này đã được định hình cấu trúc (Shape) chuẩn xác để tương thích hoàn toàn với lớp đầu vào (Input Layer) của mạng chuỗi thời gian BiLSTM. Cụ thể, các ma trận đặc trưng đầu vào (X) đều mang cấu trúc tensor 3 chiều với định dạng: [Số lượng mẫu, Chiều dài chuỗi, Số lượng đặc trưng] (tương đương [Samples, Time-steps, Features]). Trong bài toán này, cấu trúc đầu vào được xác định cụ thể là [N, 60, 70], mang ý nghĩa:
 - N (Samples): Là tổng số lượng các cửa sổ trượt (windows) đã được trích xuất từ các video tương ứng trong từng tập Train, Val và Test.
 - 60 (Time-steps): Tương ứng với biến SEQ_LENGTH = 60, nghĩa là mỗi mẫu dữ liệu là một chuỗi liên tục kéo dài đúng 60 khung hình (frames).
 - 70 (Features): Là số lượng đặc trưng động học đã được kỹ thuật hóa trong một khung hình (bao gồm 34 tọa độ X-Y, 34 vận tốc tương đối, 1 độ cao trọng tâm và 1 góc cơ thể).

3.2.3. Huấn luyện mô hình

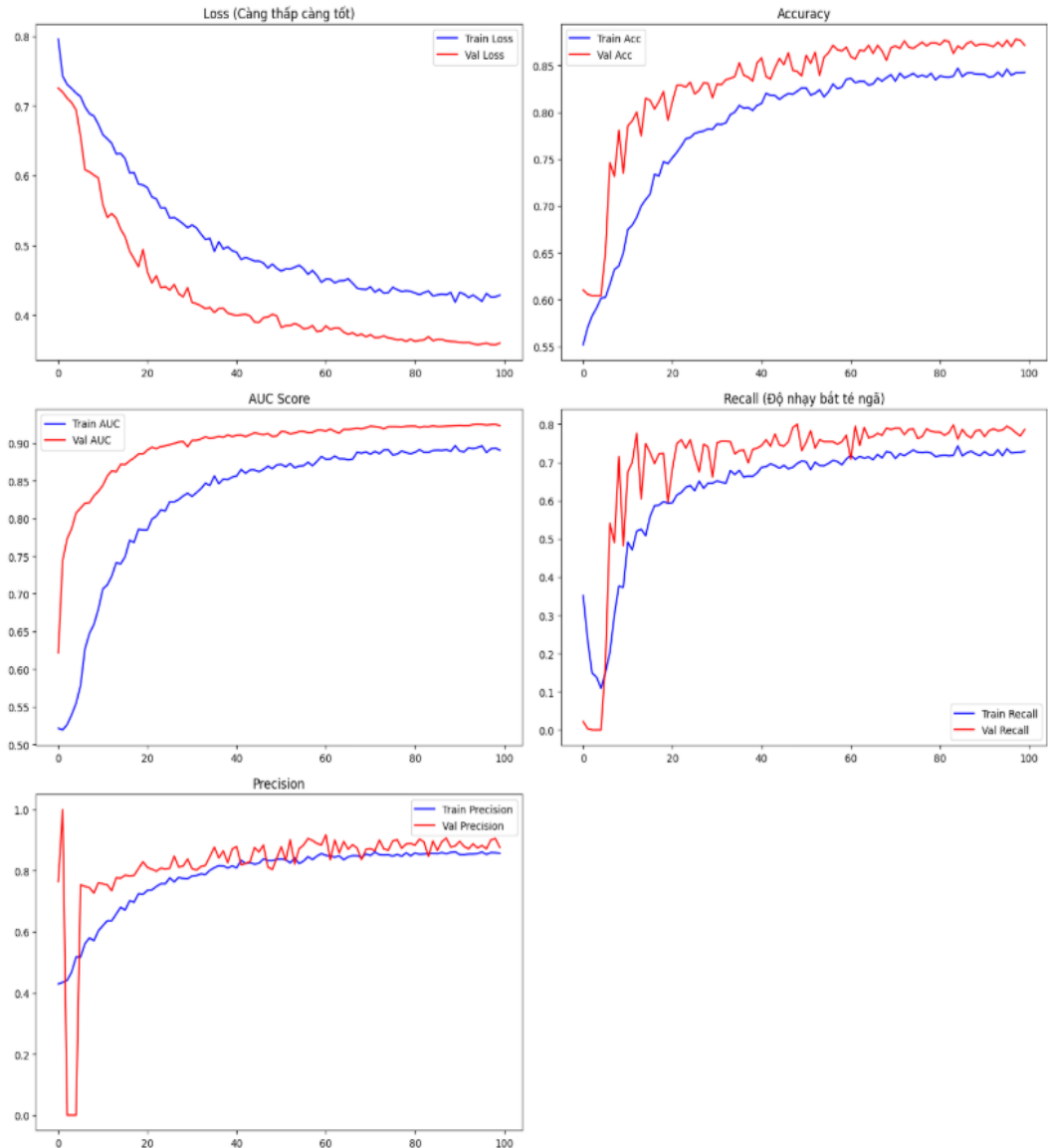
Quá trình huấn luyện mô hình mạng nơ-ron hồi quy hai chiều (BiLSTM) là giai đoạn cốt lõi để hệ thống học các quy luật động học của hành vi té ngã. Để đảm bảo mô hình đạt hiệu suất cao nhất, tính tổng quát tốt và tốc độ xử lý nhanh, quá trình này được thiết lập thông qua các bước tối ưu hóa phần cứng, cấu trúc mạng và chiến lược siêu tham số.

a) Tối ưu hóa phần cứng và Luồng dữ liệu

- Do đặc thù của dữ liệu chuỗi thời gian (có số lượng chiều lớn: 60 time-steps x 70 features), quá trình huấn luyện đòi hỏi tài nguyên tính toán lớn.

- **Mixed Precision (Độ chính xác hỗn hợp):** Hệ thống được cấu hình sử dụng chính sách `mixed_float16`. Kỹ thuật này cho phép GPU NVIDIA thực hiện các phép toán ma trận ở định dạng 16-bit (giúp tăng tốc độ tính toán lên gấp 2 lần và giảm một nửa VRAM), trong khi các trọng số mô hình vẫn được lưu ở định dạng 32-bit (`float32`) để đảm bảo không bị sai số (numeric stability).
 - **TensorFlow Data Pipeline (tf.data):** Dữ liệu không được nạp toàn bộ vào RAM một cách truyền thống mà được luân chuyển qua một đường ống dẫn (pipeline). Các cơ chế `shuffle()` (trộn ngẫu nhiên), `batch(256)` (đẩy từng cụm 256 mẫu vào GPU) và đặc biệt là `prefetch(AUTOTUNE)` được kích hoạt. Cơ chế này cho phép CPU chuẩn bị sẵn cụm dữ liệu tiếp theo trong khi GPU đang huấn luyện cụm hiện tại, giải quyết triệt để tình trạng "nghẽn cổ chai" (bottleneck) luồng I/O.
- b) Cấu trúc kiến trúc mạng BiLSTM và Kỹ thuật điều chuẩn (Regularization)
- Dữ liệu hành vi con người rất phức tạp và dễ làm mạng nơ-ron học vẹt (Overfitting). Do đó, cấu trúc mạng được thiết kế đặc biệt với nhiều lớp điều chuẩn bảo vệ:
 - **Lớp Masking:** Nhận diện và bỏ qua các giá trị đệm (`Padding = -99.0`) đã được thêm vào từ pha tiền xử lý, giúp mạng không tính toán sai lệch trên các khung hình ảo.
 - **Khối BiLSTM thứ nhất (64 units):** Phân tích chuỗi từ 2 hướng (Quá khứ $\rightarrow$ Tương lai và Tương lai $\rightarrow$ Quá khứ). Để chống quá khớp, khối này áp dụng đồng thời: `dropout=0.3` (tắt ngẫu nhiên các kết nối đầu vào) và `recurrent_dropout=0.2` (tắt ngẫu nhiên các kết nối trạng thái ẩn giữa các bước thời gian). Đặc biệt, hàm phạt L2 Regularization ($1e-4$) được thêm vào để kìm hãm sự phình to của các trọng số.
 - **Lớp LayerNormalization:** Khác với chuẩn hóa theo lô (Batch Normalization) thường dùng trong CNN, Layer Normalization được chứng minh là tương thích và giúp ổn định gradient tốt hơn rất nhiều đối với các mạng chuỗi thời gian như LSTM.
 - **Khối BiLSTM thứ hai (32 units) và Lớp Dense (16 units):** Tiếp tục chắt lọc các đặc trưng trừu tượng mức cao, xen kẽ với các lớp Dropout (0.3 và 0.4) để tăng tính mạnh mẽ (robustness) cho mạng.

- Lớp đầu ra (Output Layer): Sử dụng 1 nơ-ron với hàm kích hoạt Sigmoid, xuất ra xác suất từ 0.0 (Bình thường) đến 1.0 (Té ngã).
- c) Thiết lập Siêu tham số (Hyperparameters) và Chiến lược Callbacks
- Mô hình sử dụng hàm mất mát Binary Cross-Entropy (chuyên dụng cho bài toán phân loại nhị phân) và bộ tối ưu hóa Adam. Tốc độ học ban đầu (Learning Rate) được thiết lập ở mức khá nhỏ 0.0005 thay vì mặc định 0.001, giúp mô hình "đi chậm mà chắc", tránh việc bước nhảy gradient quá lớn làm trượt qua điểm cực tiểu toàn cục. Hệ thống sử dụng 3 bộ giám sát tự động (Callbacks) trong suốt 100 kỷ nguyên (Epochs):
 - EarlyStopping: Tự động dừng huấn luyện nếu hàm mất mát trên tập Val (val_loss) không giảm sau 15 epochs, đồng thời khôi phục lại bộ trọng số tốt nhất (restore_best_weights).
 - ModelCheckpoint: Liên tục theo dõi và lưu lại phiên bản mô hình (file .h5 / .keras) có val_loss thấp nhất.
 - ReduceLROnPlateau: Tự động giảm một nửa (factor=0.5) tốc độ học nếu mô hình có dấu hiệu bị kẹt ở các điểm cực tiểu địa phương (plateau) trong vòng 5 epochs.
- d) Phân tích kết quả huấn luyện
- Sau khi kết thúc quá trình đốt GPU (Training), sự tiến hóa của mô hình được biểu diễn qua 5 đồ thị đo lường (Metrics) như trong Hình dưới đây.



Hình 3.11 Kết quả huấn luyện

- Dựa trên biểu đồ huấn luyện, ta rút ra các nhận xét kỹ thuật quan trọng sau:
 - Đồ thị Loss (Hàm mất mát): Đường Train Loss (xanh) và Val Loss (đỏ) cùng giảm dần một cách mượt mà và tiệm cận nhau. Sự xuất hiện của các lớp Dropout và L2 đã phát huy tác dụng tuyệt vời: không hề xảy ra hiện tượng đường Val Loss bị hất ngược lên ở các epoch cuối. Điều này khẳng định mô hình hoàn toàn không bị Overfitting và có khả năng tổng quát hóa dữ liệu mới cực kỳ tốt.
 - Đồ thị Accuracy (Độ chính xác) và AUC: Độ chính xác trên tập validation tiệm cận mức 85% - 88%. Đặc biệt, diện tích dưới đường cong ROC (AUC Score) của tập Val nhanh chóng vượt mức 0.90 ngay từ epoch thứ 20. AUC cao chứng tỏ mô hình có năng

lực phân tách (discriminative power) rất rõ ràng giữa hai lớp "Ngã" và "Không ngã".

- Đồ thị Recall và Precision: Đối với hệ thống y tế/cảnh báo an toàn, chỉ số Recall (Độ nhạy) quan trọng hơn Accuracy vì ta thà báo động nhầm còn hơn bỏ sót người bị ngã. Đồ thị cho thấy Val Recall (Đường màu đỏ) duy trì ở mức cao và ổn định (khoảng 0.8), cùng với Precision đạt gần 0.9. Sự cân bằng này cho thấy mô hình không chỉ nhạy bén trong việc bắt các hành vi té ngã, mà còn kiểm soát rất tốt hiện tượng báo động giả (False Alarm) gây ra bởi các hành vi sinh hoạt thường ngày.

Kết luận: Quá trình huấn luyện đã diễn ra thành công. Bộ trọng số tối ưu nhất đã được lưu lại để phục vụ cho quá trình đánh giá trên tập kiểm thử và triển khai trên hệ thống theo thời gian thực.

3.2.4. Đánh giá mô hình

Để kiểm chứng khả năng tổng quát hóa và tính ứng dụng thực tế, mô hình BiLSTM sau khi huấn luyện được đánh giá trên tập dữ liệu kiểm thử (Test set) hoàn toàn độc lập. Tập dữ liệu này bao gồm 1487 mẫu chuỗi thời gian, với sự phân bố dữ liệu không cân bằng nhẹ: 905 mẫu thuộc lớp "Bình thường" (nhãn 0) và 582 mẫu thuộc lớp "Té ngã" (nhãn 1).

Trong nghiên cứu này, một tinh chỉnh quan trọng đã được thực hiện là hạ ngưỡng quyết định phân loại (classification threshold) từ mức mặc định 0.5 xuống 0.4. Đối với hệ thống cảnh báo y tế khẩn cấp, việc tối thiểu hóa tỷ lệ bỏ sót ca té ngã (False Negative) được ưu tiên cao hơn việc giảm thiểu cảnh báo giả (False Positive).

Hiệu năng của mô hình được đánh giá toàn diện qua ba khía cạnh: Các chỉ số thống kê, Ma trận nhầm lẫn (Confusion Matrix) và Đặc tuyến hoạt động (ROC Curve).

a) Đánh giá qua các chỉ số phân loại (Classification Metrics)

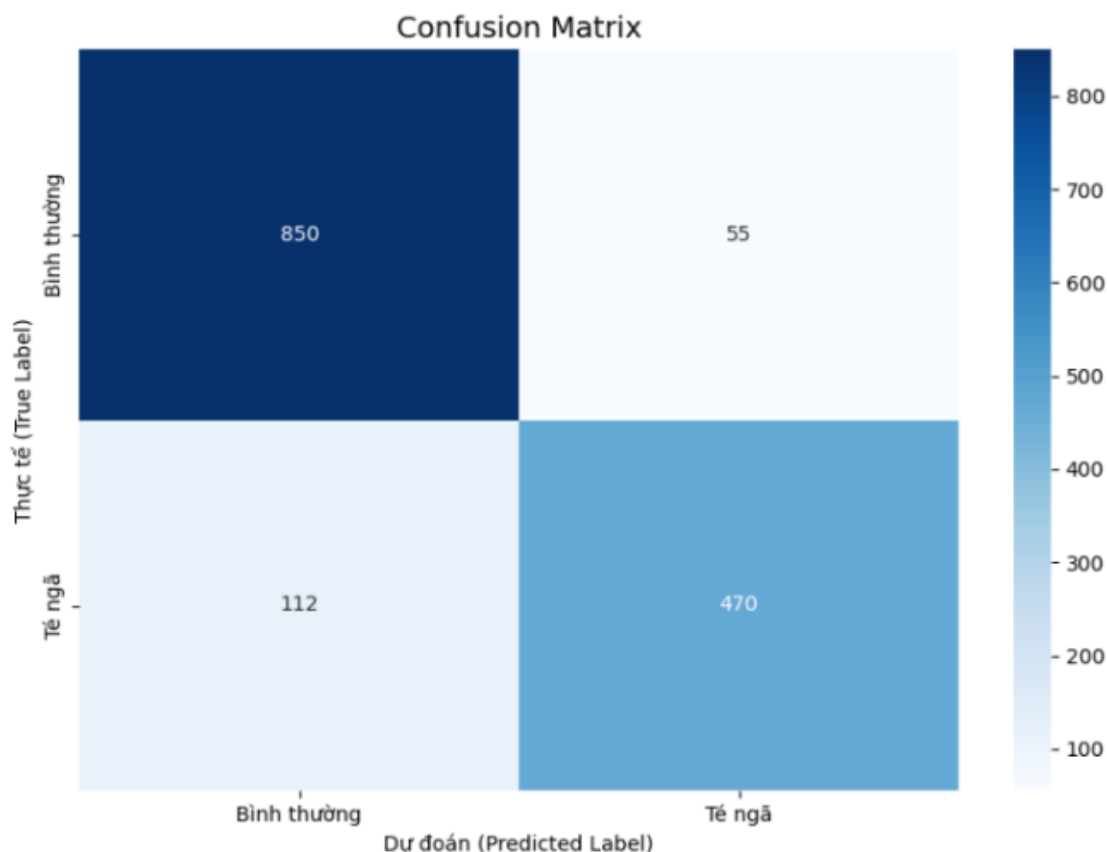
BÁO CÁO PHÂN LOẠI (CLASSIFICATION REPORT):

	precision	recall	f1-score	support
Bình thường (0)	0.88	0.94	0.91	905
Té ngã (1)	0.90	0.81	0.85	582
accuracy			0.89	1487
macro avg	0.89	0.87	0.88	1487
weighted avg	0.89	0.89	0.89	1487

Hình 3.12 Báo cáo phân loại đánh giá mô hình

- Dựa trên báo cáo phân loại, mô hình đạt độ chính xác tổng thể (Accuracy) là 89%. Tuy nhiên, do dữ liệu có sự chênh lệch giữa hai lớp, các chỉ số F1-score, Precision (Độ chụm) và Recall (Độ thu hồi) được xem xét kỹ lưỡng hơn:
 - Đối với hành vi "Bình thường" (Lớp 0): Mô hình đạt độ chụm 0.88 và độ thu hồi 0.94 (F1-score: 0.91). Kết quả này cho thấy khả năng nhận diện các hoạt động sinh hoạt hàng ngày (ADL) của hệ thống là rất ổn định.
 - Đối với hành vi "Té ngã" (Lớp 1 - Lớp mục tiêu): Khả năng dự đoán đạt độ chụm 0.90 và độ thu hồi 0.81 (F1-score: 0.85). Chỉ số Precision đạt 0.90 mang ý nghĩa thực tiễn cao: khi hệ thống phát tín hiệu cảnh báo té ngã, có xác suất 90% đó là một sự cố thực sự, giúp hạn chế sự phiền toái do báo động nhầm.
- Các chỉ số trung bình vĩ mô (Macro Avg) và trung bình có trọng số (Weighted Avg) đều ổn định ở mức 0.89, minh chứng cho việc mô hình không bị thiên lệch (overfitting) hoàn toàn về lớp có số lượng mẫu lớn hơn.

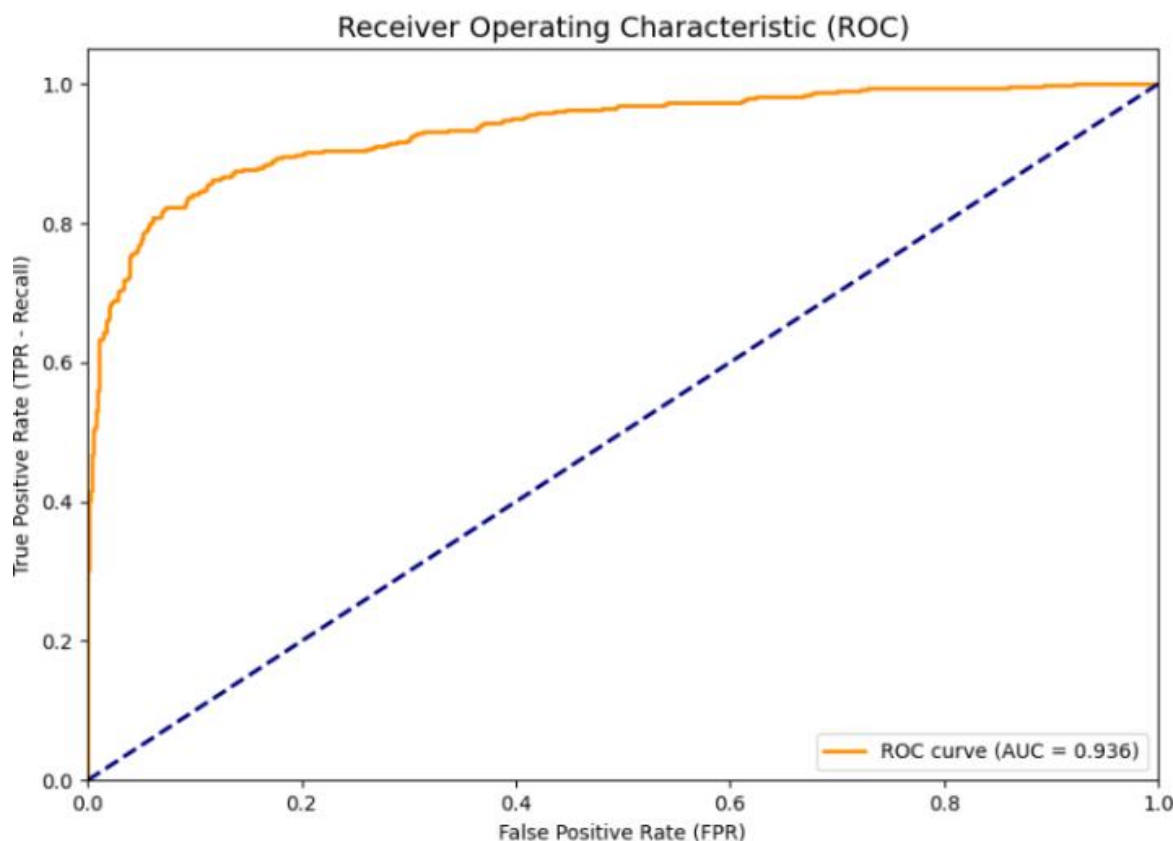
b) Phân tích Ma trận nhầm lẫn (Confusion Matrix)



Hình 3.13 Ma trận nhầm lẫn

- Ma trận nhầm lẫn cung cấp cái nhìn chi tiết về sự phân bố các dự đoán đúng và sai trên 1487 mẫu kiểm thử.
 - Nhận diện đúng (True Positive & True Negative): Mô hình phân loại chính xác 850 mẫu bình thường (TN) và 470 mẫu té ngã (TP).
 - Dương tính giả (False Positive - FP): Có 55 trường hợp người dùng sinh hoạt bình thường nhưng hệ thống đưa ra cảnh báo ngã. Với tỷ lệ FP tương đối thấp (khoảng 6% trên tổng số mẫu lớp 0), hệ thống cho thấy khả năng chống nhiễu tốt trước các hành động đột ngột dễ gây nhầm lẫn (ví dụ: ngồi phịch xuống ghế, cúi nhặt đồ).
 - Âm tính giả (False Negative - FN): Hệ thống bỏ sót 112 trường hợp ngã thực sự (dự đoán thành bình thường). Mặc dù đã hạ ngưỡng phân loại, việc tỷ lệ FN chiếm khoảng 19% (112/582) trên tổng số mẫu ngã cho thấy mô hình vẫn gặp khó khăn với các tư thế ngã phức tạp, ngã chậm hoặc tín hiệu cảm biến thu được bị mờ nhạt. Đây là một điểm hạn chế (limitation) cần được khắc phục trong các phiên bản tối ưu tiếp theo.

c) Đặc tuyến hoạt động ROC và Chỉ số AUC



Hình 3.14 Đường cong ROC

- Đường cong ROC biểu diễn mối quan hệ đánh đổi giữa tỷ lệ dương tính thật (TPR - Recall) và tỷ lệ dương tính giả (FPR) ở nhiều ngưỡng xác suất khác nhau.
- Kết quả cho thấy đường cong ROC tiệm cận sát về phía góc trên cùng bên trái của đồ thị. Chỉ số diện tích dưới đường cong (AUC) đạt giá trị 0.936. Về mặt học thuật, một mô hình có AUC > 0.9 được xếp loại có khả năng phân tách xuất sắc (excellent discrimination). Điều này khẳng định thuật toán BiLSTM đã trích xuất và học được các đặc trưng không gian - thời gian (spatio-temporal features) khác biệt rõ rệt giữa hành vi duy trì thăng bằng và sự cố mất thăng bằng đột ngột.
- ❖ **Kết luận:** Tổng hợp các hệ quả phân tích, mô hình BiLSTM đề xuất đã giải quyết tốt bài toán phân loại hành vi với độ tin cậy cao (Accuracy 89%, AUC 0.936). Mạng nơ-ron thể hiện ưu thế trong việc giới hạn các cảnh báo sai lệch (Precision cao). Mặc dù vẫn tồn tại một tỷ lệ nhất định các ca ngã bị bỏ lọt (False Negative), hiệu năng tổng thể của mô hình đã chứng minh được tính khả thi để tích hợp vào các hệ thống thiết bị đeo tay theo dõi sức khỏe người cao tuổi trong thực tế.

CHƯƠNG 4. XÂY DỰNG HỆ THỐNG

4.1. Giới thiệu chung về sản phẩm

Sản phẩm đầu ra của đề tài là một hệ thống ứng dụng Web hoàn chỉnh với tên gọi "AI Fall Detection System". Hệ thống được thiết kế để tự động giám sát, phát hiện và đưa ra cảnh báo kịp thời khi có sự cố té ngã xảy ra, đặc biệt hướng tới đối tượng là người cao tuổi hoặc bệnh nhân cần theo dõi.

Sản phẩm là sự kết hợp chặt chẽ giữa công nghệ thị giác máy tính tiên tiến (YOLOV26-Pose) để trích xuất khung xương người (Skeleton extraction) và mạng nơ-ron hồi quy hai chiều (BiLSTM) để phân tích chuỗi dữ liệu thời gian (Time-series analysis) nhằm phân loại hành vi té ngã.

Để đảm bảo hiệu năng cao và độ trễ thấp trong quá trình truyền phát video, hệ thống được xây dựng theo kiến trúc Client - Server, giao tiếp thông qua giao thức WebSocket kết hợp với nền tảng FastAPI.

4.1.1. Phân hệ Giám sát trực tuyến

Đây là chức năng cốt lõi nhất của hệ thống, cho phép kết nối trực tiếp với Camera (Webcam/CCTV) để phân tích hành vi theo thời gian thực. Trích xuất và trực quan hóa khung xương: Hệ thống liên tục nhận diện 17 điểm neo (keypoints) trên cơ thể người bằng YOLOV26-Pose và vẽ nối thành khung xương trực tiếp lên màn hình video. Các điểm bị khuất sẽ được hệ thống tự động bù đắp dữ liệu dựa trên logic lưu trữ frame trước đó, giúp tránh hiện tượng "mạng nhện" hoặc đứt gãy tín hiệu.

Tính toán xác suất (Probability): Điểm số tin cậy (Confidence score) về hành vi ngã được tính toán liên tục qua từng khung hình bằng mô hình BiLSTM và được làm mượt (smoothing) để tránh nhiễu do sai số của cảm biến.

Chế độ quyền riêng tư (Privacy Mode): Đây là một tính năng mang tính đạo đức ứng dụng cao. Khi được kích hoạt, hệ thống sẽ tự động làm mờ (Gaussian Blur) hình ảnh gốc của người dùng nhưng vẫn giữ nguyên đường nét khung xương để AI phân tích. Điều này đặc biệt hữu ích khi lắp đặt hệ thống tại các khu vực nhạy cảm như phòng ngủ, phòng tắm.

4.1.2. Phân hệ Phân tích Video ngoại tuyến

Chức năng này cho phép người dùng tải lên các tệp video có sẵn (.mp4, .avi, .mkv) để hệ thống tiến hành kiểm tra lại (review). Video sau khi tải lên sẽ được bóc tách từng khung hình để chạy qua luồng suy luận của mô hình AI.

Tiến trình phân tích được hiển thị trực tiếp (Progress bar) thông qua WebSocket.

Hệ thống sẽ tổng hợp các thông số thống kê bao gồm: Tổng số lần phát hiện ngã trong video và Mức độ tin cậy cao nhất (Max Peak Probability). Đặc biệt, hệ thống hỗ trợ kết xuất (Export) và tải xuống một đoạn video mới, trong đó đã được gắn nhãn (bounding box), vẽ khung xương và hiển thị thông số dự đoán trên từng khung hình để làm tư liệu đánh giá.

4.1.3. Phân hệ Lịch sử và Cơ chế cảnh báo đa kênh

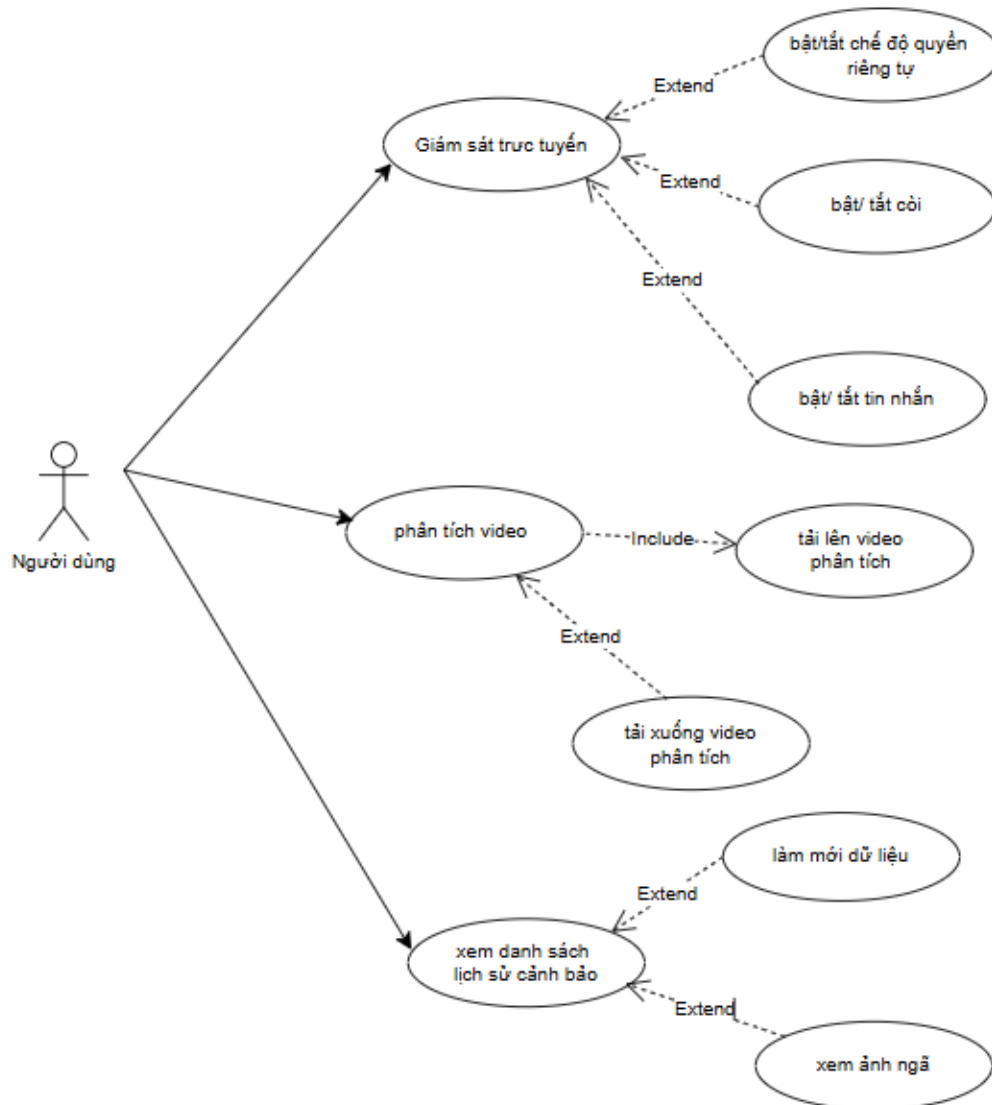
Để đảm bảo người giám hộ nhận được thông tin sự cố ngay lập tức, hệ thống được trang bị cơ chế cảnh báo đa lớp:

- Cảnh báo tại chỗ (Local Alarm): Kích hoạt còi hú (Siren) ngay trên giao diện web và đổi màu cảnh báo (sang Đỏ) khi phát hiện sự cố mất thăng bằng nghiêm trọng.
- Cảnh báo từ xa qua Telegram: Ngay tại thời điểm sự cố được xác nhận, hệ thống Backend sẽ tự động cắt khung hình (chụp ảnh màn hình) và đẩy cảnh báo trực tiếp về điện thoại người giám hộ thông qua Telegram Bot API, kèm theo thời gian thực và mức độ tin cậy của AI.

Quản lý lịch sử (History Management): Mọi sự kiện cảnh báo đều được lưu trữ vĩnh viễn trong cơ sở dữ liệu SQLite (bao gồm ID, thời gian, xác suất và hình ảnh lưu trữ tại local). Quản trị viên có thể dễ dàng truy xuất, xem lại danh sách các ca té ngã và kiểm tra hình ảnh chứng minh của từng sự kiện thông qua giao diện dạng bảng (Table).

4.2. Phân tích thiết kế phần mềm demo

4.2.1. Biểu đồ Use Case tổng quan



Hình 4.1 Usecase tổng quan

4.2.2. Đặc tả Use Case

- Use Case 1

Tên Use Case: Giám sát trực tuyến
- Mô tả vắn tắt: Cho phép người dùng bật camera trực tiếp từ thiết bị. Hệ thống sẽ thu nhận hình ảnh thời gian thực, tiến hành phân tích hành vi và đưa ra cảnh báo ngay lập tức nếu phát hiện có người bị ngã. - Luồng các sự kiện: - Luồng cơ bản: - Người dùng chọn chức năng "Giám sát trực tuyến" trên giao diện. - Người dùng nhấn nút yêu cầu bắt đầu camera. - Hệ thống yêu cầu quyền truy cập camera của thiết bị. Người dùng đồng ý. - Hệ thống bắt đầu hiển thị luồng video trực tiếp trên màn hình. - Hệ thống liên tục nhận diện và theo dõi chuyển động của đối tượng trong khung hình. - Hệ thống hiển thị trạng thái "An toàn" và mức độ nguy hiểm trên màn hình. - Người dùng nhấn nút dừng khi muốn kết thúc giám sát. Use case kết thúc. - Các luồng rẽ nhánh: - Từ chối camera: Tại bước 3, nếu người dùng từ chối cấp quyền, hệ thống thông báo không thể hoạt động và hủy bỏ phiên giám sát. Use case kết thúc.

▪ Phát hiện ngã: Tại bước 6, nếu hệ thống nhận diện hành vi ngã, trạng thái màn hình lập tức chuyển sang "Phát hiện ngã" với cảnh báo đỏ. Kích hoạt các tính năng mở rộng nếu có.
3. Các yêu cầu đặc biệt: Hệ thống cần đảm bảo độ trễ hình ảnh ở mức thấp nhất để cảnh báo kịp thời.
4. Tiền điều kiện: Người dùng đã truy cập hệ thống và thiết bị có trang bị camera hoạt động bình thường.
5. Hậu điều kiện: Ghi nhận lại sự kiện cảnh báo vào hệ thống lưu trữ (nếu có sự cố ngã xảy ra).
6. Điểm mở rộng: Bật/tắt chế độ quyền riêng tư, Bật/ tắt còi, Bật/ tắt tin nhắn.

- Use Case 2

Tên Use Case: Bật/tắt chế độ quyền riêng tư
1. Mô tả vấn đề: Người dùng tùy chọn làm mờ hình ảnh hiển thị trên màn hình giám sát để bảo vệ hình ảnh cá nhân của người được theo dõi. 2. Luồng các sự kiện: ○ Luồng cơ bản: ▪ Người dùng tích chọn tính năng "Chế độ riêng tư". ▪ Hệ thống lập tức xử lý làm mờ toàn bộ cảnh quang và khuôn mặt trên video trực tiếp.

▪ Hệ thống vẫn giữ nguyên các nét vẽ mô phỏng dáng người (khung xương) để người quản lý theo dõi được hành động. ▪ Khi người dùng bỏ tích, hình ảnh trở lại rõ nét bình thường. 3. Các yêu cầu đặc biệt: Việc làm mờ không được làm ảnh hưởng đến độ chính xác của quá trình nhận diện ngã. 4. Tiền điều kiện: Chức năng Giám sát trực tuyến đang hoạt động. 5. Hậu điều kiện: Hình ảnh hiển thị bị thay đổi trạng thái làm mờ/rõ. 6. Điểm mở rộng: Không có.
--

- Use Case 3

Tên Use Case: Bật/ tắt còi
1. Mô tả vắn tắt: Tùy chọn phát âm thanh báo động lớn tại màn hình thiết bị giám sát nhằm thu hút sự chú ý khi có người ngã. 2. Luồng các sự kiện: ○ Luồng cơ bản: ▪ Người dùng tích chọn tính năng "Còi hú tại chỗ". ▪ Khi Use Case "Giám sát trực tuyến" đi vào luồng rẽ nhánh phát hiện ngã, hệ thống tự động phát tệp âm thanh báo động. ▪ Âm thanh kéo dài cho đến khi hệ thống xác nhận đối tượng đã đứng lên hoặc người dùng chủ động tắt.

3. Các yêu cầu đặc biệt: Thiết bị của người dùng không bị tắt âm thanh (Mute).
4. Tiền điều kiện: Tính năng còi đang được bật và hệ thống phát hiện có sự kiện ngã.
5. Hậu điều kiện: Âm thanh báo động được phát ra.
6. Điểm mở rộng: Không có.

- Use Case 4

Tên Use Case: Bật/ tắt tin nhắn

1. Mô tả vắn tắt: Tùy chọn tự động gửi tin nhắn văn bản kèm hình ảnh cảnh báo đến ứng dụng nhắn tin của người quản lý từ xa.
2. Luồng các sự kiện:
 - Luồng cơ bản:
 - Người dùng tích chọn tính năng "Gửi tin Telegram" (hoặc tin nhắn).
 - Khi sự cố ngã xảy ra, hệ thống tự động chụp lại khoảnh khắc hiện tại.
 - Hệ thống soạn nội dung gồm thời gian, độ tin cậy và hình ảnh.
 - Hệ thống chuyển tiếp thông tin này đến tài khoản ứng dụng nhắn tin của người giám sát đã được thiết lập sẵn.
3. Các yêu cầu đặc biệt: Hệ thống phải có kết nối mạng internet ổn định để gửi tin đi.

4. Tiền điều kiện: Tính năng gửi tin được bật và hệ thống phát hiện ngã.
5. Hậu điều kiện: Người giám sát nhận được thông báo trên điện thoại cá nhân.
6. Điểm mở rộng: Không có.

- Use Case 5

Tên Use Case: Phân tích video

1. Mô tả vắn tắt: Hệ thống tiếp nhận một tệp video có sẵn, tự động chạy kiểm tra toàn bộ nội dung để đếm số lần ngã và đánh dấu vào khung hình.
2. Luồng các sự kiện:
 - Luồng cơ bản:
 - Người dùng chọn chức năng "Phân tích video".
 - *(Include)* Hệ thống yêu cầu người dùng thực hiện Tải lên video phân tích.
 - Hệ thống bắt đầu quét qua từng khung hình của video.
 - Giao diện hiển thị thanh tiến trình xử lý và hình ảnh đang được phân tích.
 - Hệ thống liên tục cập nhật bộ đếm "Số lần ngã" và mức độ cảnh báo lớn nhất.
 - Khi tiến trình đạt 100%, hệ thống thông báo hoàn thành. Use case kết thúc.
 - Các luồng rẽ nhánh:

▪ Chưa có video: Nếu người dùng yêu cầu phân tích nhưng chưa cung cấp tệp, hệ thống hiện cảnh báo nhắc nhở và ngưng thực thi.
3. Các yêu cầu đặc biệt: Hệ thống phải duy trì phản hồi để người dùng biết quá trình không bị đứng.
4. Tiền điều kiện: Người dùng có tệp video hợp lệ trên máy tính.
5. Hậu điều kiện: Tổng hợp kết quả thống kê số lần ngã và tạo ra một tệp video kết quả lưu tạm trên hệ thống.
6. Điểm mở rộng: Tải xuống video phân tích.

- Use Case 6

Tên Use Case: Tải lên video phân tích
1. Mô tả vắn tắt: Cho phép người dùng chuyển một tệp video từ thiết bị cá nhân lên hệ thống máy chủ để chuẩn bị xử lý. 2. Luồng các sự kiện: ○ Luồng cơ bản: ▪ Người dùng nhấn vào nút chọn tệp. ▪ Hệ điều hành hiển thị cửa sổ duyệt file. Người dùng chọn một tệp video. ▪ Hệ thống hiển thị tên tệp đã chọn. ▪ Hệ thống tải tệp đó lên khu vực lưu trữ an toàn.

3. Các yêu cầu đặc biệt: Chỉ cho phép chọn các định dạng video được quy định (mp4, avi...).
4. Tiền điều kiện: Đang ở giao diện Phân tích video.
5. Hậu điều kiện: Tập video được tải thành công lên hệ thống.
6. Điểm mở rộng: Không có.

- Use Case 7

Tên Use Case: Tải xuống video phân tích

1. Mô tả vắn tắt: Cho phép người dùng lưu lại tập video đã được hệ thống vẽ thêm các khung đánh dấu cảnh báo để làm tài liệu đối chứng.
2. Luồng các sự kiện:
 - Luồng cơ bản:
 - Sau khi quá trình phân tích hoàn tất, hệ thống hiển thị nút "Tải video kết quả".
 - Người dùng nhấn vào nút này.
 - Hệ thống chuyển giao tập video kết quả về trình duyệt của người dùng.
 - Video được lưu trữ thành công vào ổ cứng của người dùng.
3. Các yêu cầu đặc biệt: Video tải về phải giữ nguyên tỷ lệ khung hình và chứa rõ các ghi chú cảnh báo.
4. Tiền điều kiện: Quá trình "Phân tích video" phải hoàn tất 100%.

5. Hậu điều kiện: Tập video xuất hiện trong thư mục tải xuống của thiết bị người dùng.
6. Điểm mở rộng: Không có.

- Use Case 8

Tên Use Case: Xem danh sách lịch sử cảnh báo

1. Mô tả vắn tắt: Liệt kê toàn bộ các sự kiện ngã đã được hệ thống tự động ghi nhận trong quá trình giám sát, giúp người dùng tra cứu lại thông tin.
2. Luồng các sự kiện:
 - Luồng cơ bản:
 - Người dùng chọn chức năng "Lịch sử cảnh báo".
 - Hệ thống tìm kiếm các bản ghi lưu trữ về sự kiện ngã.
 - Hệ thống trình bày danh sách dưới dạng bảng, bao gồm các thông tin: Mã sự kiện, Thời gian xảy ra, và Mức độ nguy hiểm.
 - Use case kết thúc.
 - Các luồng rẽ nhánh:
 - Không có sự kiện nào: Tại bước 3, nếu hệ thống chưa ghi nhận bất kỳ sự cố nào, hệ thống hiển thị thông báo "Chưa có dữ liệu cảnh báo".
3. Các yêu cầu đặc biệt: Danh sách phải được sắp xếp theo thời gian mới nhất nằm ở trên cùng.

4. Tiền điều kiện: Không có.
5. Hậu điều kiện: Bảng danh sách được hiển thị rõ ràng trên màn hình.
6. Điểm mở rộng: Làm mới dữ liệu, Xem ảnh ngã.

- Use Case 9

Tên Use Case: Làm mới dữ liệu

1. Mô tả vắn tắt: Cho phép người dùng tải lại bảng danh sách để cập nhật các cảnh báo mới nhất vừa mới xảy ra tức thì.
2. Luồng các sự kiện:
 - Luồng cơ bản:
 - Người dùng nhấn nút "Làm mới dữ liệu".
 - Hệ thống kiểm tra lại kho lưu trữ và lấy các bản ghi mới nhất.
 - Hệ thống xóa nội dung bảng hiện tại và điền dữ liệu mới vào bảng.
3. Các yêu cầu đặc biệt: Quá trình tải lại dữ liệu diễn ra mượt mà không cần phải tải lại toàn bộ trang web.
4. Tiền điều kiện: Người dùng đang ở giao diện danh sách lịch sử.
5. Hậu điều kiện: Danh sách hiển thị các thông tin cập nhật mới nhất.
6. Điểm mở rộng: Không có.

- Use Case 10

Tên Use Case: Xem ảnh ngã

1. Tên Use Case: Xem ảnh ngã
2. Mô tả vắn tắt: Cung cấp hình ảnh chi tiết cắt từ camera tại đúng thời điểm đối tượng bị ngã để người dùng kiểm chứng tính chính xác của cảnh báo.
3. Luồng các sự kiện:
 - Luồng cơ bản:
 - Tại một bản ghi bất kỳ trong danh sách, người dùng nhấn nút "Xem ảnh".
 - Hệ thống tìm kiếm tệp hình ảnh tương ứng với thời điểm của bản ghi đó.
 - Hệ thống hiển thị một khung nổi lên giữa màn hình, phóng to hình ảnh cảnh báo.
 - Người dùng quan sát xong, nhấn chuột ra bên ngoài hoặc nút đóng để tắt hình ảnh đi.
4. Các yêu cầu đặc biệt: Hình ảnh hiển thị phải rõ ràng và chứa các mốc đánh dấu của hệ thống.
5. Tiên điều kiện: Danh sách lịch sử đang có dữ liệu và hình ảnh không bị xóa khỏi hệ thống.
6. Hậu điều kiện: Giao diện trở về danh sách bình thường sau khi đóng ảnh.
7. Điểm mở rộng: Không có.

4.3. Công cụ và công nghệ sử dụng

Hệ thống cảnh báo ngã được xây dựng dựa trên kiến trúc client-server, kết hợp giữa các framework phát triển web hiện đại và các thư viện Trí tuệ nhân tạo (AI) tiên tiến. Chi tiết các công cụ và công nghệ được áp dụng như sau:

4.3.1. Ngôn ngữ lập trình

- Python (Backend & AI): Là ngôn ngữ lập trình chính được sử dụng để xây dựng máy chủ, xử lý logic hệ thống, tích hợp các mô hình học sâu (Deep Learning) và thao tác với cơ sở dữ liệu.
- JavaScript (ES6), HTML5 & CSS3 (Frontend): Sử dụng để xây dựng giao diện người dùng (UI) trực quan. HTML5 Canvas và Video tag được dùng để trích xuất luồng ảnh từ Webcam, trong khi JavaScript đảm nhiệm việc giao tiếp thời gian thực với máy chủ mà không cần tải lại trang.

4.3.2. Trí tuệ nhân tạo và xử lý ảnh

- Ultralytics YOLO (YOLOv26-pose): Mô hình học sâu (phiên bản yolo11n-pose.pt) được sử dụng ở giai đoạn trích xuất đặc trưng không gian. YOLO làm nhiệm vụ phát hiện người và trích xuất tọa độ 17 điểm khung xương (skeleton keypoints) trên cơ thể một cách liên tục theo thời gian thực.
- Mạng nơ-ron hồi quy hai chiều (Bi-LSTM): Mô hình phân loại chuỗi thời gian được huấn luyện sẵn (lưu dưới dạng tệp .onnx). Nhận đầu vào là chuỗi tọa độ, vận tốc và góc của khung xương trong 60 khung hình liên tiếp để dự đoán hành vi ngã.
- ONNX Runtime (onnxruntime): Động cơ suy luận (inference engine) hiệu năng cao được sử dụng để chạy mô hình Bi-LSTM. Giúp tối ưu hóa tốc độ dự đoán trên CPU, đảm bảo hệ thống phản hồi theo thời gian thực.
- OpenCV (cv2): Thư viện thị giác máy tính hàng đầu, được ứng dụng để:
 - Đọc, thay đổi kích thước và giải mã/mã hóa luồng ảnh (base64).
 - Vẽ các đường nối khung xương người (skeleton tracking) và thêm chữ cảnh báo lên video.
 - Áp dụng bộ lọc làm mờ (cv2.GaussianBlur) để thực hiện tính năng bảo vệ quyền riêng tư.
- NumPy & Joblib: Xử lý các phép toán ma trận, tính toán vận tốc, góc nghiêng của cơ thể và chuẩn hóa dữ liệu (scaler.pkl) trước khi đưa vào mô hình AI.

4.3.3. Framework Backend

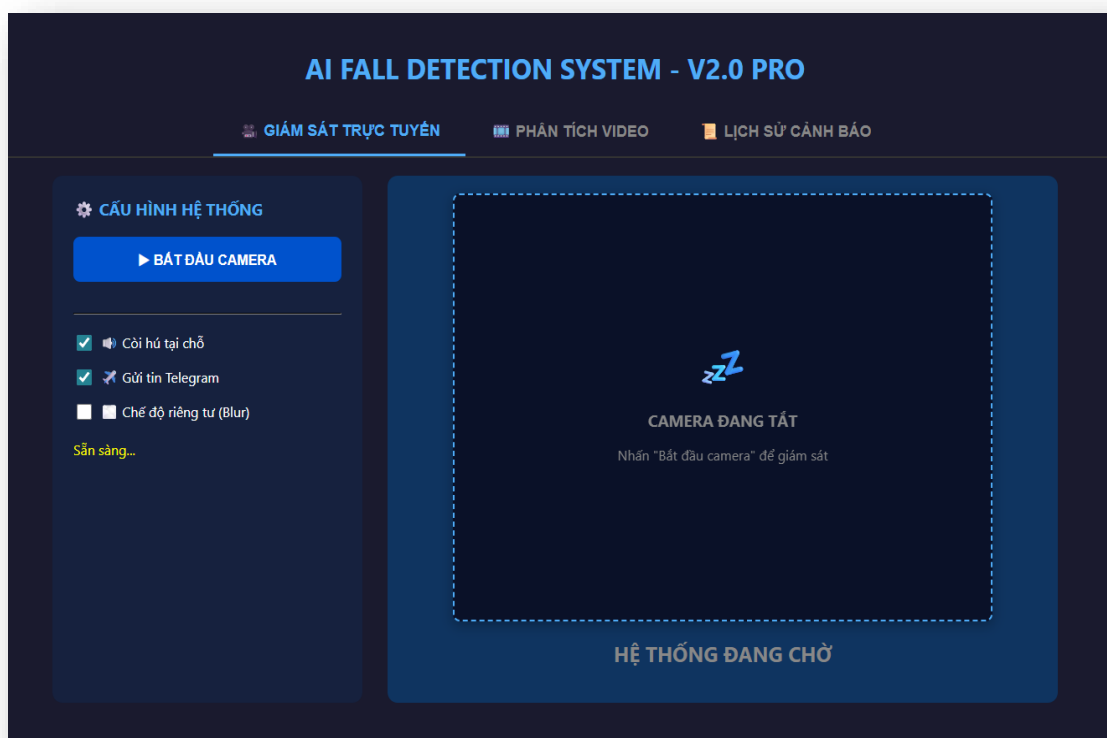
- FastAPI: Framework web hiện đại, hiệu năng cao của Python. Được chọn nhờ khả năng hỗ trợ lập trình bất đồng bộ (async/await) cực tốt, giúp định tuyến các API (RESTful) và quản lý WebSocket hiệu quả.
- Uvicorn: Web server ASGI tốc độ cao dùng để chạy ứng dụng FastAPI.
- Giao thức WebSocket: Đóng vai trò cốt lõi trong tính năng "Giám sát trực tuyến" và "Phân tích video". Cho phép thiết lập luồng giao tiếp hai chiều liên tục giữa Client và Server: Client gửi ảnh base64 liên tục, Server lập tức trả về ảnh đã xử lý AI và mức độ cảnh báo (prob) với độ trễ tính bằng mili-giây.
- ThreadPoolExecutor & Asyncio: Kỹ thuật xử lý đa luồng được áp dụng để đưa tác vụ phân tích AI nặng (YOLO + Bi-LSTM) chạy ngầm trên các luồng CPU riêng biệt. Nhờ đó, luồng chính của FastAPI không bị chặn (non-blocking), giữ cho kết nối video luôn mượt mà.

4.3.4. Cơ sở dữ liệu và mở rộng

- SQLite (sqlite3): Hệ quản trị cơ sở dữ liệu quan hệ gọn nhẹ, được tích hợp trực tiếp vào tệp hệ thống (fall_detection_logs.db) mà không cần cài đặt phần mềm bên thứ ba. Sử dụng để lưu trữ lịch sử các sự kiện ngã (thời gian, độ tin cậy, đường dẫn ảnh chứng cứ).
- Telegram Bot API: API tích hợp của bên thứ ba (requests.post). Khi AI xác nhận có hành vi ngã, hệ thống sẽ sử dụng API này để đẩy tin nhắn báo động khẩn cấp kèm theo hình ảnh hiện trường vào nhóm chat hoặc tài khoản Telegram của người giám sát.

4.4. Hướng dẫn sử dụng hệ thống

- Giao diện khi bắt đầu chạy:

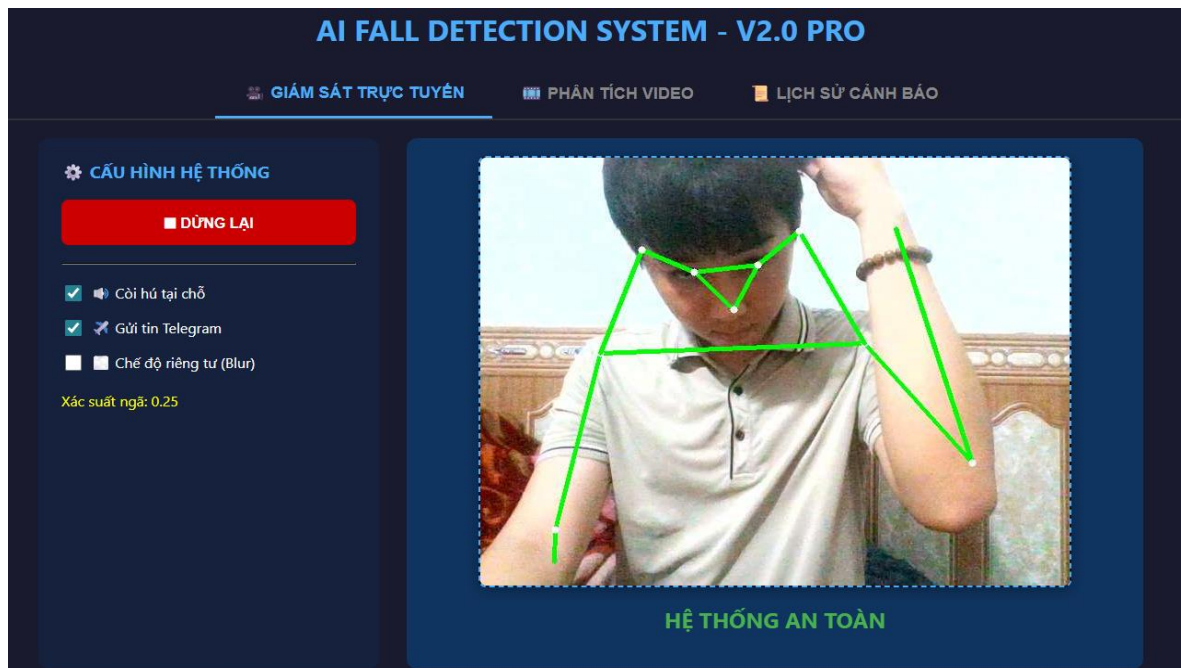


Hình 4.2 Giao diện giám sát trực tuyến

Hệ thống được thiết kế với giao diện trực quan, bao gồm hai khu vực chính: Bảng điều khiển cấu hình nằm bên trái và Màn hình hiển thị luồng camera ở bên phải. Để vận hành hệ thống, trước tiên người dùng sẽ tiến hành thiết lập các phương thức cảnh báo mong muốn tại mục "Cấu hình hệ thống". Tại đây, phần mềm cung cấp các tùy chọn linh hoạt như kích hoạt Còi hú tại chỗ để báo động cục bộ ngay lập tức, Gửi tin Telegram để thông báo từ xa cho người quản lý (kèm theo hình ảnh sự cố), và Chế độ riêng tư (Blur) giúp làm mờ hình dáng người nhằm bảo vệ quyền riêng tư mà không làm ảnh hưởng đến độ chính xác của thuật toán AI.

Sau khi hoàn tất việc lựa chọn cấu hình và hệ thống báo trạng thái "Sẵn sàng...", người dùng chỉ cần nhấn vào nút xanh "BẮT ĐẦU CAMERA". Ngay lập tức, khu vực màn hình chờ ở giữa sẽ kết nối và hiển thị luồng video trực tiếp từ camera, đồng thời dòng trạng thái phía dưới cùng sẽ chuyển từ "HỆ THỐNG ĐANG CHỜ" sang chế độ đang hoạt động. Kể từ lúc này, lõi AI sẽ liên tục phân tích hình ảnh theo thời gian thực. Bất cứ khi nào phát hiện hành vi té ngã, hệ thống sẽ tự động kích hoạt đồng thời các cảnh báo đã được người dùng thiết lập trước đó.

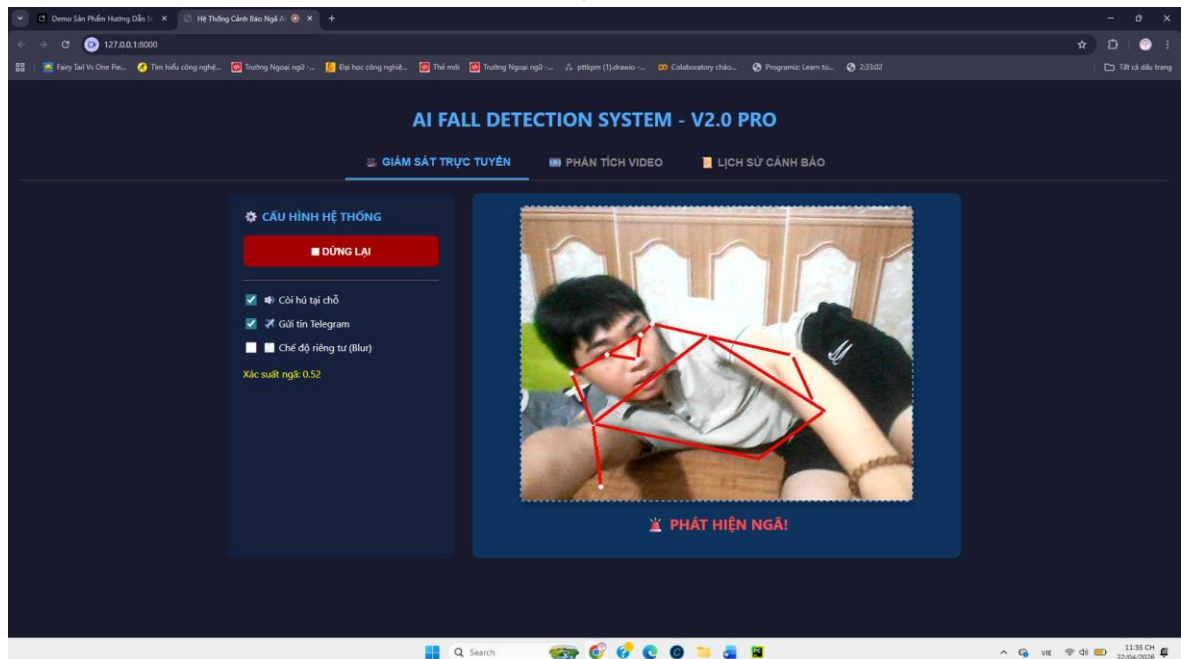
- Giao diện khi nhận diện khung xương



Hình 4.3 Giao diện nhận diện khung xương trạng thái bình thường

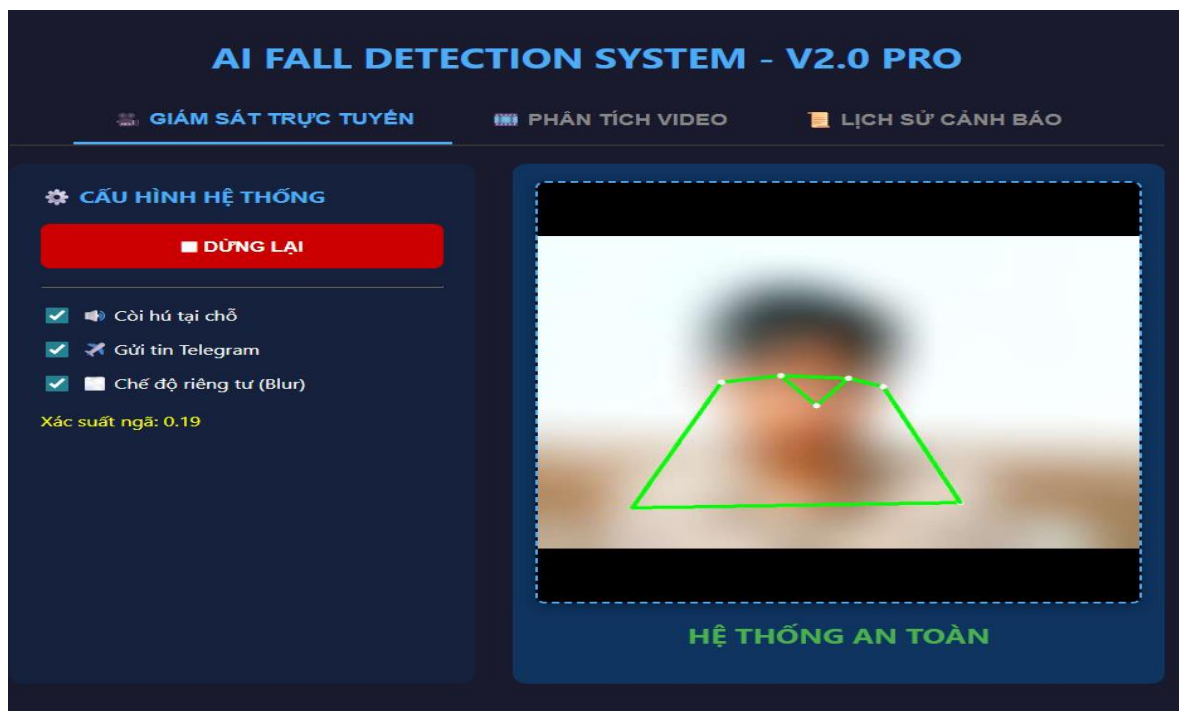
Ở trạng thái bình thường khung xương sẽ hiển thị là màu xanh và xác suất ngã dao động dưới 0.5 khi xác suất ngã tăng lên cao khung xương sẽ chuyển sang màu đỏ và thông báo té ngã, đồng thời sẽ có còi hú và tín hiệu cảnh báo bên telegram. Người dùng có thể tùy chỉnh bật/tắt các tính năng còi thông báo và tin nhắn cảnh báo.

- Giao diện khi nhận diện té ngã



Hình 4.4 Giao diện nhận diện khung xương ở trạng thái té ngã

- Giao diện khi nhận diện ở chế độ riêng tư



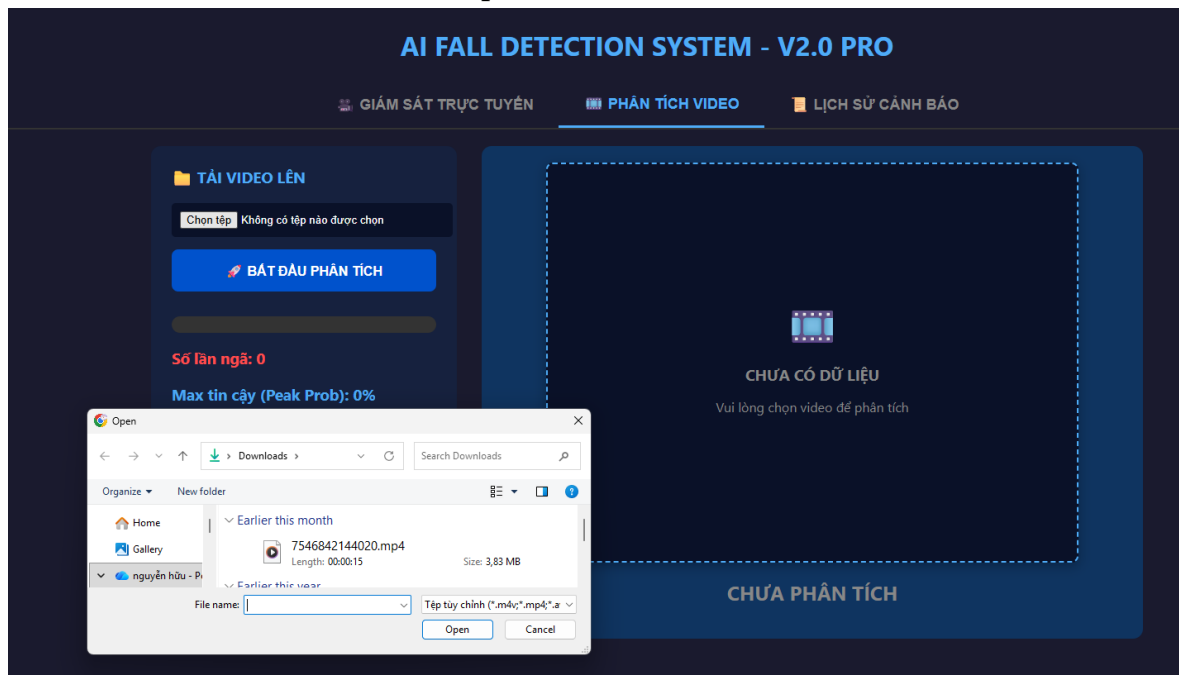
Hình 4.5 Giao diện nhận diện khung xương ở chế độ riêng tư

- Giao diện nhận tin nhắn cảnh báo Telegram



Hình 4.6 Cảnh báo tin nhắn té ngã telegram

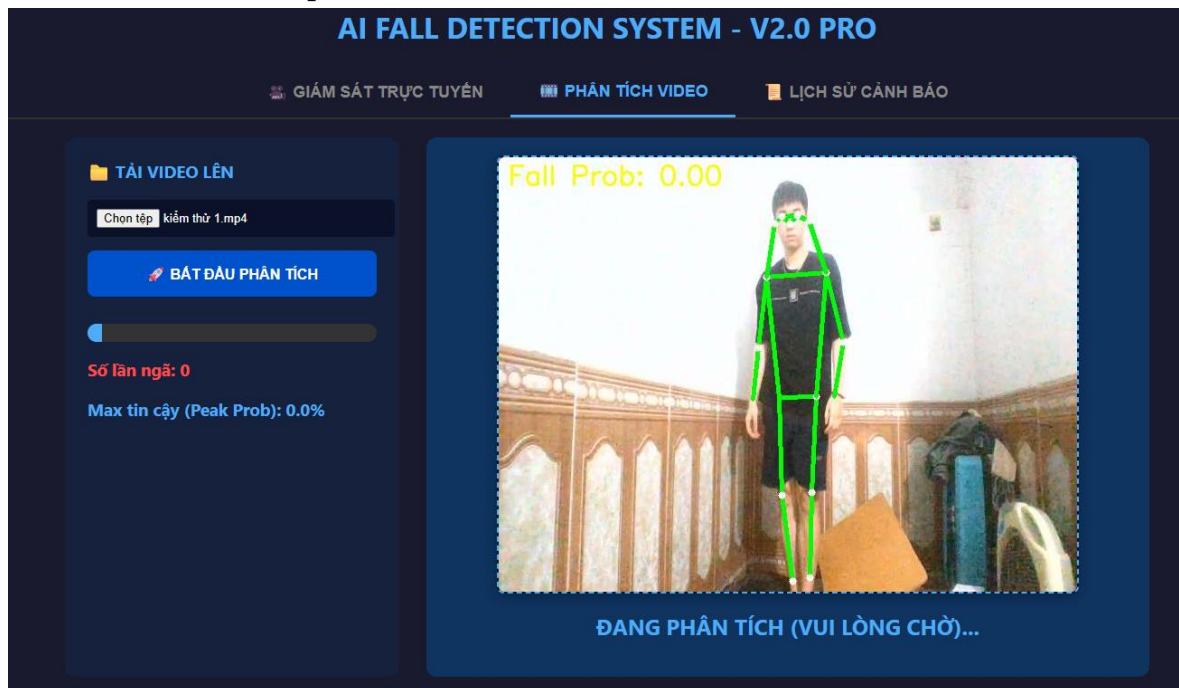
- Giao diện chọn video phân tích



Hình 4.7 Giao diện chọn video phân tích

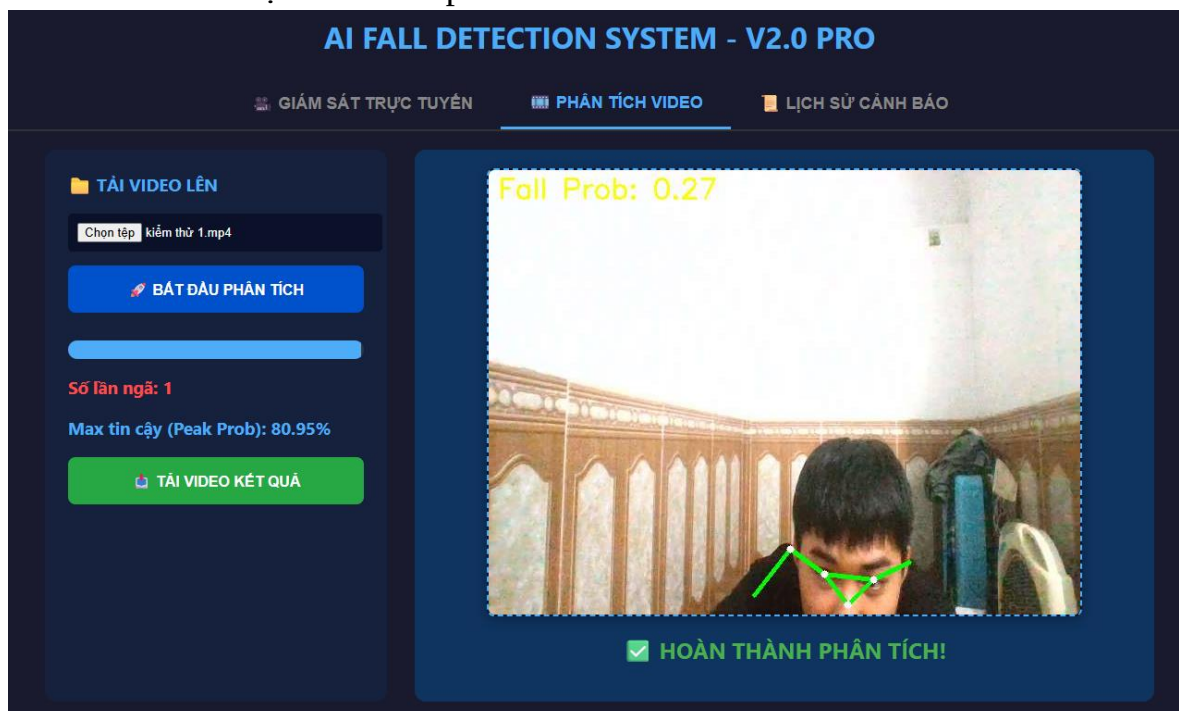
Để bắt đầu quá trình này, tại bảng điều khiển "Tải video lên" nằm ở phía bên trái màn hình, người dùng nhấn vào nút "Chọn tệp". Thao tác này sẽ lập tức mở ra cửa sổ duyệt tệp tin tiêu chuẩn của hệ điều hành. Từ đây, người dùng có thể điều hướng đến các thư mục trên máy tính (ví dụ như thư mục Downloads) để tìm kiếm và lựa chọn đoạn video cần kiểm tra. Hệ thống hỗ trợ xử lý tốt các định dạng video phổ biến hiện nay như MP4. Sau khi file video đã được chọn và tải lên hệ thống, người dùng chỉ cần nhấn vào nút "BẮT ĐẦU PHÂN TÍCH" màu xanh để kích hoạt lõi thuật toán AI chạy quét toàn bộ khung hình. Trong giai đoạn thiết lập ban đầu này, khi quá trình phân tích chưa chính thức diễn ra, khu vực màn hình chính sẽ ở trạng thái chờ với thông báo "CHƯA CÓ DỮ LIỆU", đồng thời các chỉ số thống kê bên trái như "Số lần ngã" và "Max tin cậy" đều hiển thị giá trị 0.

- Giao diện phân tích video



Hình 4.8 Giao diện phân tích video đã chọn

- Giao diện tải video phân tích



Hình 4.9 Giao diện tải video đã phân tích khung xương

Sau khi phân tích xong hệ thống sẽ hiện "TẢI VIDEO KẾT QUẢ" để người dùng tải video phân tích khung xương xuống

- Giao diện lịch sử cảnh báo

GIÁM SÁT TRỰC TUYẾN PHÂN TÍCH VIDEO LỊCH SỬ CẢNH BÁO			
LÀM MỚI DỮ LIỆU			
ID	Thời gian	Độ tin cậy	Hành động
#11	2026-04-22 22:13:12	68.1%	Xem Ảnh
#10	2026-04-22 22:12:26	72.9%	Xem Ảnh
#9	2026-04-11 19:27:31	69.2%	Xem Ảnh
#8	2026-04-10 21:29:33	72.9%	Xem Ảnh
#7	2026-04-10 21:28:17	75.3%	Xem Ảnh
#6	2026-04-10 21:27:41	72.7%	Xem Ảnh
#5	2026-04-10 18:32:58	70.3%	Xem Ảnh

Hình 4.10 Giao diện xem lịch sử cảnh báo

Chuyển sang tab "Lịch sử cảnh báo", hệ thống cung cấp một không gian quản lý lưu trữ toàn diện, hoạt động như một quyển nhật ký điện tử ghi nhận mọi sự kiện té ngã đã được hệ thống phát hiện trước đó. Tại giao diện này, toàn bộ dữ liệu cảnh báo được trình bày một cách trực quan và khoa học dưới dạng một bảng thống kê chi tiết. Người dùng có thể sử dụng nút "LÀM MỚI DỮ LIỆU" màu xanh ở góc trên bên trái để lập tức đồng bộ và cập nhật các bản ghi sự cố mới nhất vừa được hệ thống ghi nhận. Bảng dữ liệu được tổ chức theo cấu trúc rõ ràng, sắp xếp các sự kiện theo thứ tự từ mới nhất đến cũ nhất thông qua mã số "ID".

Từng dòng bản ghi sẽ cung cấp các thông tin minh bạch cho người quản trị, bao gồm mốc "Thời gian" xảy ra sự cố chính xác đến từng giây và tỷ lệ "Độ tin cậy" của cảnh báo đó (được hiển thị nổi bật bằng màu đỏ). Đặc biệt, nhằm hỗ trợ tối đa cho quá trình rà soát và xác minh sự cố, hệ thống tích hợp sẵn nút "Xem Ảnh" tại cột "Hành động" tương ứng với mỗi sự kiện. Khi nhấn vào nút này, người quản lý có thể truy xuất ngay lập tức hình ảnh bằng chứng tĩnh được camera chụp lại tại đúng khoảnh khắc thuật toán phát hiện người ngã, từ đó có cơ sở để đánh giá chính xác tình hình đã diễn ra.

KẾT LUẬN

Những kết quả đạt được:

Về công nghệ:

Hiểu và áp dụng thành công kiến thức về học sâu, thị giác máy tính và xử lý chuỗi thời gian trong việc xây dựng hệ thống phát hiện té ngã từ video.

Xây dựng pipeline xử lý dữ liệu hoàn chỉnh: từ video → trích xuất keypoints (YOLO-Pose) → chuẩn hóa dữ liệu → huấn luyện mô hình BiLSTM.

Kết hợp hiệu quả hai mô hình YOLO-Pose và BiLSTM (học đặc trưng thời gian) để nhận diện hành vi té ngã. Tối ưu hóa quá trình huấn luyện với GPU, tf.data pipeline, và các kỹ thuật chống overfitting như Dropout, EarlyStopping.

Xây dựng hệ thống có khả năng phát hiện té ngã theo thời gian thực, phát cảnh báo đến người thân và trực quan hóa kết quả.

Hướng phát triển:

Mở rộng và đa dạng hóa tập dữ liệu (góc quay, ánh sáng, che khuất) để tăng độ chính xác trong môi trường thực tế.

Tối ưu hệ thống để triển khai trên thiết bị thực (Edge AI) như camera thông minh.

Phát triển thêm các chức năng cảnh báo như âm thanh, gửi thông báo đến người thân hoặc nhân viên y tế.

Cải thiện giao diện người dùng, hướng tới tích hợp trong hệ thống nhà thông minh.

TÀI LIỆU THAM KHẢO

- [1] “You Only Look Once: Unified, Real-Time Object Detection”, Redmon J. et al., IEEE CVPR, 2016.
- [2] “Fall Detection Based on Skeleton Keypoints and BiLSTM”, Zhang Y. et al., IEEE Access, 2021.
- [3] “Computer Vision: Algorithms and Applications”, Szeliski R., Springer, 2022.
- [4] “Vision-based Fall Detection using ST-GCN and Skeleton Data”, Keskes H. et al., Multimedia Tools and Applications, 2021.
- [5] “A Fall Detection System using Skeleton Information from Kinect v2”, Galvão Y. C. et al., IEEE International Conference on Systems Man and Cybernetics (SMC), 2017.
- [6] “A Fall Detection Method Based on Recurrent Neural Networks”, Theodoridis T. et al., ACM International Conference on Pervasive Technologies Related to Assistive Environments (PETRA), 2017.
- [7] “YOLOv26-Pose Estimation Tutorial”, LearnOpenCV, Sudip Chakrabarty, 2026.
- [8] “Tìm hiểu LSTM - Bí quyết giữ thông tin lâu dài hiệu quả”, Viblo, Long Nguyễn Thành, 2025.
- [9] “Recurrent Neural Network (Phần 1): Tổng quan và ứng dụng”, Viblo, Do Duong , 2018.
- [10] “Real-Time Human Fall Detection using Pose Estimation and Bi-directional LSTM”, Sharma A. et al., International Conference on Computer Vision and Image Processing, 2022.
- [11] “Bidirectional Recurrent Neural Networks”, Schuster M., Paliwal K.K., IEEE Transactions on Signal Processing, 1997.
- [12] “A Review on Vision-Based Fall Detection Systems”, Wang J. et al., Expert Systems with Applications, 2020.